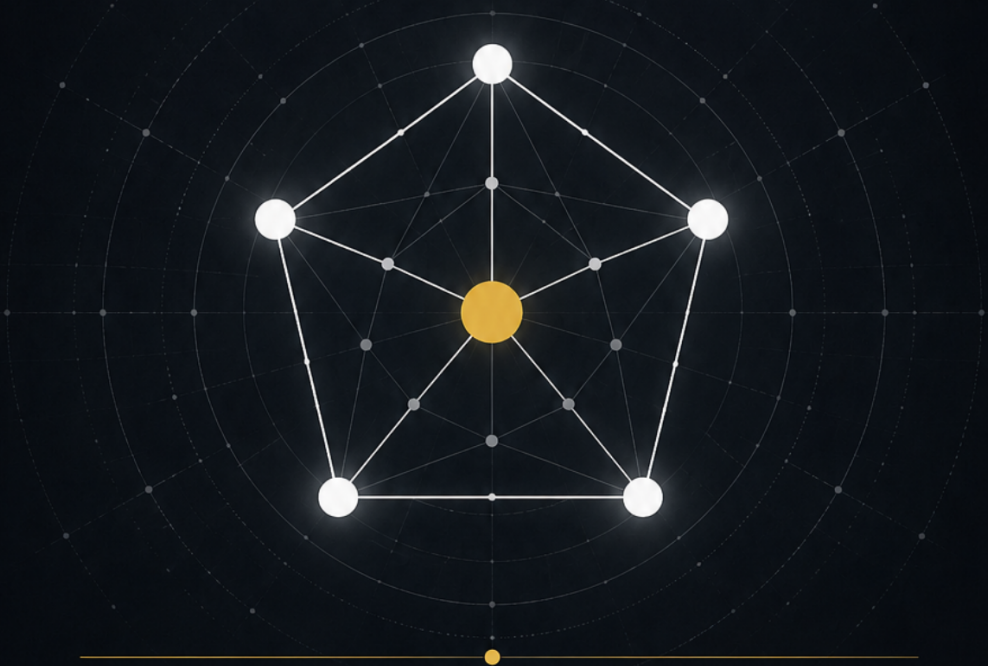
CÓDIGO SINTÉTICO

El arte de orquestar agentes de IA: arquitectura, economía y criterio de ingeniería en la era multiagente





Una guía integral para diseñar, coordinar y escalar sistemas multiagente con inteligencia, eficiencia y propósito.

SERGIO PEREZ RUIZ

Índice

Página legal 1

Dedicatoria 3

Sobre el autor 4

El camino a este libro 5

Obra pública destacada del autor 6
Perfiles y recursos adicionales 6

Agradecimientos 7

Créditos 8
Herramientas de composición y compilación 8
Entornos de edición 8
Verificación técnica de código y ejemplos 9
Diseño y diagramas de arquitectura 9
Herramientas de validación editorial 9
Recursos y referencias técnicas 9
Organización del libro 10
Convenciones tipográficas 10
Sobre los ejemplos de código 11

Prefacio: Delegar no es diseñar 12
¿Por qué este libro? 13
A quién está dirigido 13
Lo que este libro es 13

Manifiesto 15
Antes 15
Ahora 15
El puente de la Ingeniería de Software 15

1 El despertar agéntico 17
El desarrollador actual 17

Qué es, formalmente, un agente 18
Reactivo y proactivo: las dos mitades de la agencia 18
La proactividad ambiental 19
Marco de evaluación de riesgo y router 20
Marco de niveles de autonomía de sistemas IA 21
Ejercicio para clasificar tus sistemas 25
Lo esencial del capítulo 1 26

2 El AI Engineer 28
Un rol emerge 28
La matriz de diferenciación 28
Las siete competencias 29
El camino de conversión: de developer a AI Engineer 34
Ejercicio resuelto: diseñe el programa de conversión 35
Lo esencial del capítulo 2 35

3 Platform Engineering en la era agéntica 37
La plataforma como producto interno 37
El AI Gateway: componente crítico 37
Plantillas de equipo 40
Matriz de arquitectura según el tamaño del proyecto 41
El costo cognitivo de la deuda que no ves en la factura 44
Ejercicio resuelto que añade corte por proyecto 47
Arquitectura asíncrona como sistema de eventos 47
Límite de tokens en enjambres de agentes 51
Lo esencial del capítulo 3 56
Puente al Capítulo 10: cuándo migrar a Kafka 56

4 Patrones fundamentales 59
De la cadena de texto al sistema cognitivo 59
El ciclo ReAct que atiende el razonamiento que actúa 61
Tree of Thoughts (ToT) con bifurcación y poda de hipótesis . . 63
El coste de la exploración heurística 63
Reflexion es la autocrítica evaluada 66
La necesidad de un evaluador objetivo 67
Comparativa de patrones fundamentales 67
La evolución hacia sistemas de ingeniería 68
Lo esencial del capítulo 4 72

5 El arnés del agente 74
El código que el proveedor no te da 74
Componentes del arnés 74
Trazabilidad y registro estructurado 76

Lo esencial del capítulo 5 80

6 La variable de confianza 81
La fórmula que faltaba 81
La ecuación reformulada 81
La variable de verificabilidad 83
Patrón Shadow Agent 85
Lo esencial del capítulo 6 86

7 RACI en enjambres 87
La paradoja de la claridad 87
La regla cardinal 87
Eisenhower aplicado para delegar a un agente 88
Ejercicio resuelto de ajuste dinámico 91
Lo esencial del capítulo 7 92

8 SOLID para agentes 93
Por qué SOLID es gestión de complejidad 93
Los 5 principios reescritos 94
Refactoring de enjambre en 4 pasos con checklist real 95
Lo esencial del capítulo 8 97

9 Invariantes de producción 98
Los 5 invariantes que todo sistema debe satisfacer 98
La verificabilidad es igual a la nueva confianza 98
El costo real es igual a la deuda de verificación 99
La autonomía se gana y no se asigna 99
El contexto es capital y no es gratis 100
La arquitectura domina al modelo 101
Caso de estudio de un despliegue que falló 102
Lo esencial del capítulo 9 103

10 DDD + Kafka como sistema nervioso 104
Bounded contexts para agentes 104
Lo esencial del capítulo 10 106

11 Gobernanza de deuda técnica agéntica 107
Por qué la deuda agéntica se acumula más rápido 107
Métricas automáticas para detectar deuda 107
Revisiones de arquitectura que incluyen output de agentes . . . 108
CI/CD para bloquear deuda agéntica 109
Caso de estudio de un deploy sin validación 112
Lo esencial del capítulo 11 113

12 Revisión de código con agentes **114**
Matriz RACI para PR review 114
El agente observa bugs que pasamos por alto 114
Caso de estudio de reducción de bugs con agentes 115
Patrones para incorporar personas en el loop 116
Ejercicio resuelto que integra review agent en CI/CD 116
Lo esencial del capítulo 12 117

13 Triaje de producción y debugging autónomo **119**
Cuándo permitir que un agente depure en vivo 119
El playbook 120
Caso de estudio de un incidente resuelto por un agente 120
El agente falla sin un contexto adecuado 122
Lo esencial del capítulo 13 123

14 Generación de documentación y conocimiento **124**
Por qué la documentación generada es mejor que la manual . . 124
El riesgo de documentación que parece real y no lo es 124
Documentación de decisiones arquitectónicas **ADRs** 125
Lo esencial del capítulo 14 126

15 Atención al cliente y soporte técnico **127**
Triaje conversacional con RAG 127
Caso de estudio de tickets resueltos y reducción de costos . . . 129
Lo esencial del capítulo 15 130

16 Decisiones arquitectónicas que no puedes automatizar **131**
Qué decisiones requieren criterio humano 131
Patrón: Human-in-the-loop para decisiones irreversibles 132
Ejercicio resuelto: crea un gate para la base de datos 134
Lo esencial del capítulo 16 134

17 Seguridad en sistemas multiagente **136**
La superficie de ataque se multiplica 136
Modelo de amenazas para enjambres 136
MCP como control de acceso y no solo protocolo 139
Caso de estudio del agente mal configurado 140
Lo esencial del capítulo 17 142

18 Optimización de costos real **143**
Lo que no verás en los cálculos de precio 143
Cálculo de TCO 144
Estrategias que funcionan 145
Caso de estudio de un equipo que redujo la factura 146

Lo esencial del capítulo 18 147

19 La geopolítica como restricción arquitectónica 148
De la neutralidad técnica a la soberanía digital 148
Desglose de la arquitectura de tres capas bajo restricciones geopolíticas 149
Matriz de trazabilidad: Componentes arquitectónicos vs. Dimensiones geopolíticas 151
El riesgo de la cadena de suministro 152
Tabla comparativa: USA, China, EU, LATAM 153
Diseñar para la sustituibilidad **Backend Agnóstico** 153
Caso de estudio del equipo atado a un proveedor 155
Lo esencial del capítulo 19 157

20 El proyecto y sus patrones 159
Cuándo un agente es innecesario y un script basta 159
Cuándo un workflow supera a un sistema multiagente 160
Cuándo la arquitectura cuesta más de lo que ahorra 160
El costo de supervisión humana y gobierno organizacional . . . 161
Los límites del backend agnóstico 162
La evaluación cuantitativa de modelos 165
Marco de decisión basado en evidencia 167
Escalar la caché semántica sin destruir la latencia 168
Cómo evitar la sobreingeniería en proyectos de IA 168
Matriz de decisión 169
Lo esencial del capítulo 20 170

21 De código a criterio 171
Lo que permanece 171
El AI Engineer como garante y no como ejecutor 171
Los 10 principios finales del libro 172
Epílogo hacia la próxima generación 173

Repositorio de referencia 174

Tesis académicas de referencia 177

Referencias 179

Obras y publicaciones del autor 184
Artículos técnicos y ensayos 184
Publicaciones técnicas en LinkedIn 184
Proyectos y comunidades 185

Página legal

Código sintético *El arte de orquestar agentes de IA: arquitectura, economía y criterio de ingeniería en la era multiagente*

© 2026 Sergio Perez Ruiz. Todos los derechos reservados.

Ficha Técnica Editorial:

- **Autor-Editor (Persona Natural):** Sergio Perez Ruiz
- **Documento de identidad:** DNI N° 46581382 — Titular legal de la obra y de los derechos de publicación
- **Dirección del autor-editor:** Jr. Juan Guillermo Moore 357, Santa Anita, Lima, Perú
- **Contacto y licencias:** sergio@specsolid.com
- **Lugar de edición:** Lima, Perú
- **Fecha de publicación:** Primera edición, agosto de 2026

Registro de Propiedad Intelectual e Identificación:

- **ISBN (Edición digital / EPUB):** 978-612-03-2914-6
- **ISBN (Edición digital / PDF):** 978-612-03-2931-3
- **Hecho el Depósito Legal en la Biblioteca Nacional del Perú Nº:** 2026-09631

Formato y distribución:

Esta obra se publica exclusivamente en formato digital (EPUB y PDF) mediante impresión bajo demanda a través de las siguientes plataformas autorizadas:

- **Amazon Kindle Direct Publishing (KDP)** — distribución global en formato Kindle / EPUB
- **Google Play Books** — distribución en la plataforma de Google
- **Lulu** — edición digital y opción de impresión bajo demanda a pedido del lector

- Otras plataformas de distribución digital autorizadas por el autor-editor

No existe tiraje físico definido de fábrica. La impresión física, de estar disponible, se realiza exclusivamente bajo pedido (*print-on-demand*) a través de las plataformas mencionadas, sin stock físico mantenido por el autor-editor.

Ninguna parte de esta publicación puede ser reproducida, almacenada en un sistema de recuperación de información ni transmitida en cualquier forma o por cualquier medio (electrónico, mecánico, fotocopiado, grabación u otro) sin el permiso previo y por escrito del autor-editor, salvo en los casos de citas breves incorporadas en reseñas críticas y ciertos usos no comerciales permitidos por la Ley sobre Derecho de Autor (Decreto Legislativo Nº 822 y Ley Nº 31250 del Perú).

Las marcas comerciales mencionadas en este libro (Claude, Anthropic, GPT, OpenAI, Gemini, Google, Python, GitHub, entre otras) son propiedad de sus respectivos titulares. Su mención es únicamente con fines educativos e informativos y no implica afiliación ni patrocinio.

Aviso de responsabilidad: La información contenida en este libro se ofrece «tal cual». Aunque el autor ha tomado todas las precauciones razonables en su preparación (incluida la verificación de cada referencia bibliográfica y la validación técnica de los ejemplos de código), ni el autor ni el editor asumen responsabilidad alguna por errores u omisiones, ni por daños resultantes del uso de la información aquí contenida. Los precios de las APIs, identificadores de modelos, *rankings* comparativos y capacidades técnicas corresponden a un **snapshot fechado en agosto de 2026** y cambian con frecuencia: cada tabla y cifra de este tipo se presenta junto con su fórmula de origen para que pueda recalcularse con precios vigentes. Verifique siempre la documentación oficial del proveedor, o el catálogo vivo de modelos y precios del repositorio complementario (`github.com/sergioide007/synthetic-code`, módulo `src/sintetico/model_registry.py`, sobreescribible por variables de entorno), antes de tomar decisiones de arquitectura o presupuesto.

Dedicatoria

A mi madre, por su amor incondicional y por ser el faro que siempre guía mi camino.

A mis amados hijos, que son mi mayor motivo para dar siempre lo mejor de mí y la razón de cada logro.

A mi hermano Alexander, por estar a mi lado aquí en Lima compartiendo el día a día, y a mis hermanas Andrea y Begoña, que desde la distancia en España siguen presentes en cada paso que doy.

A mis abuelos: de parte de mi madre, a mi abuelita María, que aún nos bendice con su vida, y a la memoria de mi abuelito Eduardo; de parte de mi padre, a mi abuelita Nena y a la memoria de mi abuelito Javier, cuya luz y enseñanzas perduran.

A mis tíos y primos, por su apoyo constante y por ser la gran familia que siempre sostiene mis sueños.

A los ingenieros que entendieron a tiempo que su trabajo nunca fue escribir código, sino resolver problemas. Y a los que están a punto de entenderlo.

> «No existe un único desarrollo, ni en tecnología ni en técnicas de gestión, que por sí solo prometa una mejora de un orden de magnitud en productividad, fiabilidad o simplicidad en el plazo de una década.», (Brooks 1987)

Cuarenta años después, los sistemas multiagente de IA nos obligan a reexaminar la advertencia. La bala de plata sigue sin existir, pero por primera vez tenemos una fundición. Este libro trata sobre cómo operarla sin quemarse. La disciplina, y no el entusiasmo, es la que convierte una tecnología poderosa en una tecnología útil.

Sobre el autor

Sergio Perez Ruiz es ingeniero de sistemas y especialista en sistemas de inteligencia artificial aplicada. Su trabajo se centra en el diseño de plataformas agénticas de producción, la economía de los sistemas basados en modelos de lenguaje y la formación de equipos de ingeniería para la era de la orquestación. Este libro condensa esa experiencia práctica con el rigor de la literatura académica revisada por pares, conectando principios clásicos de ingeniería de software con los nuevos desafíos introducidos por los sistemas basados en inteligencia artificial.

El camino a este libro

Este libro no nació de un día para otro. Es la convergencia de años de trabajo público alrededor de una convicción: *el software de calidad no se improvisa, se diseña; y en la era de la inteligencia artificial, los principios arquitectónicos importan más que nunca.* Esa convicción tomó forma en **SpecSolid**, una iniciativa académica, personal y de código abierto que une la ingeniería de software clásica, principios SOLID, arquitectura evolutiva y desarrollo guiado por especificaciones, con el potencial de los agentes de inteligencia artificial. El propósito de SpecSolid es democratizar el acceso a buenas prácticas de ingeniería para que estudiantes y profesionales de cualquier lugar puedan elevar el nivel de construcción de software. Varios capítulos de esta obra evolucionan a partir de casos, investigaciones y guías compartidas previamente con la comunidad técnica. Entre ellos:

- El caso de arquitectura de un sincronizador de datos dio origen al desarrollo presentado en el capítulo 10 (DDD + Kafka como sistema nervioso).
- La guía de migración de Java Spring Boot a Node.js asistida por inteligencia artificial evolucionó hacia los conceptos de "el modelo es un detalle" (Clean Architecture para agentes de IA); el desarrollo completo vive en el repositorio `llm-port-java-node` y se desarrollará en profundidad en el Volumen 1 de esta serie.
- La práctica de analizar y refutar argumentos mediante evidencia técnica se convirtió en la base conceptual del patrón **Council**, referenciado en el capítulo 8 y en el catálogo de patrones del capítulo 20 de esta edición; la implementación ejecutable completa vive en el repositorio `architecture-council` y se amplía en el Volumen 1.

Obra pública destacada del autor

Las siguientes publicaciones representan algunos de los trabajos que anteceden y complementan esta obra:

- *SpecSolid: A Philosophy, a Framework, and a Community, Built on the Shoulders of Giants.* Medium.
- *The Complete DevOps Course: Free, Community-Driven and Built for Real Engineers.* Medium.
- *The Model Is a Detail: Clean Architecture for AI Agents.* Medium.
- *The Silent Token Burn and the CIU Metric: The Debt That Doesn't Show on the Balance Sheet.* Medium.
- *El Código Limpio Sintético: los principios que perduran.* LinkedIn.

Perfiles y recursos adicionales

- LinkedIn: sergioide007
- Medium: @sergioide007
- GitHub: sergioide007

Agradecimientos

Este libro es, en buena parte, una deuda reconocida. A los autores cuyo trabajo llena las referencias: Robert C. Martin y Martin Fowler por décadas de principios de diseño; Gamma, Helm, Johnson y Vlissides por veintitrés patrones que ningún LLM ha dejado obsoletos; Stuart Russell y Peter Norvig por una definición de agente que aguantó tres décadas; Dario Amodei y colegas por catalizar los problemas de alineación antes de que se volvieran urgentes. A la comunidad de **SpecSolid** por demostrar que democratizar el conocimiento técnico es trabajo, no un eslogan. A los revisores técnicos que señalan errores antes de que lleguen al lector: su crítica vale más que cualquier elogio. A ti, lector: este libro fue escrito con la apuesta de que el criterio de ingeniería, más que cualquier herramienta específica, es la habilidad más duradera en esta era.

Créditos

Este libro fue producido utilizando herramientas de composición digital, desarrollo de software, validación técnica y diseño de arquitectura.

Herramientas de composición y compilación

- **Pandoc 3.9**, para la conversión del manuscrito en Markdown hacia formatos EPUB3 y PDF mediante flujos reproducibles de publicación técnica.
- **XeLaTeX**, para la composición tipográfica profesional y generación del formato PDF con control avanzado de fuentes, código y elementos editoriales.
- **Git**, para el control de versiones del manuscrito y seguimiento histórico de modificaciones.

Entornos de edición

- **Visual Studio Code**, utilizado como entorno principal de desarrollo y edición del manuscrito, incluyendo archivos Markdown, código fuente, documentación técnica y configuraciones del proceso de publicación.
- **Google Docs**, empleado de forma temporal durante una etapa intermedia del proyecto para evaluar un flujo de trabajo centrado en la edición visual y la revisión del manuscrito, antes de regresar al entorno principal basado en Markdown.
- **Antigravity**, utilizado como entorno de desarrollo asistido por inteligencia artificial para la edición de código fuente, documentación técnica y archivos de configuración del proyecto.

Verificación técnica de código y ejemplos

- **Python 3.10+**, para la validación de sintaxis, ejecución de ejemplos y comprobación de fragmentos de código.
- **Claude Code**, para la asistencia en revisión técnica, análisis de código y evaluación de consistencia en ejemplos y explicaciones.
- **Google Gemini**, para la asistencia complementaria en revisión técnica y análisis conceptual.
- **GitHub**, para la gestión de repositorios, publicación de material técnico y colaboración abierta.

Diseño y diagramas de arquitectura

- **Excalidraw y Graphviz ambos online**, para la generación de diagramas de flujos técnicos.
- **PlantUML**, para la generación de diagramas técnicos y modelos de arquitectura utilizando notación UML.
- **Magick**, para la edición de gráficos vectoriales y preparación de elementos visuales.

Herramientas de validación editorial

- **epubcheck**, para la validación de estructura y conformidad de archivos EPUB3.
- **Calibre**, para la revisión de conversión, metadatos y visualización de libros electrónicos.

Recursos y referencias técnicas

- O'Reilly Media, como referencia de prácticas editoriales para documentación y libros técnicos.
- W3C, como referencia para estándares abiertos relacionados con publicación digital y tecnologías web.

Organización del libro

- **Parte I, El nuevo paradigma** (capítulos 1 a 3): qué cambió, por qué y qué perfiles exige.
- **Parte II, Arquitectura sintética** (capítulos 4 a 10): fundamentos técnicos, desde ReAct y Tree of Thoughts hasta el arnés del agente, la confianza calibrada, RACI, SOLID para agentes, invariantes de producción y DDD con Kafka como sistema nervioso.
- **Parte III, Gobierno del código agéntico** (capítulos 11 y 12): gobernanza de la deuda técnica agéntica y revisión de código asistida por agentes.
- **Parte IV, Producción** (capítulos 13 a 16): triaje y debugging autónomo, generación de documentación, atención al cliente y las decisiones arquitectónicas que exigen criterio humano.
- **Parte V, El stack moderno y sus patrones** (capítulos 17 a 20): seguridad en sistemas multiagente (incluido el Model Context Protocol como control de acceso), optimización de costos, geopolítica como restricción arquitectónica y el catálogo de patrones del proyecto.
- **Parte VI, Horizonte** (capítulo 21): ética, regulación y el futuro del ingeniero.
- **Apéndices**: referencias de APIs, calculadora de costos, glosario y laboratorio.

Convenciones tipográficas

- `Monoespaciado` indica funciones, parámetros, modelos, rutas y código.
- El código está en Python 3.10+, salvo indicación contraria.
- Las citas siguen APA (7.ª edición). Cada afirmación empírica está anclada a una fuente verificable.
- Cuando un dato es una estimación del autor o un escenario ilustrativo, se indica explícitamente.
- **Nota** señala una recomendación práctica.

- **Advertencia** señala un riesgo de implementación.
- Un **caso real** proviene de experiencia de producción, anonimizada cuando corresponde.
- Un **concepto propuesto por el autor** se identifica explícitamente como tal, no es un estándar consolidado de la industria (ver "Nota sobre terminología propuesta").

Sobre los ejemplos de código

Los ejemplos son **replicables** con API key de Anthropic, OpenAI o Google y Python estándar. Siguiendo la convención editorial estándar para libros técnicos, los fragmentos priorizan la idea sobre la exhaustividad: donde el detalle omitido es *boilerplate* (manejo repetitivo de errores, getters, configuración), se marca con `...` y se referencia el símbolo completo. El código íntegro, ejecutable y con tests, vive en el repositorio complementario del libro, no en cada listado. Los identificadores (`claude-opus-4-8`, `claude-sonnet-4-6`, `claude-haiku-4-5`) y precios corresponden a un **snapshot fechado en agosto de 2026**, no a una referencia vigente: cada cifra en dólares se acompaña de su fórmula de origen para poder recalcularse con precios actuales.

Prefacio: Delegar no es diseñar

> *La ingeniería de software no desaparece con los agentes. Evoluciona.* — del manifiesto de esta obra.

Hacia 2026, el software atravesó una transformación tectónica. La IA dejó de ser una herramienta para «copiar y pegar código con mejor sintaxis» y se convirtió en un sistema capaz de razonar, descomponer, delegar y autoevaluarse. La literatura técnica le dio un nombre: *agencia* (Wooldridge 2009; Anthropic 2024). La evidencia llegó antes que los marcos: un experimento con GitHub Copilot encontró que los desarrolladores asistidos completaban tareas un 55,8 % más rápido (Peng et al. 2023), y estudios posteriores confirmaron mejoras sostenidas (Ziegler y Donkers 2024).

Pero esos estudios medían *asistentes*. Los sistemas *agénticos* —donde la IA ejecuta ciclos completos de percepción, razonamiento y acción sin supervisión constante— multiplican promesa y riesgo por igual (Kapoor et al. 2024). Y ahí es donde casi todos los equipos que conozco tropiezan de la misma forma: confunden velocidad de generación con capacidad de entrega.

Este libro no celebra esa velocidad. Advierte sobre sus consecuencias cuando no hay un marco de diseño detrás. Cuando delegamos el desarrollo a inteligencias artificiales sin arquitectura ni métricas claras, ocurren tres fenómenos, siempre en este orden:

- **El *Token-Burn* silencioso.** El costo crece superlinealmente mientras la calidad se estanca. Cada llamada, vista de forma aislada, parece trivial. El problema nunca se ve línea por línea: emerge a escala, en la factura del mes.
- **La deuda técnica acelerada.** Un agente que ignora el contexto del negocio genera lógica inválida a velocidad industrial. La deuda que un equipo humano acumulaba en un año, un sistema agéntico mal gobernado la produce en una semana (Sculley et al. 2015).
- **La ilusión de velocidad.** Más código en menos tiempo, pero código que no supera pruebas ni revisión. Las «líneas generadas» suben en el dashboard. Las funcionalidades que realmente llegan a

producción, no. Los tres fenómenos comparten una misma raíz: se trató la orquestación de agentes como un problema de *prompting*, cuando en realidad es un problema de ingeniería de sistemas.

¿Por qué este libro?

La solución no pasa por ralentizar, sino por *orquestar*: sistemas donde los agentes se revisan entre sí, debaten, se alinean con el negocio y rinden cuentas de cada decisión que toman de forma autónoma. La evidencia respalda el enfoque: los sistemas multiagente producen respuestas más factuales que un único modelo trabajando solo (Du et al. 2023), y superan de forma consistente a los agentes monolíticos en tareas complejas (Hong et al. 2024; Qian et al. 2024).

Ese es el arte que este libro enseña: no *si* delegar en agentes, sino *cómo* construir la arquitectura, la economía y el gobierno que hacen que delegar sea seguro, medible y rentable. Por eso el libro no termina en un capítulo de arquitectura ni en uno de costos. Termina en un capítulo sobre criterio —el juicio de ingeniería que decide, en cada nivel de autonomía, cuánto delegar y cuánto retener. Las herramientas cambiarán; ese criterio es lo único que se queda contigo de un modelo al siguiente.

A quién está dirigido

Al **ingeniero senior** que domina la ingeniería clásica y quiere aprender las primitivas agénticas sin abandonar lo que ya sabe que funciona. Al **arquitecto** que debe construir una plataforma interna para que cincuenta equipos usen IA sin quemar el presupuesto ni el uptime. Al **directivo técnico** que necesita separar el ruido del marketing de la evidencia operativa, y tomar decisiones de inversión con datos, no con entusiasmo.

Lo que este libro es

Es un manual de ingeniería, donde descubrimos que la disciplina convierte a una tecnología poderosa en una tecnología útil. El modelo importa, pero aún más importante es el criterio, pues te convierte en alguien capaz de operar esta tecnología en producción.

Un mensaje para quien diseña la plataforma

Es muy probable que hoy ya estés intentando construir un arnés o una arquitectura técnica con todo lo necesario para tu caso de uso. Incluso es posible que, al incorporar los conceptos y técnicas de estas páginas, desarrolles soluciones aún mejores y más adaptadas a tu contexto particular.

La premisa es simple: lo que hoy construyes con criterio técnico perdura y se convierte en una base sólida. Esto aplica tanto para el arquitecto individual como para los equipos de ingeniería de software que necesitan fijar sus estándares sobre un terreno firme, auditable y escalable.

Sergio Perez Ruiz, 2026

Manifiesto

Los agentes desarrollan código muy rápido, muy bueno y eficaz; incluso te dicen dónde está el error y ejecutan las pruebas.

Antes

Cuando iniciaba en el mundo de la programación y me adentraba en el desarrollo de sistemas de información, hace más de una década, algo me llamaba la atención: la complejidad con la que desarrollábamos. Invertíamos mucho tiempo en revisar; incluso, algunas veces, por querer aprender más, me quedaba desarrollando más tiempo, creyendo que alcanzaría un buen nivel pronto. Luego, con la ayuda de un buen amigo y maestro, aprendí que la clave no era ir tras la moda o lo último en código, sino dominar la arquitectura, los principios de diseño y el rigor metodológico.

Ahora

Hoy en día no necesitamos escribir cada línea de código de forma manual. Sin embargo, como los agentes producen comportamientos no especificados línea por línea, el trabajo del ingeniero de software evoluciona: nos enfocamos en diseñar las condiciones, los límites, las invariantes y las arquitecturas sobre los cuales estos sistemas autónomos pueden operar de forma segura y consistente.

El puente de la Ingeniería de Software

Unificamos el ciclo de vida (SDLC) y la arquitectura (Runtime) en una matriz de transición compacta:

Capa / Fase	Software Clásico	Puente de Integración	Sistemas Multiagente
1. Entidades	Requisitos, Clases y Servicios	Esquemas e Interfaces	Roles, Metas e Invariantes
2. Interacción	Monolitos/Micro, REST y RPC	Middlewares y Brokers	Redes, ACL y Negociación
3. Decisión	Algoritmos y Código OOP	APIs y Orquestadores	Razonamiento BDI y Prompts
4. Control/Riesgo	Unit Tests, DB y Transacciones	Sandboxes y Data Fabric	*Evals*, Vectores y *Guardrails*
5. Operación	CI/CD, Binarios y Observabilidad	Infraestructura distribuida	Trazabilidad y Autonomía
6. Evolución	Parches y Refactorización	Estándares de Calidad	Alineamiento y Aprendizaje

Esta matriz ilustra cómo la ingeniería clásica evoluciona en seis capas hacia el paradigma agéntico a través de su puente de integración. De ella se desprende nuestra máxima: la ejecución técnica del código se delega como tarea operativa, pero la arquitectura, la gobernanza y los límites se ejercen activamente como acto de autoridad sobre el sistema. Aunque los agentes prometen agilidad, la complejidad no ha desaparecido; por el contrario, aumentan la complejidad accidental y las dependencias de proveedores cuyos modelos responden a sus propios intereses. Por esta razón, delegar sin ejercer produce software opaco, frágil y supeditado a terceros, mientras que ejercer antes de delegar da lugar al código sintético que necesitamos: un software seguro, alineado con el negocio, rápido en su ejecución y robusto en su diseño.

> Código Sintético te ayuda a eso: a retomar el rumbo hacia la transformación digital agéntica, porque como me lo enseñó mi maestro, a quien citaré con mucho aprecio: *la única forma de ir rápido es ir bien.* (Martin 2017)

Nota: Aunque Robert C. Martin popularizó masivamente esta máxima en la industria del software (*«The only way to go fast, is to go well»*) a través de *Clean Architecture* y *Clean Code*, el propio autor acredita que la premisa proviene de los principios de diseño y gestión de sistemas clásicos (en particular, de la aviación militar y del entrenamiento de respuesta rápida): *«Slow is smooth, smooth is fast»* (Lento es fluido, fluido es rápido).

— **Sergio Perez Ruiz**

El despertar agéntico

El desarrollador actual

Durante cuarenta años medimos la productividad de un programador en líneas de código por hora. Tickets cerrados por sprint. Pull requests integrados por semana. Métricas industriales aplicadas a un trabajo puramente intelectual. Había un techo físico: la velocidad de escritura en el teclado. Ese techo desapareció. No ocurrió en un año específico; fue una evolución en cadena:

- **2017:** Transformer (Vaswani et al. 2017). La arquitectura que lo hizo todo posible.
- **2020:** GPT-3 (Brown et al. 2020). *Few-shot learning*. El modelo resolvía tareas nuevas sin necesidad de reentrenamiento.
- **2021:** Codex (M. Chen et al. 2021). Un modelo especializado capaz de escribir código con un éxito que sorprendió a sus propios creadores.
- **2022:** RLHF (Ouyang et al. 2022). Ajuste mediante retroalimentación humana. Emergencia de capacidades operativas visibles de inmediato.
- **2023:** ReAct (Yao, Zhao, et al. 2023). Los modelos aprendieron a actuar, a usar herramientas externas y a alternar razonamiento con ejecución.
- **2024-2026:** Sistemas multiagente basados en LLM (S. Chen et al. 2024). Arquitecturas con múltiples agentes especializados, coordinación de tareas y supervisión humana para aplicaciones complejas.

El desarrollador actual ya no escribe sintaxis desde cero; opera herramientas que la automatizan. La destreza técnica no desaparece, pero el cuello de botella físico se elimina. Pasamos de programar código manualmente a gobernar el flujo de código generado ya no por nuestras

propias capacidades intelectuales, sino mediante la ayuda de los modelos de inteligencia artificial.

Qué es, formalmente, un agente

Un agente no es un chatbot. La industria confunde este límite constantemente debido al marketing. La definición formal viene de la inteligencia artificial distribuida y es muy anterior a los LLMs. Wooldridge la planteó en 2009, antes de los Transformers, de GPT y de que existiera el término *prompt*. Su definición sigue siendo exacta: > Un agente es una entidad situada en un entorno, capaz de percibir ese entorno, razonar sobre él, actuar mediante efectores y aprender de los resultados. Traducido a la ingeniería de software actual con LLMs:

- **Percibe:** Lee el contexto de la conversación, mensajes entrantes y resultados de herramientas ejecutadas.
- **Razona:** Procesa información mediante su conocimiento paramétrico combinado con el contexto inyectado en el *prompt*.
- **Actúa:** Ejecuta llamadas a funciones, herramientas locales y APIs externas.
- **Aprende:** Adapta su comportamiento dentro de la misma sesión a través del historial de contexto, no modificando los pesos del modelo.

Un chatbot se queda a mitad de camino: percibe y razona, pero no actúa sobre el sistema subyacente. No cierra el ciclo operativo.

Reactivo y proactivo: las dos mitades de la agencia

La teoría clásica descompone la agencia en una tensión constante entre dos capacidades que rara vez conviven de forma nativa (Wooldridge 2009):

- **Reactivo:** Responde a los cambios del entorno en tiempo real. El bug imprevisto, el test que se rompe, la alerta de infraestructura que salta.
- **Proactivo:** Toma la iniciativa, descompone la meta principal en subtareas, planifica y persiste hasta cumplir el objetivo.

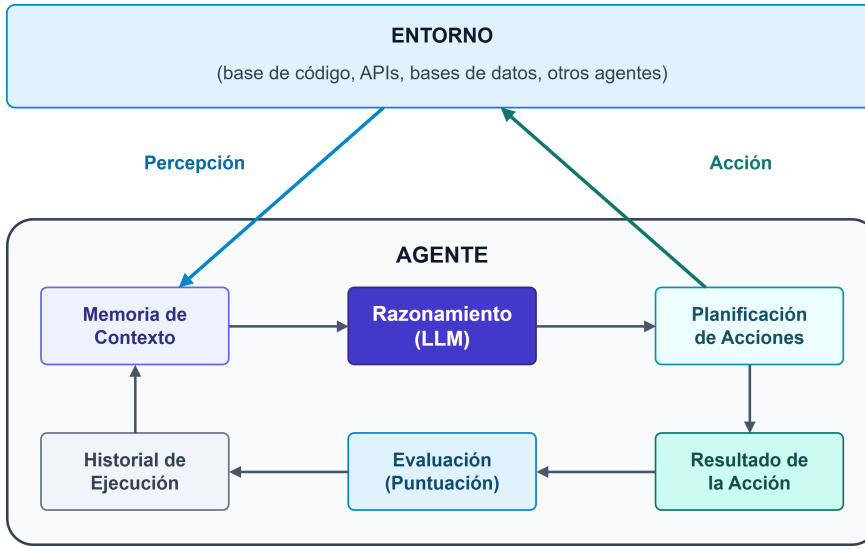


Figura 1.1: Ciclo de un agente autónomo

Un agente puramente reactivo es incapaz de emprender proyectos por sí mismo; responde rápido a estímulos, pero no resuelve tareas complejas de varios pasos. Un agente puramente proactivo diseña planes perfectos para un entorno estático, ignorando que el mundo real se mueve mientras calcula. El equilibrio técnico en producción se llama **ReAct (Reasoning + Acting)**. El patrón detallado en el Capítulo 4 intercala observación y razonamiento técnico en cada ciclo para resolver esta desconexión.

La proactividad ambiental

La tendencia actual del mercado apunta hacia los "agentes ambientales". Sistemas que corren en segundo plano, se disparan mediante eventos de arquitectura y solo nos notifican cuando detectan anomalías que requieren atención. La promesa de automatización es alta, pero exige un escrutinio estricto: cada grado de proactividad que gana el sistema es un grado de control operativo que cede la organización. Un agente ambiental que toma decisiones críticas sin supervisión ejecuta criterios que aún no has auditado en producción. *(Consúltese el Capítulo 6 para profundizar en el concepto de confianza calibrada).*

Marco de evaluación de riesgo y router

Mide la **criticidad y el costo** (*Blast Radius*) de las tareas para derivarlas a carriles con mayor o menor grado de supervisión humana (Niveles N1 a N5).

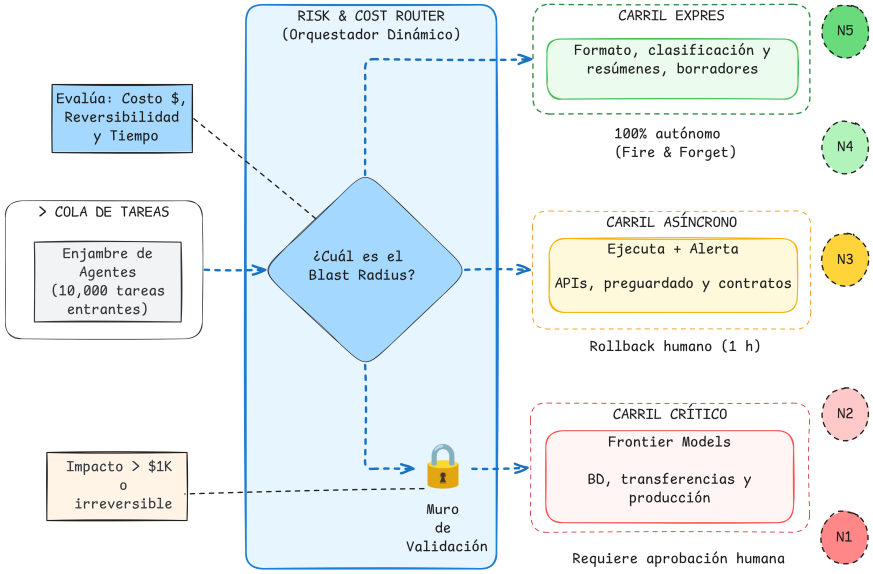


Figura 1.2: Evaluación de riesgo y router

Para representar adecuadamente el flujo del **Risk & Cost Router**, se debe emplear la siguiente matriz:

Nivel	Carril Asignado	Criterio de Riesgo / Blast Radius	Acciones / Alcance	Mecanismo de Control
N5 / N4	**Carril Exprés**	Bajo costo, alta reversibilidad y tiempo corto.	Formatos, clasificación, resúmenes y borradores.	**100% Autónomo** (*Fire & Forget*).

Nivel	Carril Asignado	Criterio de Riesgo / Blast Radius	Acciones / Alcance	Mecanismo de Control
N3	**Carril Asíncrono**	Riesgo moderado.	Ejecución + Alerta, llamadas a APIs, preguardados y contratos.	**Autónomo con opción de Rollback** (ej. ventana de 1 hora).
N2 / N1	**Carril Crítico**	Alto impacto (> $1K) o acción irreversible.	Uso de *Frontier Models*, modificación de BD, transferencias y despliegue a producción.	**Muro de Validación:** Requiere aprobación humana previa (*Human-in-the-loop*).

Marco de niveles de autonomía de sistemas IA

La agencia no es un interruptor binario; es un espectro gradual de autonomía. Definimos cinco niveles operativos para determinar quién controla el flujo del sistema.

Nivel	Sistema	Quién decide	Ejemplo concreto
0	Llamada única	El desarrollador	Responder una pregunta frecuente o extraer información puntual de un documento.
1	Cadena (*workflow*)	El desarrollador	Un pipeline secuencial fijo: extracción, validación y carga en ERP.

Nivel	Sistema	Quién decide	Ejemplo concreto
2	Enrutador	El modelo elige entre rutas fijas	Triaje automatizado: ticket severo, sprint inmediato; ticket bajo, backlog.
3	Agente con herramientas	El modelo (bajo presupuesto)	Depuración de un test: el agente decide qué archivos leer y cuándo detenerse.
4	Sistema multiagente	Orquestador + agentes especializados	Desarrollo y despliegue de una funcionalidad completa de punta a punta.

La tesis económica fundamental: El nivel de autonomía correcto es el mínimo que resuelve el problema de negocio, no el máximo que la tecnología permite. No implementes un Nivel 3 si un Nivel 1 soluciona el caso de uso. El diseño simple es más barato, predecible y fácil de auditar.

Mapeo de relación entre ambos marcos

Si se desea integrar la **Tabla de Arquitectura (0-4)** con el **Diagrama de Riesgo (N1-N5)**, la correspondencia de ejecución sería la siguiente:

1. **Agente/Sistema Entrada (Nivel 3/4 de la tabla):** El *Enjambre de Agentes* genera tareas complejas.
2. **Enrutamiento (Nivel 2 de la tabla):** El *Risk & Cost Router* actúa como un enrutador inteligente determinando la ruta fija (Carril Exprés, Asíncrono o Crítico).
3. **Ejecución y Control (N1 a N5 del diagrama):** Determina si el resultado requiere intervención directa del desarrollador/operador humano antes de su aplicación en producción.

Mapeo Integrado: Capa de Autonomía (0-4) vs. Capa de Gobernanza (N1-N5)

Ambos esquemas no se contradicen, sino que operan en **dos capas distintas del ciclo de vida de una tarea de IA**:

1. **Capa de Generación y Ejecución (Niveles 0 a 4):** Define *cómo piensa y actúa el sistema.* Un **Sistema Multiagente (Nivel 4)** o un **Agente con herramientas (Nivel 3)** genera las tareas o planes de acción.
2. **Capa de Control y Gobernanza (Niveles N1 a N5):** Define *cuánta libertad tiene el sistema para aplicar los cambios.* Un **Router de Riesgo (Nivel 2 de arquitectura)** intercepta cada tarea generada, mide su *Blast Radius* (Impacto financiero, tiempo y reversibilidad) y la clasifica en un carril de política de seguridad.

Flujo de Interacción

- **Generación:** Un **Enjambre de Agentes (Nivel 4 de la tabla)** coloca miles de tareas en la *Cola de Tareas.*
- **Evaluación:** El **Risk & Cost Router (Nivel 2 de la tabla)** actúa como un *Proxy/Middleware* de seguridad que decide la ruta evaluando el *Blast Radius.*
- **Despacho:**
  - Si la tarea del agente es cosmética (resumen, formato), baja a **N5/N4 (Carril Exprés)** y ejecuta como **Workflow Fijo (N1 de la tabla)** sin supervisión.
  - Si la tarea ejecuta acciones en API con estado pero reversibles, va a **N3 (Carril Asíncrono)** con política de *Rollback.*
  - Si la tarea implica escribir en Base de Datos de producción o transferencias monetarias (> $1K), el Router la frena en el **Muro de Validación (N2/N1)** hasta requerir aprobación humana (*Human-In-The-Loop*), independientemente de qué tan autónomo sea el agente que la creó.

Caso de estudio para analizar los niveles

Un equipo de ingeniería en Buenos Aires intentó automatizar por completo su soporte técnico mediante un enjambre de 7 agentes orientados a clasificar tickets, buscar documentación técnica, sugerir

parches de código y escalar incidentes críticos. Implementaron un Nivel 4 de autonomía sin haber dominado antes las bases del Nivel 1. Resultados tras tres meses en producción:

- **Costo de tokens:** $2.400 dólares al mes.
- **Tasa de acierto operativo:** 34 % (sin revisión humana).
- **Inversión en debugging:** 3 ingenieros asignados a tiempo completo para corregir fallos del sistema.
- **Satisfacción del cliente (CSAT):** Cayó un 12 %. **El error de diagnóstico:** Asumieron que añadir la complejidad de una arquitectura multiagente resolvería la complejidad intrínseca de su propio negocio. En producción ocurre lo contrario: la arquitectura multiagente añade fricción técnica, no la elimina. **La estrategia de corrección:**

1. **Paso 1 (Nivel 1):** Diseñaron un *workflow* determinista fijo para los 3 tipos de tickets más frecuentes.
2. **Paso 2 (Nivel 2):** Implementaron un enrutador basado en LLM para derivar solicitudes entre 5 categorías claras.
3. **Paso 3 (Nivel 3):** Agregaron un agente con acceso a herramientas de consulta para tickets de investigación profunda.
4. **Paso 4 (Nivel 4):** Reservaron el enjambre multiagente en producción exclusivamente para el 5 % de los tickets con mayor ambigüedad técnica.

Métricas tras seis meses de la corrección:

- **Costo de tokens:** Se redujo a $380 dólares al mes.
- **Tasa de acierto operativo:** Subió al 87 %.
- **Tiempo de ingeniería en debugging:** Reducido a 0.5 FTE.
- **Satisfacción del cliente (CSAT):** Incrementó un 8 %.

La decisión de nivel

Antes de delegar autonomía operativa en producción, tu comité técnico debe responder tres preguntas críticas:

1. **¿Sabemos con precisión cuánto gastamos al mes en tokens y qué servicios los consumen?**

Si la respuesta es negativa, tu arquitectura está en nivel 1. Detén el desarrollo agéntico y construye primero un *AI Gateway*. Un equipo de 4 a 6 ingenieros de plataforma dedicados a la observabilidad multiplica la eficiencia de 100 desarrolladores. No contar con visibilidad de costos multiplica el gasto operativo por un factor de 3 a 5.

2. **¿Qué porcentaje del código generado de forma autónoma llega a producción sin necesidad de retrabajo humano?**

Si la métrica es inferior al 40 %, estás inyectando deuda técnica acelerada en tus repositorios. *(Consúltese el Capítulo 11).*

3. **¿Tenemos un plan estratégico para transformar a nuestros ingenieros senior en orquestadores de sistemas?**

La transición no ocurre de manera orgánica. Requiere formación deliberada en diseño de sistemas distribuidos y control de flujo. *(Consúltese el Apéndice C: Roadmap de aprendizaje).*

Ejercicio para clasificar tus sistemas

Enunciado: Clasifica el nivel de autonomía (0 al 4) de los siguientes cuatro sistemas corporativos: - (a) Un servicio que recibe un ticket de soporte técnico y devuelve un resumen ejecutable en una sola línea. - (b) Un asistente que inspecciona el repositorio de código, reproduce un fallo de integración de forma local y propone un parche de corrección. - (c) Un flujo de agente automatizado que extrae datos estructurados de facturas en PDF, los valida contra reglas de negocio fijas y los carga en el ERP corporativo. - (d) Un sistema compuesto por un agente coordinador que descompone una migración de base de datos y reparte las tareas de refactorización entre agentes especialistas de código, bases de datos y testing.

Solución comentada:

- **(a) Nivel 0 (Llamada única):** El sistema realiza una única ejecución por ticket mediante API. No existen decisiones de flujo ni ramificaciones en tiempo de ejecución; es una clasificación simple.
- **(b) Nivel 3 (Agente con herramientas):** El modelo evalúa las herramientas disponibles en su entorno, decide autónomamente qué archivos examinar, cuándo ejecutar la suite de pruebas y en qué momento detener la iteración bajo un presupuesto de tokens controlado.
- **(c) Nivel 1 (Cadena / Workflow):** Las etapas del proceso son completamente secuenciales y deterministas. El uso de la palabra "agente" aquí es solo marketing; el flujo es un pipeline tradicional de software.
- **(d) Nivel 4 (Sistema multiagente):** Requiere descomposición dinámica de tareas en múltiples dominios técnicos en paralelo. Este nivel solo es viable si los agentes individuales de Nivel 3 operan de forma estable y aislada primero.

Lección de producción: El nivel de autonomía se selecciona basándose en la estructura técnica de la tarea, nunca por la ambición de los equipos de desarrollo.

Lo esencial del capítulo 1

La secuencia evolutiva 2017-2026: Pasamos de la arquitectura Transformer elemental al *few-shot learning*, la generación de código, la instrucción humana estructurada, el uso de herramientas de entorno y, finalmente, a los sistemas multiagente en producción.

- **Agente frente a chatbot:** Un agente cierra el ciclo operativo en producción porque percibe, razona y actúa sobre sistemas externos. Un chatbot tradicional solo ejecuta las dos primeras acciones.
- **El patrón ReAct equilibra el sistema:** Intercala pasos secuenciales de observación y razonamiento en cada ciclo para evitar que el agente ejecute planes desactualizados en entornos dinámicos.
- **El espectro consta de 5 niveles:** La regla de oro arquitectónica exige implementar siempre el nivel mínimo que resuelva el problema de negocio de forma predecible.

- **Saltar niveles destruye el ROI:** El caso de estudio de Buenos Aires demuestra cómo omitir la progresión de niveles genera sobrecostos en tokens y consume horas críticas de ingeniería en depuración.
- **El desplazamiento del costo operativo:** El cuello de botella de la ingeniería de software se trasladó de la fase de escritura de código a las fases de validación, gobernanza y auditoría automatizada.

El AI Engineer

Un rol emerge

En 2023, Shawn Wang ("swyx") publicó *The Rise of the AI Engineer*. No se trataba de una simple variación de títulos; emergía un profesional nuevo que no encajaba como ML Engineer (enfocado en entrenar modelos) ni como Data Scientist (orientado a analizar datos). Nacía el perfil encargado de construir aplicaciones reales sobre modelos ya entrenados. Entrenar un modelo base cuesta cientos de millones de dólares. Construir un producto encima cuesta una clave de API y la arquitectura correcta. Donde existe una asimetría económica y operativa de tal magnitud, emerge una nueva profesión de forma inevitable. Ese profesional, el AI Engineer, es el verdadero arquitecto de la era agéntica.

La matriz de diferenciación

Tres roles, tres direcciones de flujo de trabajo completamente distintas:

Rol	Dirección	Entrada	Salida	Qué mide su éxito
Data Scientist	Hacia los datos	Datos históricos	Modelos predictivos	AUC / F1-score en validation set
ML Engineer	Hacia el modelo	Datos + algoritmos	Pesos entrenados	Latencia de inferencia, escalabilidad de infraestructura

Rol	Dirección	Entrada	Salida	Qué mide su éxito
AI Engineer	Desde el modelo	Prompts, contexto, herramientas	Sistemas agénticos	Valor de negocio por token consumido

La distinción fundamental radica en la dirección del flujo. El ML Engineer trabaja *hacia* el modelo: su día a día consiste en limpiar datasets, entrenar arquitecturas, ajustar hiperparámetros y validar pesos. El AI Engineer trabaja *desde* el modelo: asume que el modelo base ya es inteligente y se enfoca en diseñar prompts de producción, gestionar ventanas de contexto, definir el scope de las herramientas mediante APIs y evaluar el rendimiento del sistema vivo. No hablamos de "más de lo mismo": es un cambio radical de perspectiva.

¿Por qué no es solo un desarrollador con IA?

Un desarrollador senior en Python que usa ChatGPT para generar *boilerplate* no es AI Engineer. Está en el lado equivocado del flujo: sigue siendo ejecutor de código. El AI Engineer diseña sistemas donde el modelo es un componente. Define las interfaces, verifica las salidas y mide el costo por decisión.

Vamos a dar un ejemplo concreto: un desarrollador pide a un agente: "genera una función que ordene por fecha". El agente devuelve código y el desarrollador lo aprueba. En cambio, un AI Engineer trabaja de esta forma con el agente: "diseña un *pipeline* que recupere documentos, los procese en lotes, valide formato, depure errores y genere métricas de costo. Cada paso tiene un agente especializado. El *pipeline* tiene *circuit breakers*. Si un paso falla, hay *fallback*. El costo por documento se rastrea.". Dos trabajos distintos, uno produce *scripts*, mientras que el otro produce sistemas.

Las siete competencias

A diferencia de las "habilidades blandas", son competencias técnicas medibles. Un AI Engineer senior domina las siete. Un junior empieza con 2-3 y las desarrolla en 8-12 semanas con proyecto real.

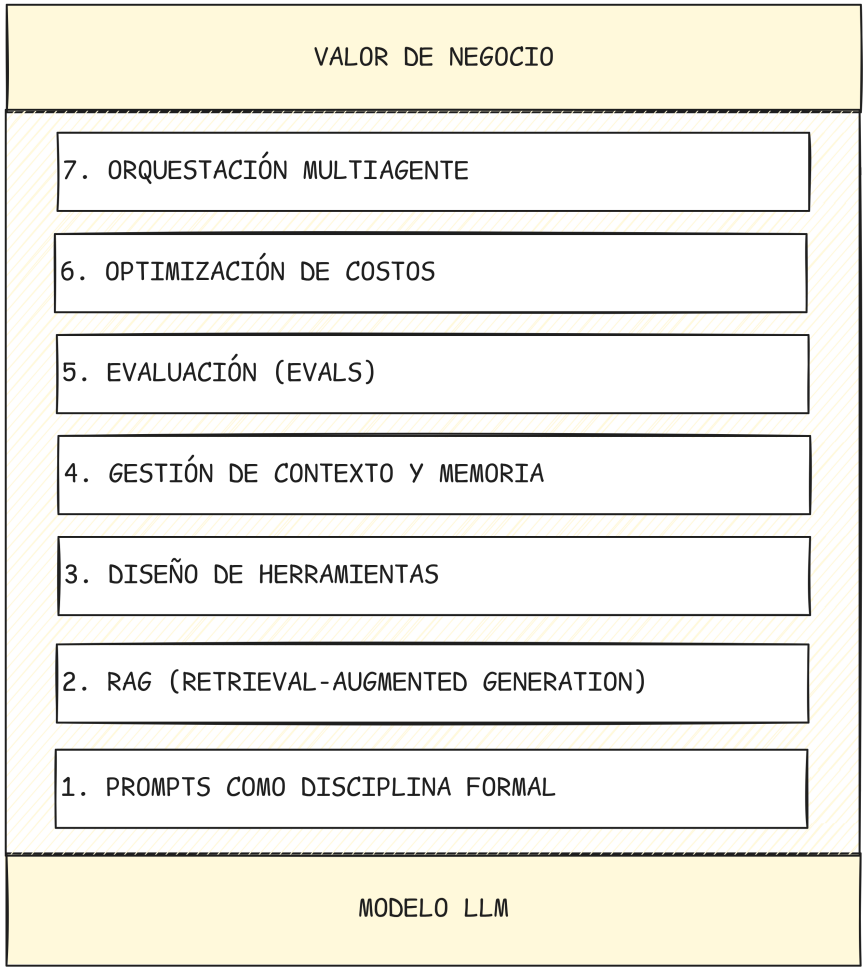


Figura 2.1: Arquitectura de capas

Prompts como disciplina formal

Debemos considerar al AI Engineer más que un "ingeniero de prompts", pues une las disciplinas de *testing* y diseño de APIs. Veamos que más nos dice la investigación lo respalda:

- Chain-of-Thought mejora razonamiento multi-paso (Wei et al. 2022).
- Few-shot learning es emergente (Brown et al. 2020).
- DSPy demuestra que los prompts son programas optimizables (Khattab et al. 2023).

Un prompt de producción no se escribe una vez. Se versiona, se testea, se revisa en PR como código. Comparación entre un prompt genérico y uno de producción:

> **Mal — prompt genérico:** *"Analiza este código y dime si tiene bugs"*
>
> **Bien — prompt prescriptivo con formato de salida:** *"Eres un revisor de código senior especializado en Python.*
>
> *TU TAREA: 1. Identifica bugs de lógica (no estilo, ni naming) 2. Para cada bug: proporciona línea, descripción, prueba mínima que falla 3. Si no encuentras bugs: explica por qué el código es correcto*
>
> *CADA BUG DEBE CONTENER: - Tipo: [null-pointer] [off-by-one] [race-condition] [logic-error] [data-type] - Severidad: [critical] [high] [medium] [low] - Prueba mínima que expone el bug*
>
> *FORMATO DE SALIDA: JSON con array de bugs."*

La diferencia: el primer prompt genera texto. El segundo genera datos estructurados que un sistema puede consumir.

RAG (Retrieval-Augmented Generation)

El patrón para conectar LLM con conocimiento privado: recuperar fragmentos relevantes e inyectarlos en contexto. El error común: inyectar el documento completo. El resultado: el modelo se distrae, alucina o depende del fragmento relevante. RAG bien hecho:

1. Segmentación: *chunk size* óptimo varía por dominio (200-800 tokens típicamente).
2. Embeddings: elegir modelo según idioma y dominio técnico.
3. Indexación: *vector store* con metadatos extraíbles.
4. Re-ranking: después de recuperar, filtrar por relevancia.

El costo de RAG mal hecho: un proyecto que inyecta *chunks* gigantescos sin re-ranking ve un 40 % de alucinación frente al 8 % con estrategias de dos etapas.

Diseño de herramientas

La descripción de una herramienta es su prompt. El modelo decide cuándo invocarla leyendo esa descripción, no el código. Descripción prescriptiva de herramienta:

```
{
    "name": "buscar_documentacion_tecnica",
    "description": "Busca en la base de conocimiento técnica de la
    empresa. USAR cuando: el usuario pregunta sobre arquitectura
    interna, APIs propias, decisiones de diseño pasadas, estándares del
    equipo. NO USAR para: código de librerías externas (usa docs
    oficiales), conceptos generales de programación, información que no
    esté documentada internamente. La consulta debe ser específica. Si
    el usuario dice 'cómo envío email', debes preguntar 'qué servicio
    interno usas: SendGrid, SES, o SMTP propio?'. Formato de salida:
    array de {doc_id, title, snippet, confidence}.",
    "input_schema": {
        "type": "object",
        "properties": {
            "consulta": { "type": "string" },
            "dominio": {
                "type": "string",
               "enum": ["arquitectura", "apis", "seguridad", "database"]
            }
        },
        "required": ["consulta", "dominio"]
    }
}
```

Patrón clave: descripciones prescriptivas (USAR cuando / NO USAR para) mejoran la tasa de invocación correcta entre un 25 % y un 35 %.

Gestión de contexto y memoria

Las ventanas crecieron de 4.000 a más de un millón de tokens. Pero espacio no equivale a atención. El fenómeno "lost in the middle" (Liu et al. 2024): los modelos recuperan peor la información del medio del contexto. Mejor lo del principio y lo del final. Curva en U. Consecuencias de diseño:

- Posición importa: instrucciones críticas al principio (*system*) o al final (último mensaje).
- Menos es más: fragmento relevante inyectado > documento completo volcado.
- Memoria larga se externaliza: MemGPT gestiona jerarquía contexto/disco.

Evaluación (evals)

Los sistemas LLM son no deterministas. No puedes "testearlos" como código tradicional. Necesitas arneses de evaluación. El equivalente de TDD para sistemas probabilísticos. La implementación de referencia (`EvalSet`, en el repositorio del libro) acumula pares de entrada/resultado-esperado con su criterio de aceptación, y ejecuta el agente contra todos ellos para calcular una tasa de aciertos (`pass_rate`).

Uso en CI/CD: si pass_rate < 0.80, bloquea el merge del PR, eso sí, el LLM como juez alcanza un porcentaje mayor al 80 % de acuerdo a lo establecido. No es perfecto, pero escalable.

Optimización de costos

La factura de tokens no se optimiza en la configuración. Se optimiza en el código. Lo que más mueve la factura: - Contexto inyectado innecesariamente (cada token cuesta, aunque no se use). - Llamadas a modelo "quality" para tareas "simple". - Reintentos sin *backoff*. - Sin caché de respuestas recurrentes. Ejemplo de ahorro: un equipo redujo la factura un 65 % cambiando: *a)* De "opus" a "haiku" para triaje de tickets (3 veces más barato, precisión 91 % frente a 94 %). *b)* Caché de respuestas comunes (20 % de las consultas son duplicadas). *c) Chunk*

size de RAG optimizado de 1000 a 400 tokens (más relevante, menos costo).

Orquestación multiagente

Sistemas donde múltiples agentes colaboran y se gobiernan. Requiere las 6 optimizaciones de costos anteriores. Sin evals ni control de costos, multiagente es incendio distribuido. **Patrón de enjambre mínimo:** la implementación de referencia (`SwarmOrchestrator`, en el repositorio del libro) descompone la tarea en subtareas, verifica —antes de ejecutar cada una— si su rol requiere aprobación humana y si su *trust gate* supera el umbral mínimo (0,65), y solo entonces delega la ejecución al agente correspondiente, consolidando los resultados al final.

El orquestador descompone tareas, verifica trust gates antes de ejecutar y consolida resultados. Requiere evals por agente antes de habilitar el enjambre.

El camino de conversión: de developer a AI Engineer

No pasa por un máster. Pasa por 8-12 semanas sobre las 7 competencias, aplicadas a proyecto real, con presupuesto de tokens y métricas definidas al día:

1. Escenario real: Cuatro desarrolladores senior en Python, sin experiencia previa en IA. Lima, Perú.

- Semanas 1-2: Prompts y herramientas. Construyen chatbot con 3 herramientas reales.
- Semanas 3-4: RAG y contexto. Indexan documentación técnica de la empresa.
- Semanas 5-6: Evals. Construyen arnés de evaluación con 50 muestras reales.
- Semanas 7-8: Costos y orquestación mínima. *Pipeline* con presupuesto.

Resultado a 8 semanas: sistema de revisión de contratos que antes consumía 40 horas semanales, ahora procesa lotes con céntimos por documento y revisión humana final. El ROI se recupera en el primer trimestre. Dos advertencias:

1. "Precisión del 94 %" no significa nada sin definir el *eval-set* y el costo de cada error.
2. El sistema es *workflow* con puntos de revisión sin agente autónomo, pertenece al nivel 2.

Ejercicio resuelto: diseñe el programa de conversión

Enunciado. Programe la conversión para 3 desarrolladores senior en su organización. Defina secuencia, proyecto vehículo y métricas semanales. **Solución comentada.**

Semana 1-2: Prompts y herramientas - Proyecto: chatbot que responde sobre documentación interna. - Métrica: % de respuestas correctas (*eval-set* de 20 muestras). - Meta: 80 % de precisión con *prompts* de pocas muestras.

Semana 3-4: RAG y contexto - Proyecto: QA sobre código y documentación. - Métrica: relevancia de *chunks* recuperados (*precision@5*). - Meta: 75 % de *chunks* relevantes en top-5.

Semana 5-6: Evals - Proyecto: arnés de evaluación completo. - Métrica: F1-score sobre *eval-set* vivo. - Meta: F1 $>$ 0,82.

Semana 7-8: Costos y orquestación - Proyecto: *pipeline* con presupuesto por día. - Métrica: costo por documento procesado. - Meta: $<$ \$0,01 por documento.

Error común: empezar por multiagente. La competencia 7 sin las 6 anteriores es incendio distribuido.

Lo esencial del capítulo 2

AI Engineer construye desde el modelo, en cambio el ML Engineer construye hacia el modelo.

- Las 7 competencias tienen literatura y métricas medibles.
- La descripción de una herramienta es su prompt: el modelo decide cuándo invocarla leyendo la descripción.
- La prima salarial es dato estratégico: los roles de IA están en percentiles superiores.

- Conversión interna (8-12 semanas, proyecto real) supera a un máster.
- Prompt de producción: se versiona, se testea, se revisa como código.
- RAG mal diseñado: incorpora documentos completos sin selección ni compresión de contexto, lo que aumenta el riesgo de respuestas incorrectas.
- Evals es TDD de sistemas LLM: F1-score sobre *eval-set* vivo.

Platform Engineering en la era agéntica

La plataforma como producto interno

Platform Engineering construye los caminos pavimentados por donde transitan los equipos. No es infraestructura: es la abstracción que convierte operaciones complejas en acciones de un clic. *Team Topologies* (Skelton y Pais 2019): la estructura debe minimizar la carga cognitiva. El equipo de plataforma absorbe esa carga. En la era agéntica, gana una responsabilidad adicional: exponer la IA como primitiva de primera clase. Si cada equipo negocia su propia clave, elige su modelo e implementa su control de costos, la organización tiene tantos fallos como equipos.

El error de cinco equipos con cinco claves

Caso real de una empresa de 80 ingenieros:

- 5 equipos, 5 claves de API distintas.
- Cada equipo pensaba que "su" gasto era razonable.
- Costo total mensual: $47.000.
- Visibilidad central: cero.
- Cuando uno agotó la cuota, no hubo *fallback*.

Corrección: *gateway* central. Presupuesto por equipo. *Fallback* automático. *Tracking* único. El costo mensual bajó a $18.000 después de la optimización.

El AI Gateway: componente crítico

Punto único de acceso a todos los modelos. Enrutamiento, contabilidad, presupuestos, auditoría. > El AI Gateway es ante todo un flujo: una

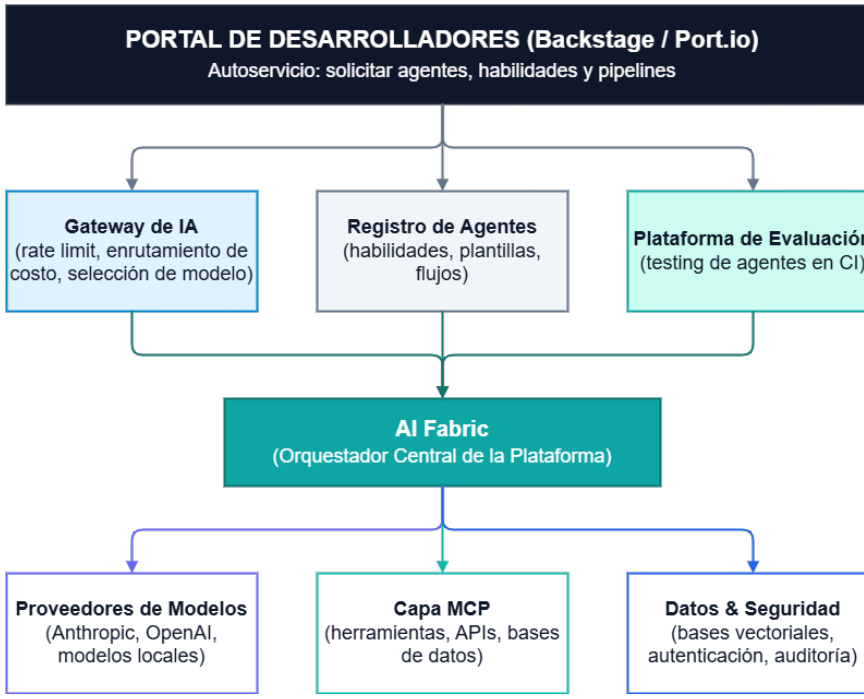


Figura 3.1: Plataforma interna de desarrollo de IA Flujo con Gateway de IA

solicitud entra, pasa por capas (validación, enrutamiento, ejecución, registro), y sale con un costo. Lo que importa es el *orden temporal* de esos pasos: cuánto cuesta cada uno, cuánto tiempo toma, dónde se revisa antes de proseguir. Sin *gateway* no hay inventario. Sin inventario no hay respuesta para el auditor, un requisito fundamental para el gobierno y la trazabilidad de los sistemas de IA (Tabassi 2023; European Parliament and Council of the European Union 2024).

Cómo funciona el AI Gateway

El código completo de este ejemplo —registro de modelos, gestión de presupuesto, selección automática, cálculo de costo, ejecución de la solicitud y adaptadores por proveedor— está disponible en el repositorio complementario del libro. En esencia:

- Un **registro de modelos** (`MODEL_REGISTRY`) asocia cada nivel de tarea ("fast", "balanced", "quality") con un modelo concreto, su

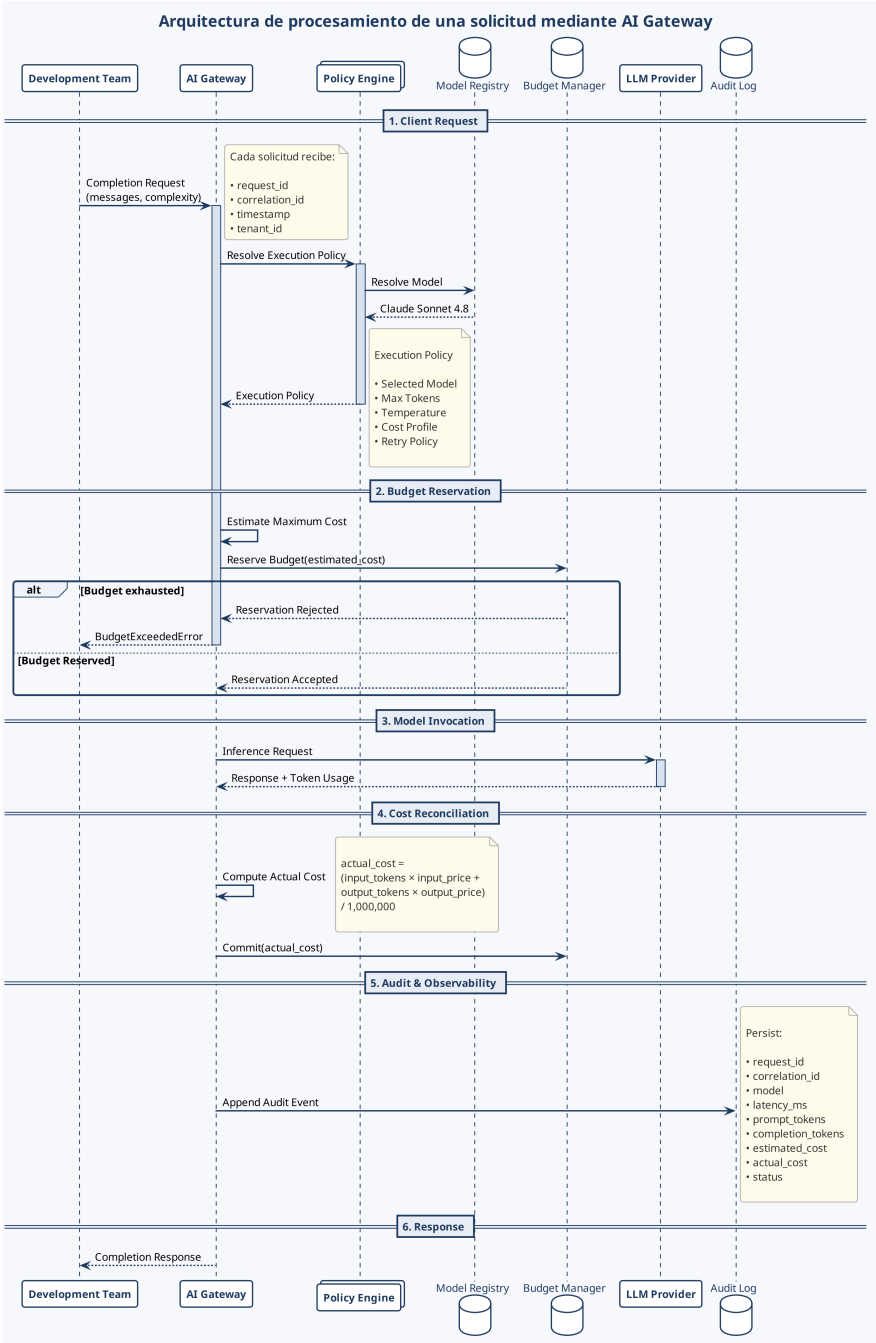


Figura 3.2: Arquitectura de procesamiento de una solicitud mediante AI Gateway

costo por millón de tokens de entrada/salida y su tamaño máximo de contexto.
- El **gateway** recibe cada solicitud, elige el modelo según la complejidad de la tarea (o uno forzado explícitamente), verifica que el equipo no haya excedido su presupuesto mensual, llama al proveedor, calcula el costo real a partir de los tokens consumidos y registra la solicitud para auditoría.
- Los **adaptadores** (Claude, OpenAI, Gemini) implementan un contrato común —un único método `complete(system, user, model, max_tokens)`— de modo que cambiar de proveedor no exige tocar la lógica de negocio, solo intercambiar el adaptador. Este es el patrón Backend-Agnóstico que se profundiza en el Capítulo 19.

La decisión de enrutamiento (qué modelo usar para cada nivel de tarea) es de negocio: se toma con evaluaciones propias sobre tareas propias y se revisa trimestralmente, pues la frontera de calidad-precio entre proveedores se mueve varias veces al año.

Plantillas de equipo

Métricas de ROI del Platform Engineering

Un equipo de 4-6 ingenieros puede multiplicar la productividad de 50-100. La diferencia: el "servidor" es acceso a inteligencia.

Métrica	Línea base	Objetivo (6 meses)
Integración de IA	2-3 semanas	2-3 días
Costo por request	Sin visibilidad	$<$ \$0,01 en tareas simples
Ruteo correcto	0 % (manual)	$>$ 80 % (automático)
Cobertura de evals	0 %	$>$ 70 %

Caso de estudio del CTO que descubrió el desperdicio

Un CTO aplicó análisis de costos al *gateway* central. Descubrió:

- 60 % del presupuesto en tareas nivel 0 (clasificación simple).
- 25 % en tareas nivel 1 (*workflows* fijos).
- Solo 15 % en tareas nivel 3-4 (autonomía real).

Acción:

1. Mueve nivel 0 a haiku (3 veces más barato): ahorro del 42 %.
2. Introdujo *threshold* de confianza: por debajo, degradación a humano: ahorro del 18 %.
3. Reasignó presupuesto a nivel 3: más autonomía donde importa. Resultado: mismo presupuesto, 3 veces más "inteligencia entregada".

Matriz de arquitectura según el tamaño del proyecto

Un error común es pensar que necesitas toda la plataforma desde el día 1. Kafka, Prometheus, FAISS, Gateway, MCP, Circuit Breakers... Eso no es arquitectura, es una trampa. Cada patrón tiene un costo. Debe justificarse económicamente antes de implementarse.

Principio rector

> "Cada patrón debe justificar su costo económico antes de ser incorporado." No implementes un Council de 5 agentes si un script simple resuelve el problema. No construyas un AI Gateway si solo tienes 2 equipos usando IA.

Matriz de decisión

Tamaño del proyecto	Equipos	Arquitectura recomendada	Patrones esenciales	Qué no necesitas aún
PoC / Prototipo	1-2	Script + API directa	ReAct básico, logging simple	Gateway, caching, evals, monitoreo
Startup (< 10 desarrolladores)	2-5	Gateway ligero + logs	Routing básico, presupuesto por equipo, 1-2 providers	FAISS, CI/CD automático, CIU, Council

Tamaño del proyecto	Equipos	Arquitectura recomendada	Patrones esenciales	Qué no necesitas aún
Empresa mediana (10-50 desarrolladores)	5-20	Gateway + métricas + evals básicos	Caching semántica, TrustScore básico, presupuesto por proyecto, fallback provider	Kafka, Event sourcing, plataforma completa
Enterprise (> 50 desarrolladores)	20+	Plataforma completa	Todos los patrones: Gateway, FAISS, CIU, Council, CI/CD, event-driven, auditoría	Nada, ya estás en capacidad máxima

Reglas prácticas

PoC: - Herramientas: Python nativo, 1 provider, API key en variable de entorno. - Si funciona en 2 semanas y genera valor, escala. Si no, descártalo. - No inviertas en arquitectura hasta tener validación de uso real.

Startup: - Integra un Gateway ligero desde el inicio. Más tarde es refactorización costosa. - Presupuestos por equipo para evitar sorpresas en la factura. - Evals mínimos: 10-20 casos de uso reales. - Caching semántica si tienes $>$ 100 llamadas diarias repetidas.

Empresa mediana: - Gateway central con métricas y auditoría. - TrustScore para decisiones críticas (deploy, finanzas, datos sensibles). - Fallback entre providers para evitar vendor lock-in. - Evals automatizados en CI/CD para cambios en prompts.

Enterprise: - Plataforma AI-IDP completa con onboarding automático de equipos. - Event-driven con Kafka para sistemas multiagente de alta concurrencia. - Council patterns para tareas complejas que requieren debate. - Auditoría completa: quién aprobó qué, cuándo y por qué.

Caso de estudio de la startup que sobreingenieró

Empresa: 5 desarrolladores en México. Objetivo: chatbot para soporte técnico interno. Arquitectura implementada: - Kubernetes para orquestación de contenedores - Redis para caching - PostgreSQL para logging - Grafana + Prometheus para métricas - 3 agents en Council debatiendo cada respuesta. Costo: $3.200 USD/mes en infraestructura. Tiempo de desarrollo: 3 meses. Uso real: 15 consultas diarias. Corrección: - Python simple + FastAPI. - Eliminar Redis (la caché en memoria es suficiente). - 1 agente, no Council. - logging simple en archivo. Nuevo costo: $80 USD/mes. Mantenimiento: 2 horas/semana en lugar de 10.

Lección: la complejidad no escala linealmente con el valor. Alcanzas un punto donde más arquitectura = menos productividad.

Cómo decidir tu nivel de arquitectura

Pregúntate:

1. **¿Cuántos equipos usan IA actualmente?**

- De 1-2 equipos: Gateway ligero o incluso API directa.
- 5+ equipos: Gateway central obligatorio.
- 20+ equipos: Plataforma completa con self-service.

2. **¿Cuál es tu factura mensual actual en tokens?**

- Si es <$500: Prioriza simplicidad. El costo de la plataforma supera el beneficio.
- De $500-$5.000: Gateway con métricas y presupuestos.
- Si es >$5.000: Optimización activa + fallback + caching obligatorios.

3. **¿Qué pasa si el sistema falla?**

- Ningún impacto crítico: tolerancia alta, fallback simple.
- Impacto en experiencia de usuario: TrustScore + canary deployments.
- Impacto en seguridad/finanzas: aprobación humana obligatoria + audit trail.

4. **¿Cuántos tokens consumimos por día?**

- Si es <10.000: cache semántica innecesaria.
- De 10.000-100.000: caching lineal con diccionario suficiente.

- Si es >100.000: FAISS o servicio vectorial externo.

El costo de la sobreingeniería

Patrón	Costo desarrollo	Costo mantenimiento	Valor real
AI Gateway	1-2 semanas	2-4 horas/semana	Alto (visibilidad, control)
Semantic Cache	3-5 días	1-2 horas/semana	Medio (ahorro de tokens)
TrustScore	1 semana	2-3 horas/semana	Alto (seguridad, calidad)
Council (multi-agent debate)	2-3 semanas	5-10 horas/semana	Bajo (solo para tareas complejas)
Kafka + Event sourcing	3-4 semanas	10+ horas/semana	Muy bajo (solo enterprise real)
CIU + Silent Token Burn detector	1 semana	1-2 horas/semana	Alto (detección de deuda invisible)

Regla: Si el costo de mantenimiento supera el 10 % del valor generado al mes, el patrón no vale la pena.

El costo cognitivo de la deuda que no ves en la factura

Este es un tema que ningún libro de IA aborda. Y es probablemente el más importante a largo plazo. Si tu equipo necesita entender RAG, MCP, Kafka, Event sourcing, ReAct, Guardrails, Observabilidad y Evals para mantener la plataforma, el costo de aprendizaje es enorme. La pregunta es: **¿cuántos ingenieros pueden mantener esto dentro de 5 años?**

El problema del costo cognitivo

Caso real: empresa de 40 ingenieros en Barcelona. Arquitectura implementada: - Kafka para event sourcing - Redis para caching distribuido - FAISS para búsqueda semántica - Prometheus + Grafana para métricas - 5 agentes en patrones *Council*.

Resultado después de 2 años: - 3 ingenieros especializados en mantener la plataforma - Nuevos ingresos tardan 3 meses en ser productivos - Si el equipo de plataforma se va, nadie puede mantener el sistema.

Lección: La arquitectura más elegante es inútil si nadie la entiende.

Medir el costo cognitivo

Concepto	Medición	Meta
Tiempo de onboarding	Días hasta primer PR mergeado	$<$ 2 semanas
Documentación actualizada	Porcentaje de docs con fecha $<$ 6 meses	$>$ 80 %
Single point of failure humano	Número de cosas que solo 1 persona sabe	0
Tiempo para debuggear error medio	Horas desde detección hasta causa raíz	$<$ 2 horas

Estrategias para mitigar el costo cognitivo

1. Estandarización de patrones

- No tener 3 formas de hacer caching
- No tener 2 librerías de logging distintas
- Decisiones de arquitectura documentadas en ADR (Architecture Decision Records)

2. Capas de abstracción con propósito

Ejemplo de abstracción pobre frente a una limpia: en vez de instanciar un cliente exponiendo diez parámetros de configuración (`api_key`, `model`, `temperature`, `retry_policy`, `timeout`, `fallback_model`...), se expone una función de fábrica con intención clara —`create_code_review_agent(team_budget)`— que encapsula esos detalles.

3. Documentación viva

- Diagramas que se actualizan automáticos
- Ejemplos que se prueban en CI
- Decisiones registradas con contexto

4. Rotación de conocimiento

- Pair programming entre miembros del equipo
- "Incident reviews" sin culpas
- Documentación como requisito de PR

La regla del 80/20 cognitivo

El 20 % de los patrones se usan el 80 % del tiempo:

1. AI Gateway (siempre)
2. ReAct básico (casi siempre)
3. Caching (cuando hay > 100 llamadas/día)
4. Evals (siempre, pero simples).

El otro 80 % de los patrones (Kafka, Council, FAISS avanzado) se usan solo en casos específicos. **No los enseñes primero.**

Decisión de configuración frente a especialización

Situación	Estrategia
Equipo de 5-10 personas	Todos deben conocer los patrones básicos
Equipo de 10-30 personas	2-3 especialistas + resto saben lo básico
Equipo de 30+ personas	Equipo de plataforma dedicado

Regla: Si más del 30 % de tu equipo es "plataforma", estás sobreingenierando.

Ejercicio resuelto que añade corte por proyecto

Enunciado. Finanzas pide controlar el presupuesto por proyecto. Cuando un proyecto agota su presupuesto, el gateway no debe fallar de forma abrupta: según la política configurada, debe degradar al modelo económico y avisar.

Solución comentada. Tres cambios al gateway:

1. Agrega `project_id` y registro de presupuestos por proyecto.
2. Política explícita: `HARD_STOP` o `DEGRADE`.
3. En `DEGRADE`, forzamos *tier* fast y el resultado lleva `"degraded": True`.

La solución de referencia extiende el gateway del capítulo mediante la clase `AIGatewayWithProjects`, derivada de `AIGateway`. La implementación incorpora tres cambios, disponibles en el repositorio: registro de presupuesto por `project_id`; una política explícita (`HARD_STOP` corta la ejecución al 95 % de consumo, `DEGRADE` fuerza el modelo económico); y el resultado degradado se marca explícitamente (`"degraded": True`).

Arquitectura asíncrona como sistema de eventos

El AI Gateway que hemos construido hasta aquí es síncrono y bloqueante. Para operaciones atómicas de baja latencia (*triage* rápido, extracción simple, consulta puntual) funciona perfectamente. Pero cuando pasas a encadenar agentes o establecer un sistema multiagente con alta concurrencia, el modelo síncrono se convierte en un cuello de botella crítico.

Por qué falla el gateway síncrono ante tráfico real

Las API de LLM no son instantáneas:

- Peticiones simples: 1-3 segundos.
- Ciclos ReAct con 5-10 iteraciones: 10-30 segundos.
- Análisis de arquitectura con razonamiento extendido: 1-5 minutos.

Si tu *gateway* es síncrono (`def complete(...) -> dict`) y recibes 100 peticiones concurrentes que tardan 30 segundos cada una, el pool de conexiones colapsa: cien usuarios esperando terminan en *timeout* en cascada.

La solución: arquitectura orientada a eventos (Event-Driven). Los agentes no hablan por HTTP directo; consumen mensajes de una cola, procesan y publican resultados.

Arquitectura propuesta de Kafka como sistema nervioso

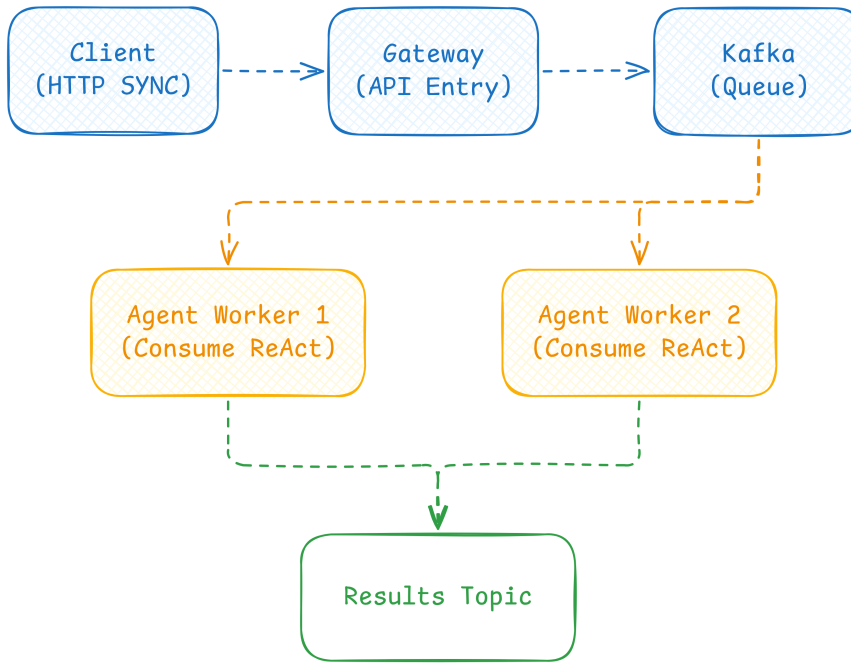


Figura 3.3: Arquitectura propuesta: Kafka como sistema nervioso

Implementación del gateway asíncrono

La implementación de referencia (repositorio del libro) define un `AgentTask` con estado (`PENDING`, `PROCESSING`, `COMPLETED`, `FAILED`, `TIMEOUT`) y payload; un `AsyncAIGateway` que publica cada tarea en Kafka y devuelve de inmediato un `task_id` para que el cliente consulte el resultado más

tarde (`submit_task` / `wait_for_result`); y un `AgentWorker` que consume esos eventos de Kafka, ejecuta el ciclo ReAct correspondiente y publica el resultado o el error. En producción, el cliente llama a `submit_task(...)` y luego espera el resultado —de forma bloqueante con `wait_for_result(task_id, timeout=...)`, o de forma no bloqueante suscribiéndose a un *webhook*.

> **Nota:** este patrón requiere Kafka como cola de mensajes (`pip install kafka-python`) y el servicio en ejecución. Es un ejemplo arquitectónico, no una implementación lista para producción sin configuración adicional (manejo de errores de red, serialización, reconexión...).

Cuándo usar síncrono vs asíncrono

La elección entre un *gateway* síncrono y uno asíncrono no es binaria ni definitiva. Depende del perfil de latencia, el volumen de peticiones y la criticidad del tiempo de respuesta. La tabla siguiente resume los casos típicos y la decisión recomendada.

Escenario	Llamada síncrona	Gateway asíncrono	Razonamiento
Extracción simple (< 3 s)	Adecuado	Overhead innecesario	La latencia es baja y el cliente espera; añadir una cola solo añade complejidad sin beneficio.
Triaje de tickets	Adecuado	Latencia extra	El usuario espera una respuesta rápida; el coste de la cola no se justifica.
ReAct con 5+ iteraciones	Timeout frecuente	Escala mejor	Los ciclos largos bloquean conexiones; Kafka permite procesar en *background*.

Escenario	Llamada síncrona	Gateway asíncrono	Razonamiento
Procesamiento en lote (1000 docs)	Imposible	Única opción	El tiempo de proceso excede cualquier *timeout* razonable; la asincronía es obligatoria.
Picos de tráfico	Colapsa	Cola gestiona picos	La cola actúa como amortiguador; los *workers* escalan horizontalmente.
Análisis de arquitectura (> 1 min)	Conexión cae	Procesa en *background*	El cliente no puede mantener una conexión HTTP abierta tanto tiempo.

Regla práctica: comienza siempre con una implementación síncrona para los casos de uso más simples. Solo migra a asíncrono cuando una tarea supere 10 segundos de forma recurrente o cuando el volumen de peticiones concurrentes empiece a generar *timeouts*. La migración debe hacerse de forma gradual, aislando primero los *endpoints* más pesados.

Nota: El *gateway* asíncrono con Kafka no es gratuito. Añade latencia en la entrega del resultado (el cliente debe hacer *poll* o recibir un *webhook*) y requiere infraestructura adicional. Úsalo solo cuando el beneficio en escalabilidad supere el coste operativo.

Caso de estudio del colapso de un gateway síncrono

La startup de e-commerce en Madrid colapsó su *Gateway* síncrono usado para triaje de tickets + generación de respuestas.

Incidente: - 5.000 tickets simultáneos durante el Black Friday. - Cada respuesta tomó 8-12 segundos. - 100 *workers* concurrentes × 10 segundos = 1000 conexiones activas. - *Pool* de conexiones agotado en 30 segundos. - 5.000 tickets en *timeout*, clientes enfurecidos.

Corrección:

1. Migrar a arquitectura asíncrona con Kafka.
2. *Workers* escalables: De 10 a 50 según crece la cola.
3. Los clientes reciben correo cuando la respuesta está lista.
4. Resultado: 0 *timeouts*, procesamiento en 15 minutos de 5K tickets.

Ahorro: 0 incidentes de colapso en 12 meses posteriores.

Límite de tokens en enjambres de agentes

El límite de tokens por sesión de enjambre debe realizarse deteniendo bucles de consumo de tokens. Cuando pasas de un agente ReAct aislado a un enjambre donde el Agente A habla con el Agente B, el peligro económico ya es el bucle de debate infinito entre agentes.

El problema de ciclos de ida y vuelta entre agentes

Escenario típico:

1. Agente A analiza un problema.
2. Envía su conclusión al Agente B para verificación.
3. Agente B encuentra ambigüedad y lo devuelve a A para aclaración.
4. Agente A responde, B vuelve a encontrar ambigüedad...
5. Resultado: 50 ciclos de conversación, $500 en tokens, ninguna decisión.

Patrón SwarmTokenOrchestrator con detector de ciclos

La implementación de referencia (repositorio del libro) combina dos piezas. `CycleDetector` examina los últimos mensajes intercambiados entre agentes buscando dos señales de bucle: pares de emisor/receptor que se repiten en patrón de ida y vuelta, y contenido casi idéntico (comparado por hash) entre mensajes consecutivos — cualquiera de las dos activa `cycle_detected`. `SwarmTokenOrchestrator` envuelve una sesión de enjambre con un presupuesto total de tokens y, en cada mensaje nuevo, decide `CONTINUE`, `DEGRADE` (al superar el 70 % del presupuesto) o `HALT` (al superar el 90 %, o al detectar un ciclo) — los umbrales de alerta y corte evitan tanto el bucle infinito de debate entre agentes como el costo sorpresa.

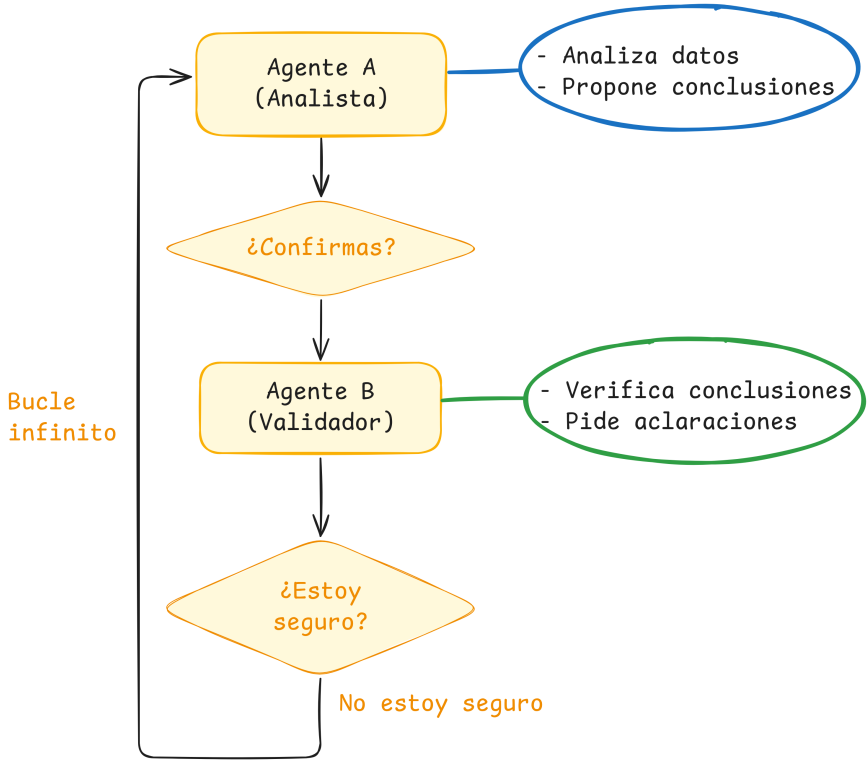


Figura 3.4: El problema: ciclos de ida y vuelta entre agentes

Trade-offs de la detección de ciclos

Las compensaciones (*Trade-offs*) de la detección de ciclos corresponden a decidir lo siguiente:

Más sensible = corta bucles reales, pero también conversaciones válidas. Menos sensible = dejas más libertad, pero consumes más en bucles.

Sensibilidad	Tasa falso positivo	Bucles que escapan	Cuándo usar
Alta (2 repeticiones)	15 %	5 %	Tareas críticas (finanzas, salud)
Media (3 repeticiones)	8 %	12 %	Producción general

Sensibilidad	Tasa falso positivo	Bucles que escapan	Cuándo usar
Baja (4+ repeticiones)	3 %	25 %	Exploración, brainstorming

Regla: Empezar con media (3 repeticiones). Ajustar después de 30 días de datos reales.

Invariante de presupuesto agregado

- Ninguna sesión de enjambre supera su presupuesto sin detenerse.
- Ningún bucle A2A consume >5 % del presupuesto mensual del equipo.

Recordemos el flujo de un orquestador de agentes:

El orquestador no sugiere, gobierna. Y gobierna porque aplica políticas, no porque da opciones. Si tu orquestador no hace cumplir esto, se queda en una simple sugerencia; no llega a ser un orquestador.

Dominio	Presupuesto máximo	Umbral	Métrica	Evaluador	Remedio si falla
Generación de arquitectura	500K tokens	0,70	Coherencia estructural	ArchValidator	Reintentar 2x, luego cambiar agente
Investigación de seguridad	300K tokens	0,80	Cobertura de CVE	SecScoreAPI	Reintentar 1x, luego escalar a una persona
Soporte técnico	30K tokens	0,85	Relevancia semántica	RAGAS	Rechazar directamente

Con esta tabla sabemos:

- Qué se mide (métrica)
- Quién lo mide (evaluador)
- Qué pasa si falla (remedio)

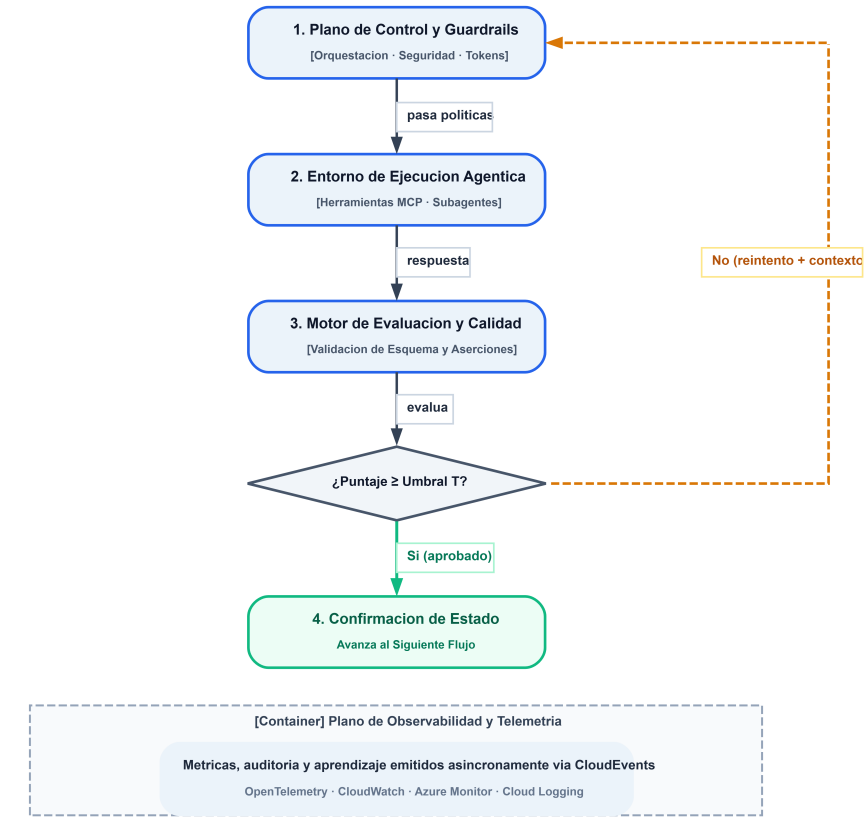


Figura 3.5: Flujo de un orquestador de agentes

Y pasamos de la sugerencia a la orquestación que es el gobierno.

Presupuesto máximo: Cantidad máxima de tokens permitidos para completar la tarea, incluyendo el consumo definido por la política de ejecución del orquestador.

Umbral de aceptación: Puntuación mínima que debe obtener el resultado validado para que el orquestador permita avanzar.

Métrica: Criterio de evaluación específico para ese dominio. El orquestador utiliza esta métrica para calcular la puntuación.

Evaluador: Componente o servicio que implementa la métrica y devuelve la puntuación. Debe estar disponible antes de iniciar cualquier ejecución.

Remedio si falla: Nos determina el criterio de qué hacer si falla:

- Arquitectura: fallar es caro, pero recuperable, por tanto 2 reintentos, luego otro agente
- Seguridad: fallar es peligroso, entonces 1 reintento, luego acudir al personal
- Soporte: fallar es barato, y por eso rechazar directamente sin gastar más

Si el resultado no alcanza el umbral, el orquestador debe rechazarlo, solicitar corrección, cambiar de agente o escalar a revisión humana.

Regla: Comienza conservador. Si el enjambre está siempre en HALT antes de llegar a conclusiones, aumenta el presupuesto. Si nunca supera el 50 % del presupuesto, lo configuraste demasiado alto.

Caso de estudio del enjambre que consumió demasiados tokens

El enjambre que consumió $800 en 10 minutos, se trata de una startup de Fintech en São Paulo. Enjambre de 3 agentes analizando transacciones sospechosas.

Configuración original (fallida): - Presupuesto: sin límite global. - Sin detector de bucles. - Los agentes se respondían libremente.

Incidente: - Agente A detectó transacción sospechosa. - Pidió verificación al Agente B. - Agente B pidió contexto al Agente C. - Agente C respondió... pero el resultado no le llegó a A. - A pidió

contexto de nuevo a C. - Ciclo repetitivo entre los agentes A, B y C consumió 847 mensajes, con un costo de 800 USD en tokens.

Corrección con SwarmTokenOrchestrator:

1. Presupuesto agregado de 200K tokens por transacción.
2. Detector de bucles con *threshold* 0,80.
3. *Fallback* a humano si HALT.
4. Resultado: 0 incidentes de bucle en 12 meses.

Ahorro: \$800 × ~2 incidentes/semana = \$64.000/mes menos un desastre.

Lo esencial del capítulo 3

La plataforma es producto:

- Expone IA como primitiva de primera clase.
- Gateway es crítico: enrutamiento, costo, presupuesto, auditoría.
- Sin *gateway* no hay inventario para el auditor.
- Las API comparten forma: el *routing* se decide con evaluaciones propias; se revisa trimestralmente.
- 4-6 ingenieros multiplican a 50-100: el ROI está en la visibilidad + automatización.
- Caso real: un CTO descubrió que el 60 % del presupuesto se gastaba en tareas nivel 0.
- Degradar en silencio es mentir: el resultado debe indicar degradación.
- Async vs Sync: síncrono para < 100 QPS; asíncrono con Kafka para colas de trabajo.
- Puente al Capítulo 10: cuando el *Gateway* se escala a múltiples equipos, Kafka se vuelve el *backbone* para la comunicación entre agentes.

Puente al Capítulo 10: cuándo migrar a Kafka

El Capítulo 3 mostró el AI Gateway en modo síncrono y asíncrono. Pero hay un punto de inflexión: Cuando pasas de 1 equipo a 5+ equipos usando los mismos agentes, necesitas Kafka. **Por qué:**

1. Acoplamiento temporal: HTTP sync requiere que productor y consumidor estén activos.
2. Reintentos: sin cola persistente, los reintentos son llamadas HTTP repetidas.
3. Auditoría: el log de Kafka es la fuente única de verdad de todas las interacciones.
4. Multi-consumidor: un mismo evento puede interesar a 3 agentes diferentes.

Criterio de migración a Kafka:

Señal	Acción
$>$ 500 tareas/día	*Gateway* asíncrono
$>$ 5000 tareas/día	Kafka obligatorio
Varios equipos	Kafka obligatorio
Auditoría	Kafka obligatorio
Tareas $>$ 5 minutos	Async con Kafka

El Capítulo 10 profundiza en la arquitectura basada en eventos con Kafka como bus central.

Criterio	Riesgo Bajo (NIVEL 4 / NIVEL 5)	Riesgo Medio (NIVEL 3)	Riesgo Alto/Crítico (NIVEL 1 / NIVEL 2)
Costo Financiero	$<$ \$10 por ejecución	\$100 - \$5,000	$>$ \$5,000 o transferencias directas
Reversibilidad	Instantánea (un comando git checkout o sobreescribir un caché)	Mitigable en minutos (Rollback de base de datos, apagar un servicio)	Catastrófica / Irreversible (Borrado de datos de clientes, infracción de regulaciones como HIPAA o GDPR)

Criterio	Riesgo Bajo (NIVEL 4 / NIVEL 5)	Riesgo Medio (NIVEL 3)	Riesgo Alto/Crítico (NIVEL 1 / NIVEL 2)
Impacto en Negocio	Interno, invisible para el usuario final.	Degradación temporal del servicio o retraso en entregas secundarias.	Caída del Core del negocio, demandas legales, exposición de llaves de API / PII.

Patrones fundamentales

De la cadena de texto al sistema cognitivo

Para un ingeniero de software tradicional, el primer choque mental consiste en entender que un agente moderno deja atrás al *chatbot* glorificado para convertirse en un **sistema cognitivo** con estructura interna: percepción, estado, razonamiento, acción y memoria, organizados alrededor de un modelo de lenguaje que opera como motor de inferencia. La arquitectura de referencia hereda los fundamentos de la literatura clásica de agentes (Russell y Norvig 2020), traduciéndose en cinco componentes de infraestructura cuando se implementa con modelos de lenguaje:

1. **El modelo (motor de razonamiento):** Un LLM accesible por API, intercambiable, parametrizable y versionado.
2. **El contexto (percepción y memoria de trabajo):** Instrucciones de sistema, historial de ejecución, salidas estructuradas de herramientas y fragmentos de conocimiento recuperado.
3. **Las herramientas (efectores):** Funciones tipadas con esquemas declarativos (como JSON Schema) que el modelo puede decidir invocar de manera autónoma.
4. **La memoria (persistencia):** Almacenamiento externo desacoplado (sistemas de archivos, bases de datos vectoriales o relacionales) que trasciende la vida útil de un único ciclo de inferencia.
5. **El arnés (*arnés*, el chasis):** El código imperativo que ejecuta el bucle de control, intercepta excepciones, valida esquemas, aplica políticas de seguridad y presupuesta recursos. Esta es la capa de control que el proveedor del modelo no proporciona y donde reside la resiliencia real de la aplicación.

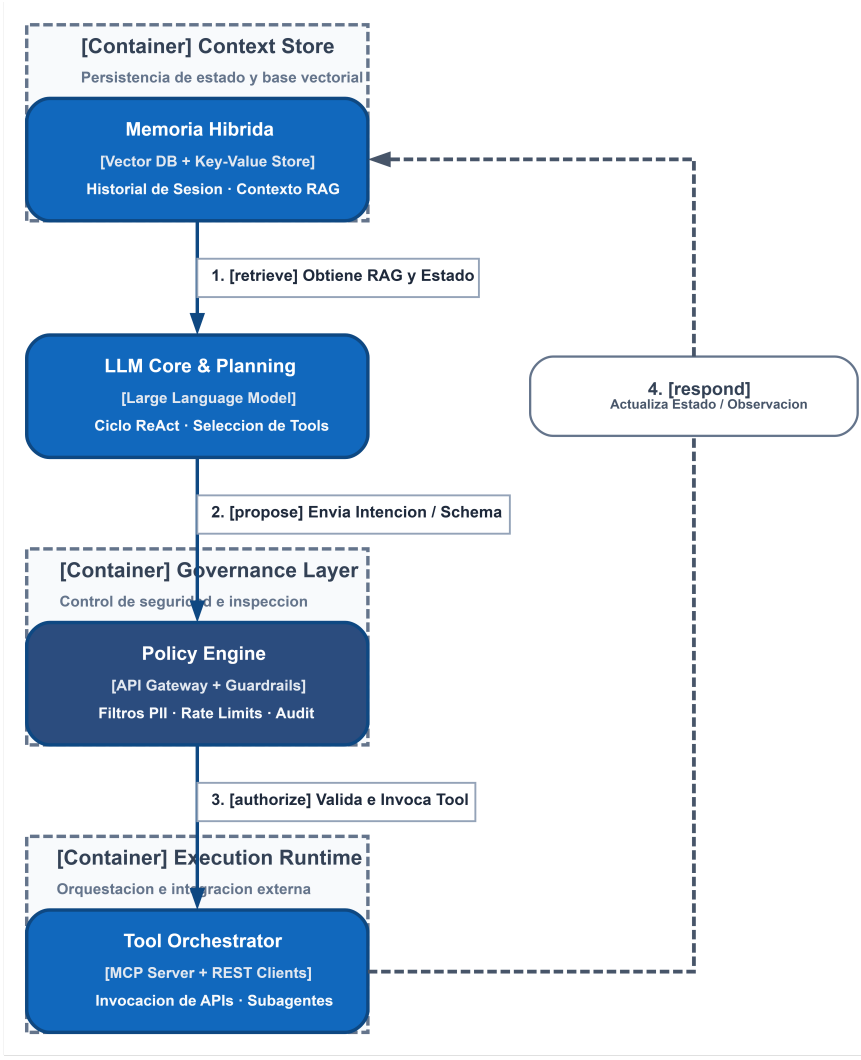


Figura 4.1: Arquitectura interna de un agente moderno

El ciclo ReAct que atiende el razonamiento que actúa

Un agente no destaca por la elocuencia de su texto, sino por su capacidad para intercalar el razonamiento explícito con la ejecución táctica hasta resolver un problema complejo (Yao, Zhao, et al. 2023). El patrón ReAct (*Reasoning and Acting*) formaliza este comportamiento estructurando la interacción en ciclos cerrados de tres fases secuenciales:

```
Pensamiento (Thought) → Acción (Action) → Observación (Observation)
```

- **Pensamiento:** El modelo evalúa el estado actual, desglosa los subobjetivos inmediatos y decide el siguiente paso lógico.
- **Acción:** El modelo selecciona una herramienta de su catálogo declarativo y genera los parámetros exactos para invocarla.
- **Observación:** El arnés ejecuta físicamente la herramienta, captura el resultado (o el mensaje de error) y lo inyecta de vuelta en la ventana de contexto del modelo para iniciar la siguiente iteración.

El peligro del bucle infinito y el despilfarro financiero

En entornos de producción, la autonomía del patrón ReAct introduce un vector de riesgo crítico: el **bucle de pensamiento infinito** (o *Agent Loop*). Si una herramienta devuelve un error inesperado, o si el modelo malinterpreta de manera consistente la salida de una función, el agente puede entrar en un ciclo redundante de reintentos idénticos. Sin barreras de contención imperativas, este comportamiento puede agotar las cuotas de API y generar costos imprevistos de cientos de dólares en cuestión de minutos.

Para mitigar este riesgo, es obligatorio que el arnés implemente un mecanismo de *Circuit Breaker* (cortocircuito) y un presupuesto de recursos estricto (*Token Budget*).

Implementación de un motor ReAct robusto

El siguiente ejemplo en Python muestra la implementación de un arnés de ejecución para un agente `ReActAgent`. El diseño separa la llamada pura de inferencia de la lógica imperativa que limita el consumo de

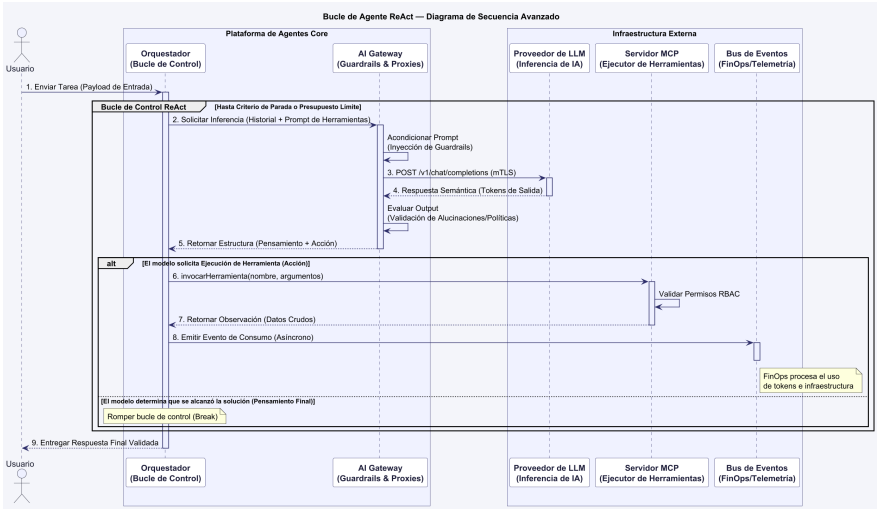


Figura 4.2: Secuencia temporal del ciclo ReAct

tokens y detiene proactivamente los bucles de repetición lógica. La implementación completa está en el repositorio del libro; en esencia: `ReActAgent.run()` ejecuta el ciclo pensamiento-acción-observación hasta un máximo de ciclos, y en cada iteración un `_should_stop` actúa como *Circuit Breaker* — aborta si los últimos tres pensamientos son más de 85 % similares entre sí (comparación con la distancia de Levenshtein, con una alternativa simple si la biblioteca no está instalada), señal de que el agente está atrapado en un bucle semántico y no de progreso real.

Cicatriz de Producción: La similitud de texto pura sobre el pensamiento (*thought*) es una primera línea de defensa útil, pero vulnerable en entornos dinámicos de producción. Si el LLM altera sutilmente la redacción (por ejemplo, agregando marcas de tiempo, IDs únicos de transacciones o modificando adjetivos), el porcentaje de similitud textual caerá por debajo del umbral del 85 %, burlando el cortocircuito analizado.

Un diseño robusto de grado industrial debe complementar la similitud de texto con una **firma de herramientas**: registrar las llamadas consecutivas y bloquear la ejecución si el agente invoca exactamente la misma herramienta con los mismos argumentos estructurados JSON más de dos veces sin generar cambios de estado medibles.

De regex a Structured Outputs

El otro punto frágil de la implementación de referencia es su `_parse_thought`: un regex sobre el texto `Thought/Action/Action Input` que el modelo *debería* producir, pero que rompe en cuanto el modelo cambia una coma, agrega una frase de cortesía o anida el JSON con una indentación distinta. La forma correcta de eliminar esa fragilidad no es escribir un regex más sofisticado, sino dejar de parsear texto libre: se define un esquema con Pydantic (`AgentStep`, con `thought`, `action` opcional y `action_input`) y se le pide al proveedor que **garantice** —no que "intente"— que la respuesta cumple ese esquema, en vez de reconstruirlo a mano con expresiones regulares. La variante resiliente (`ReActAgentResilient`, en el repositorio) reutiliza exactamente el mismo `_should_stop` como segunda capa de defensa, pero ya no depende de que el parseo por regex haya tenido éxito primero: el esquema garantiza estructura, el Circuit Breaker garantiza progreso.

Tree of Thoughts (ToT) con bifurcación y poda de hipótesis

Cuando el espacio de soluciones es complejo, linealizar el razonamiento (como hace ReAct estándar o Chain-of-Thought) suele resultar en callejones sin salida de inferencia. El patrón Tree of Thoughts (ToT) soluciona esto permitiendo que el sistema explore múltiples caminos de razonamiento en paralelo, evalúe el valor de cada rama de forma independiente y aplique algoritmos clásicos de búsqueda en árboles (como búsqueda en anchura o BFS, y búsqueda en profundidad o DFS) para encontrar la ruta óptima (Yao, Yu, et al. 2023).

El coste de la exploración heurística

El poder analítico de ToT tiene una contrapartida financiera y operativa muy alta. Al multiplicar las ramas del árbol, se multiplican de forma exponencial las llamadas al LLM. Un solo flujo ToT de profundidad 3 y ancho de bifurcación 3 puede requerir más de 15 llamadas de inferencia para resolver una sola consulta del usuario. Por este motivo, ToT **no debe ser la opción por defecto** en tareas estructuradas, lineales o deterministas. Debe reservarse exclusivamente para escenarios con alto costo de error donde el análisis predictivo, la planificación logística o la

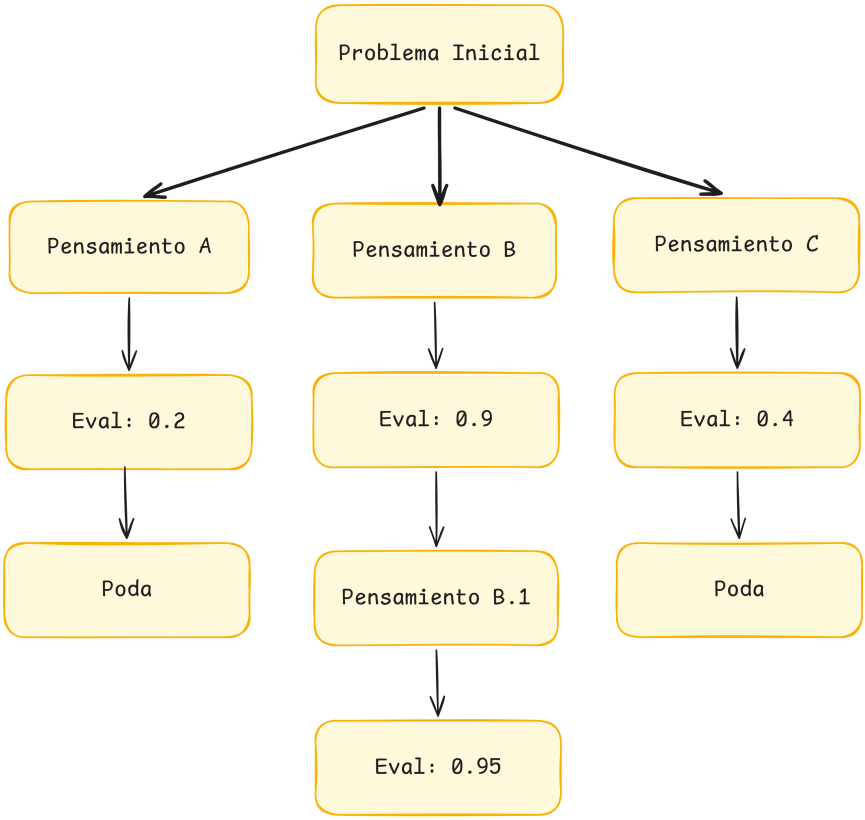


Figura 4.3: Bifurcación y poda de hipótesis

depuración de sistemas críticos justifiquen un incremento drástico en la latencia y en el consumo de tokens.

Implementación de un árbol de pensamiento determinista

La implementación de referencia (repositorio del libro) estructura la evaluación heurística y la selección del mejor nodo de razonamiento utilizando un modelo de lenguaje externo como evaluador cuantitativo: `TreeOfThoughts.solve()` parte de un nodo raíz y, en cada nivel de profundidad, pide al LLM generar varias hipótesis por nodo activo (`_generate_children`), puntúa cada una entre 0.0 y 1.0 (`_evaluate_thought`) y conserva solo las `beam_width` ramas mejor puntuadas (poda por *beam search*) antes de avanzar al siguiente nivel; al terminar, reconstruye el camino desde el mejor nodo hasta la raíz.

Cicatriz de Producción: El método `_generate_children` de referencia está fuertemente acoplado a la capacidad del modelo para seguir instrucciones de formato plano ("Opción [número]:"), parseadas con expresiones regulares. En entornos productivos con modelos de menor escala (como Llama-3-8B), esas expresiones regulares fallarán con frecuencia si el modelo cambia la sintaxis del prefijo.

Para dotar de verdadera resiliencia a este patrón, es imperativo forzar el uso de **Structured Outputs** (mediante JSON Schema o validación nativa con Pydantic) en las llamadas a la API de inferencia. Esto garantiza que el método reciba una estructura de datos limpia, fuertemente tipada y lista para ser parseada sin depender de *hacks* de expresiones regulares sobre texto libre.

En la práctica, esto significa reemplazar el contrato implícito ("el modelo debería responder con líneas *Opción N: ...*") por un esquema explícito que el proveedor valida antes de que el código lo vea: un `Hypothesis` con `thought` y un `score` acotado entre 0.0 y 1.0, agrupados en un `HypothesisBatch`. La variante resiliente (`TreeOfThoughtsResilient`, en el repositorio) reemplaza `_generate_children` para pedir directamente ese lote validado, sin negociar con un prefijo de texto plano. Nótese que `score` ya no llega como texto a convertir con `float(...)` dentro de un `try/except` silencioso: el esquema rechaza la respuesta en el momento de la validación si el modelo devuelve, por ejemplo, `"alto"` en vez de un número entre 0 y 1 — el error ocurre en el punto exacto donde se generó el dato malo, no tres funciones más abajo cuando algo intenta compararlo con un umbral.

Reflexion es la autocrítica evaluada

El patrón Reflexion introduce un ciclo de auto-corrección estructurado mediante la adición de un **evaluador objetivo e independiente** dentro de la arquitectura del agente (Shinn et al. 2023). En lugar de aceptar la primera respuesta generada por el generador, el resultado se somete a un agente crítico o a un conjunto de pruebas de validación automatizadas. Si no se cumplen los criterios de aceptación, el agente crítico devuelve un reporte detallado de errores que el generador utiliza como retroalimentación directa para corregir su propio código o solución.

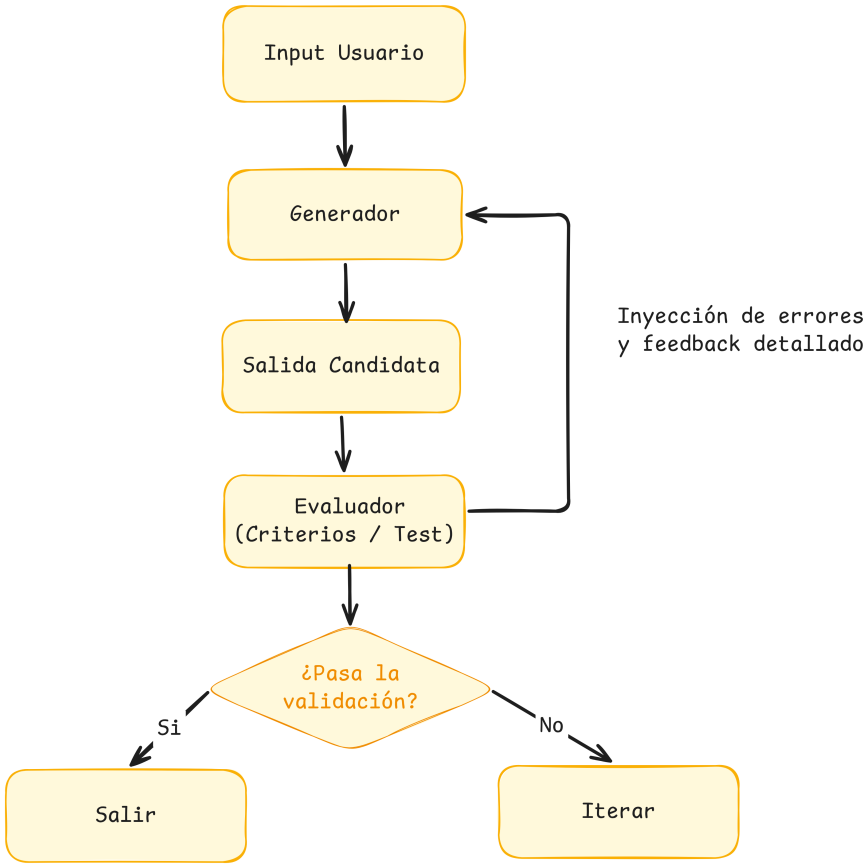


Figura 4.4: flujo de reflexion con autocrítica evaluada

La necesidad de un evaluador objetivo

La trampa común al implementar Reflexion es dejar que el mismo modelo evalúe su propio resultado en el mismo hilo de conversación. Los modelos de lenguaje tienden a sufrir de **sesgo de confirmación cognitivo**: si un modelo generó un error lógico en su primer intento, es altamente probable que valide su propio error como correcto si se le pregunta de forma directa dentro de la misma ventana de contexto. La resiliencia del patrón Reflexion depende de tres principios de diseño:

1. **Aislamiento de contexto:** El evaluador (crítico) debe ejecutarse en un entorno o hilo de conversación independiente, utilizando instrucciones del sistema diferenciadas y enfocadas únicamente en el control de calidad.
2. **Criterios de aceptación deterministas:** Siempre que sea posible, el evaluador debe incorporar validaciones de código tradicionales (como ejecuciones de tests unitarios, validadores de esquemas JSON o compiladores) en lugar de depender únicamente de evaluaciones heurísticas del LLM.
3. **Retroalimentación accionable:** La retroalimentación enviada de vuelta al generador no debe ser un mensaje genérico de error (ej. "El código falló"). Debe incluir el rastro exacto de la pila de llamadas (*stack trace*), los *inputs* de prueba que causaron el fallo y los resultados esperados frente a los obtenidos.

Comparativa de patrones fundamentales

A continuación se presenta un análisis comparativo de los tres patrones principales bajo criterios estrictos de viabilidad en entornos de producción:

Patrón	Latencia Relativa	Costo en Tokens	Casos de Uso Recomendados	Complejidad del Arnés
ReAct	Baja (Lineal, 3-5 pasos)	Moderado	Automatización de flujos de trabajo, orquestación de herramientas, consultas dinámicas de APIs.	Media (Requiere control de estado y *parsers* de expresión regular).
Tree of Thoughts	Alta (Exponencial por nivel)	Muy Alto (Múltiples ramificaciones en paralelo)	Planificación estratégica, diseño arquitectónico complejo, optimización de algoritmos complejos.	Alta (Requiere gestión de árboles de decisión y búsquedas estructuradas).
Reflexion	Media-Alta (Determinada por ciclos de corrección)	Alto (Inferencia repetida con contexto acumulativo)	Generación de código fuente, creación de *pipelines* de datos, automatización de pruebas de integración.	Media-Alta (Requiere desacoplamiento del generador/crítico e integración de *testing* externo).

La evolución hacia sistemas de ingeniería

Los componentes individuales que acabamos de diseccionar (motores de inferencia, contextos enriquecidos, arneses imperativos y herramientas declarativas) no proporcionan robustez operativa por sí solos. De la misma forma en que conocer la sintaxis de las clases en un lenguaje de programación no te convierte en arquitecto de software, entender la API de OpenAI o Anthropic no te capacita para diseñar sistemas agénticos de nivel empresarial. El software de producción exige patrones de diseño sistemáticos. No podemos construir arquitecturas distribuidas utilizando *prompts* gigantescos e impredecibles como único controlador del flujo de

ejecución. El salto real de componentes a sistemas de ingeniería ocurre cuando encapsulamos la flexibilidad cognitiva del modelo de lenguaje dentro de interfaces estables, límites de estado predecibles y presupuestos financieros rígidos.

Los principios para la era no determinista

El código limpio tradicional asume determinismo: a un input X, el sistema siempre responde con el output Y. En el desarrollo de software clásico, el flujo es predecible y la lógica está codificada de forma explícita en el backend. En la ingeniería de sistemas agénticos introducimos el **Código Sintético**: código generado, interpretado u orquestado de forma autónoma por un modelo de lenguaje en tiempo de ejecución. Aquí, el motor de ejecución es probabilístico. Para guiar esta impredecibilidad sin estrangular la flexibilidad del modelo, adaptamos los principios tradicionales de Clean Code bajo tres nuevas directrices:

1. **Aislamiento de la lógica determinista:** El código que interactúa con bases de datos, APIs de terceros o sistemas de archivos debe residir exclusivamente en el arnés imperativo (*runtime wrapper*). El modelo de lenguaje jamás debe ejecutar lógica de negocio de manera directa; solo debe emitir la intención de ejecución mediante llamadas estructuradas a herramientas (JSON Schema).
2. **Contratos semánticos estrictos:** Las entradas y salidas de cada nodo agéntico deben pasar por capas de validación física (tipado estricto con Pydantic o TypeScript) antes de que el siguiente componente del *pipeline* intente procesarlas. Si el modelo alucina en la estructura del *output*, el error debe interceptarse y resolverse en los límites del componente (mediante autocrítica de ciclo cerrado), impidiendo la propagación del estado corrupto.
3. **Idempotencia en herramientas destructivas:** Si un agente tiene acceso a herramientas capaces de alterar el estado del sistema (como modificar un registro en base de datos o emitir un pago), el diseño de la herramienta debe garantizar la idempotencia. Si el agente repite la acción debido a un ciclo de reintento (*retry loop*), el sistema debe neutralizar los efectos colaterales de las llamadas duplicadas.

SOLID como interrogatorio arquitectónico

Esta reflexión parte de una publicación técnica realizada en LinkedIn en 2024: *Principios SOLID y refactorización, ¿cómo los aplico correctamente?* (Pérez Ruiz 2024).

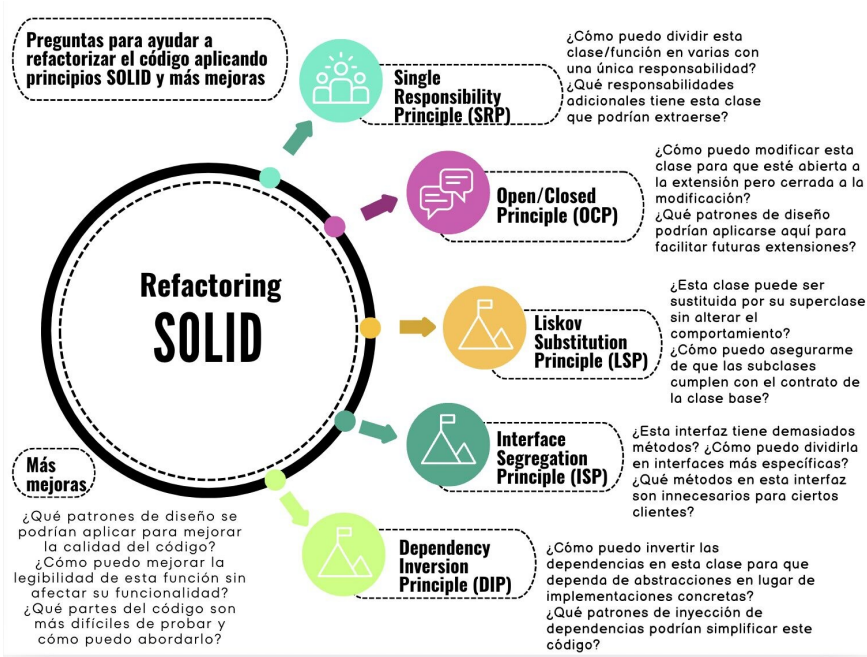


Figura 4.5: Principios SOLID y refactorización, ¿cómo los aplico correctamente?

El objetivo original de aquella publicación era explorar cómo aplicar principios SOLID durante procesos de refactorización en equipos ágiles, especialmente mediante preguntas de diseño que ayudaran a tomar mejores decisiones arquitectónicas. En ese contexto señalaba: > La refactorización debe realizarse de forma controlada, aislando componentes, módulos o APIs cuando sea necesario, documentando los cambios y las razones detrás de cada decisión. Aunque aquella publicación no estaba orientada a sistemas basados en inteligencia artificial, el principio subyacente permanece vigente: una arquitectura robusta comienza con preguntas correctas. Los principios SOLID, creados originalmente para programación orientada a objetos, pueden reinterpretarse hoy como un marco de evaluación arquitectónica para sistemas agénticos. el enfoque radica en cuestionar responsabilidades,

dependencias, extensibilidad y límites antes de desplegar agentes en producción:

S: Principio de Responsabilidad Única (*Single Responsibility Principle*)

El interrogatorio: ¿Este agente está intentando procesar la llamada, clasificar la intención, formatear la salida y ejecutar el *script* al mismo tiempo? **La traducción agéntica:** Un *prompt* de sistema no debe superar un único dominio de competencia. Si un agente es responsable de validar la entrada, razonar sobre el problema y formatear la respuesta, divida el sistema. Diseñe un *pipeline* donde un clasificador de un solo token filtre el flujo, un modelo especializado procese el razonamiento y un estructurador ligero parsee el resultado.

O: Principio de Abierto/Cerrado (*Open/Closed Principle*)

El interrogatorio: ¿Necesito modificar el *prompt* principal de sistema de mi agente cada vez que añado una nueva herramienta o API al ecosistema? **La traducción agéntica:** El agente debe estar abierto a la extensión de capacidades pero cerrado a la modificación de su *prompt* base. Logramos esto exponiendo esquemas de herramientas dinámicos (como inyectar dinámicamente un array de esquemas JSON en la llamada de la API de *chat*) en lugar de documentar las herramientas de forma fija en la directiva de sistema (*system prompt*).

L: Principio de Sustitución de Liskov (*Liskov Substitution Principle*)

El interrogatorio: ¿Si cambio mi proveedor de LLM en el *backend* (por ejemplo, de GPT-4o a un modelo local cuantizado como Llama-3 8B), el sistema completo colapsa por problemas de parseo o pérdida de adherencia? **La traducción agéntica:** Cualquier *backend* de inferencia debe ser sustituible sin alterar el comportamiento esperado del sistema. En producción, esto es sumamente complejo debido al acoplamiento semántico (la sutil variación de vocabulario en la que un modelo interpreta las instrucciones). Para cumplir este principio, el arnés de control debe implementar adaptadores de *prompt* específicos por modelo, asegurando que la interfaz expuesta a la lógica de negocio permanezca intacta.

I: Principio de Segregación de Interfaces (*Interface Segregation Principle*)

El interrogatorio: ¿Le estamos pasando la especificación de nuestras 50 herramientas empresariales en la ventana de contexto a un agente que solo necesita dos de ellas para resolver su tarea actual? **La traducción agéntica:** No abrume al modelo de lenguaje con contextos innecesarios. Un exceso de herramientas en el *prompt* degrada la precisión de la atención (*attention decay*), aumenta la latencia y dispara el costo de procesamiento. Segregue las capacidades inyectando selectivamente herramientas a través de un enrutador semántico de herramientas (*tool router*) que solo exponga las opciones relevantes basadas en la tarea activa.

D: Principio de Inversión de Dependencias (*Dependency Inversion Principle*)

El interrogatorio: ¿Nuestra lógica de negocio depende directamente de las APIs propietarias o de los SDKs de un proveedor de modelos específico? **La traducción agéntica:** Dependa de abstracciones, no de implementaciones concretas. La lógica del sistema agéntico debe comunicarse con un *Gateway* unificado de IA. Las llamadas de inferencia no se hacen directo al SDK del proveedor, sino a través de una interfaz desacoplada. Esto facilita el balanceo de carga, el enrutamiento inteligente por costo y los mecanismos de *fallback* automáticos si la API principal experimenta fallas o degradación del servicio.

Lo esencial del capítulo 4

ReAct no es un *script*: Consiste en ciclos continuos de pensamiento, acción y observación estrictamente controlados por un *circuit breaker* en el arnés de control.

- **El bucle de ejecución infinito es real:** Sin límites rígidos en los ciclos del agente o un presupuesto estricto de tokens, un agente atrapado en una excepción semántica puede facturar miles de dólares en minutos.
- **El Circuit Breaker es obligatorio:** Debe implementarse un detector de repetición de estados en el arnés. Si el agente repite

exactamente los mismos pensamientos y acciones tres veces consecutivas, el ciclo debe abortarse de inmediato.

- **Tree-of-Thoughts es para exploración compleja:** No utilice arquitecturas de árbol para flujos de trabajo lineales. Su ejecución cuesta hasta un 200 % más en inferencia y tiempo de respuesta.
- **Reflexion necesita un *eval* objetivo:** Sin criterios de aceptación deterministas y externos, el agente no puede discernir si sus intentos de corrección están mejorando el resultado o simplemente consumiendo tokens.
- **De componentes a patrones:** Los agentes robustos solo se construyen organizando las piezas agénticas mediante patrones arquitectónicos y principios de diseño tradicionales heredados de la ingeniería de software.
- **El Código Limpio Sintético:** Principios clásicos de desarrollo de software reinterpretados para aislar la aleatoriedad probabilística del modelo del *backend* determinista del negocio.
- **SOLID como interrogatorio:** Una guía sistemática para auditar la arquitectura de tus agentes, garantizando la modularidad, escalabilidad y resistencia a fallos antes del despliegue real en producción.

El arnés del agente

El código que el proveedor no te da

Compras un modelo y luego configuras la clave de API. Escribes el *prompt*. El modelo responde. Listo, ¿cierto? No. Terminado es cuando puedes responder en menos de cinco minutos: - Cuánto gasté hoy en tokens. - Qué decisión tomó el agente y por qué. - Si el *output* fue corrompido por un *prompt injection*. - Por qué falló la tarea precedente. Si no tienes esas respuestas, no tienes un sistema. Tienes un chatbot con facturación invisible. El arnés del agente es todo lo que rodea la llamada al LLM. Es el código que el proveedor (Anthropic, OpenAI, Google) no te da porque no le importa tu auditoría, tu presupuesto ni tu seguridad.

Componentes del arnés

Presupuesto de tokens con sobregiro controlado. Idealista es fijar un límite y fallar si se excede. Realista es permitir un sobregiro controlado con notificación escalonada: la implementación de referencia (`TokenBudget`, en el repositorio del libro) no es un binario OK/fallo, sino un espectro de cuatro estados (`OK`, `WARNING` al 70 %, `CRITICAL` al 90 %, `PROHIBITED` al 100 %) que se degrada progresivamente en vez de cortar en seco.

Circuit Breaker cuando el agente se atasca

El Circuit Breaker separa un sistema que se recupera de uno que factura ilimitadamente mientras falla. Detecta dos patrones de bucle: **repetición de pensamiento** (mismo *hash* de razonamiento N ciclos seguidos) e **inundación de herramientas** (misma herramienta invocada de forma desproporcionada en una ventana reciente). La implementación de referencia (`AgentCircuitBreaker`, con solo biblioteca estándar: `collections.deque`, `hashlib`, `time`) pasa por tres estados:

- `closed`: funcionamiento normal.

- `open`: el agente está bloqueado; no se permiten nuevas ejecuciones.
- `half_open`: después de 60 segundos, permite un reintento para verificar si el problema se ha resuelto.

El log que un auditor entiende

Si tu log dice "agent_processed_request", no tienes auditoría. Tienes un mensaje vacío. La implementación de referencia (`AgentAuditLogger` + `AuditEvent`, en el repositorio del libro) construye un log inmutable: cada evento enlaza y firma (HMAC-SHA256) el hash del evento anterior, formando una cadena verificable. Se verifica releyendo el archivo y recalculando el hash de cada evento con `prev_event_hash` igual al `own_hash` del anterior: cualquier discrepancia delata una manipulación posterior. El logger también expone `export_audit_report()`, que agrega costo total, tokens totales y aprobaciones humanas de una sesión completa para auditoría externa.

El arnés orquesta presupuesto, breaker y auditoría

El arnés integra las tres piezas anteriores en un único punto de entrada: antes de ejecutar el agente verifica presupuesto y *circuit breaker*; después, registra en auditoría. La implementación de referencia (`AgentHarness`, en el repositorio del libro) hace un *preflight check* (bloquea si el presupuesto o el breaker ya lo impiden), ejecuta el agente, calcula el costo real a partir de los tokens consumidos, registra el ciclo en el breaker y el evento en el log de auditoría, y devuelve un `AgentResult` con el contenido, costo y presupuesto restante — o un resultado bloqueado con la razón, si el *preflight* falló.

El agente que refactoriza su propio código

Los agentes generan código que se acumula y degrada. El patrón self-cleaning encadena tres roles especializados, **Analyzer**, **Refactor**, **Tests**, en un bucle que se detiene solo cuando el código queda limpio o los tests fallan. La implementación de referencia (`SelfCleaningCodeLoop`, en el repositorio del libro) itera hasta tres veces: el `Analyzer` detecta *code smells* (`DUPLICATE_CODE`, `LONG_FUNCTION`, `DEAD_CODE`...) vía LLM, el `Refactor` aplica la corrección correspondiente sin cambiar el comportamiento, y `Tests` genera pruebas de regresión que deben seguir pasando — si fallan,

el bucle se detiene y reporta `tests_failed` en vez de devolver código roto silenciosamente. Cada colaborador es un agente delgado: un `SYSTEM_PROMPT` especializado más una llamada al LLM con salida estructurada. **Caso real, Fintech en Londres:** tras 3 meses de agentes generando código (12K líneas, 40 % de duplicación), implementaron este bucle tras cada generación. Resultado a 1 mes: duplicación −60 %, *onboarding* de 3 semanas a 1, complejidad ciclomática −35 %, bugs en producción sin cambios (los tests no fallaron). El *caveat* no es opcional: **un humano revisa cada refactorización**, el agente no entiende el contexto de negocio. Una variante del arnés (`AgentHarnessWithSelfCleaning`) detecta automáticamente si la respuesta del agente contiene bloques de código y, de ser así, los pasa por este bucle antes de devolver el resultado.

Trazabilidad y registro estructurado

Para sistemas agénticos de producción

> ■ **PISTA EJECUTIVA**
>
> Cuando un incidente ocurre en producción, y más en sistemas agénticos, donde las decisiones son probabilísticas, la diferencia entre 10 minutos de diagnóstico y 10 horas de frustración está en la calidad de los registros. Un log estructurado no es un lujo: es la única forma de reconstruir por qué un agente decidió lo que decidió. Esta sección extiende el arnés (Capítulo 5) y la auditoría con un enfoque de producción.

Caso de estudio de depuración de errores

Fueron 3 horas de debugging por un log mal hecho. **Empresa:** fintech en Madrid. **Sistema:** agente de aprobación de préstamos. Un préstamo de €15.000 fue aprobado para un cliente con historial crediticio negativo. El log completo del incidente decía:

```
2026-07-21 14:23:45 ERROR: Loan approval failed. Check logs.
```

No decía qué agente tomó la decisión, qué prompt usó, qué herramientas invocó ni cuál fue su razonamiento. El equipo tardó 3 horas en descubrir que el agente había usado un prompt desactualizado que ignoraba el historial crediticio. Con logs estructurados, `correlation_id`, `agent_id`,

`prompt_version`, cadena de razonamiento y `cost_usd`, el mismo incidente se diagnostica en 5 minutos: esa proporción, 36×, es el retorno real de invertir en trazabilidad antes de necesitarla.

Qué información es crítica para un log agéntico

No todos los atributos pesan igual. Estos son los que permiten reconstruir cualquier decisión:

Categoría	Atributos clave	Para qué sirven
Identificación	`correlation_id`, `session_id`, `agent_id`, `agent_version`, `team_id`	Rastrear una tarea a través de múltiples agentes y saber a quién facturarla
Contexto de decisión	`system_prompt_hash`, `model_used`, `input/output_tokens`	Saber qué versión y qué modelo generó el resultado
Razonamiento (CoT)	`reasoning_chain`, `actions_taken`, `confidence_score`	Reconstruir el "por qué", no solo el "qué"
Estado de ejecución	`status`, `latency_ms`, `retry_count`, `budget_used`	Diagnosticar rendimiento y costo
Seguridad	`pii_detected`, `human_approval_required`, `trust_score`	Auditoría de cumplimiento

De texto plano a JSON estructurado

Un log de texto plano como `INFO: Loan approved` no se puede filtrar, correlacionar ni auditar. Su equivalente estructurado, en formato JSON Lines (una línea = un evento, ideal para Datadog, ELK o Splunk), se ve así:

```
{"timestamp":"2026-07-
21T14:23:50.345Z","level":"INFO","correlation_id":"task-20260721-
143245-
abc123","event_type":"decision","decision":"approved","rationale":"0
impagos en 36 meses, riesgo
bajo","confidence":0.92,"trust_score":0.85,"cost_usd":0.0432}
```

Cada campo responde una pregunta de auditoría distinta: `correlation_id` ata el evento a la tarea completa, `rationale` explica el "por qué" en lenguaje natural, `trust_score` es el número que decide si la salida necesita revisión humana.

La pieza mínima es el registro estructurado

Toda la implementación gira en torno a una única estructura de datos y un logger delgado que la serializa. Esto es lo esencial; el resto, ocho métodos de conveniencia (`reasoning_step`, `tool_result`, `decision`, `error`...), sanitización automática de secretos y agregación de métricas por sesión, sigue exactamente el mismo patrón y está en el repositorio del libro. `StructuredLogEntry` define un evento como una línea JSON (`timestamp`, `level`, `correlation_id`, `event_type`, `agent_id`, y campos opcionales de costo/latencia); `StructuredAgentLogger` es el logger de producción que la serializa a JSON Lines y sanea automáticamente claves sensibles (`api_key`, `password`, `token`...) antes de escribir.

Integración en el ciclo ReAct

El logger no se llama "por fuera" del agente: se invoca en cada transición del ciclo pensar→actuar→observar, de modo que ningún paso queda sin traza. La variante instrumentada (`ReActAgentWithLogging`, en el repositorio del libro) registra cada pensamiento con `reasoning_step`, cada herramienta invocada con su resultado, y al concluir, una `decision` final con `confidence`, `trust_score` y costo acumulado — firma completa, manejo de errores y reintentos en el notebook `codigo_sintetico_trazabilidad.ipynb` del repositorio.

Query para diagnóstico de incidentes

Con logs estructurados en tu plataforma de observabilidad, una pregunta como "¿qué decisiones tomó este agente con confianza baja hoy?" se responde en segundos:

```
SELECT timestamp, correlation_id, payload.decision,
payload.trust_score, cost_usd
FROM logs
WHERE event_type = 'decision'
  AND agent_id = 'loan-approval-agent-v2.1'
```

```
  AND payload.trust_score < 0.7
  AND timestamp >= NOW() - INTERVAL 1 DAY
ORDER BY timestamp DESC;
```

El repositorio incluye variantes para reconstruir una cadena de razonamiento completa por `correlation_id` y para agregar costo por agente.

Dashboard mínimo de trazabilidad

Panel	Métrica	Fuente
Trazabilidad	Búsqueda por `correlation_id`	Todos los logs
Razonamiento	Grafo de pensamientos	`reasoning_step`
Herramientas	Tasa de éxito	`tool_result`
Costos	Costo por agente/sesión	`agent_finish`
Confianza	Distribución de TrustScore	`decision`

Lo esencial de esta sección

1. Un log de texto plano no sirve en sistemas agénticos: sin `correlation_id` ni estructura, no puedes reconstruir el "por qué" de una decisión.
2. El log estructurado es JSON Lines: una línea por evento, lista para Datadog, ELK o Splunk.
3. El logger es parte del arnés, no un añadido posterior: se invoca en cada paso del ciclo del agente.
4. El caso de la fintech resume el argumento completo: 3 horas de debugging se convierten en 5 minutos.

Ejercicio práctico

Toma el agente de revisión de código del Capítulo 12 y añade `correlation_id`, `reasoning_step` por cada pensamiento, `tool_result` por

cada herramienta invocada, y una `decision` final con `rationale` y `trust_score`. La solución comentada está en el repositorio oficial del libro.

> **Nota:** la implementación completa, logger, agente instrumentado, herramientas de ejemplo e integración con Datadog/ELK/OpenTelemetry, está en el repositorio del libro como notebook ejecutable.

Lo esencial del capítulo 5

El arnés es el código que el proveedor no da: presupuesto, *breaker*, auditoría, trazabilidad. - El presupuesto ideal es con umbrales: 70 % *warning*, 90 % *critical*, 100 % prohibido. - Circuit Breaker detecta bucles: mismo *hash* de pensamiento = atascado; misma herramienta 4 veces = inundación. - Auditoría es *append-only*: cadena de *hashes*, marca de tiempo UTC, firmado con HMAC. - Trazabilidad es JSON Lines con `correlation_id`, integrada en el propio ciclo del agente, no añadida después. - El *arnés* orquesta todo: antes de ejecutar, verifica presupuesto y *breaker*; después, registra y audita.

La variable de confianza

La fórmula que faltaba

Cuando un agente falla en producción, nadie pregunta por su F1-score. Preguntan por qué nadie lo verificó a tiempo. La confianza no es el F1-score. La industria habla de "trust scores" como si fueran métricas de modelo. Confianza no es *accuracy*. Confianza es calibración entre capacidad del agente y costo del error en tu dominio. Un agente con un 94 % de precisión puede corromper tu base de datos en producción. La pregunta correcta no es "¿qué tan bueno es el agente?" La pregunta correcta es "¿qué tan caro es que falle en este contexto específico?"

La ecuación reformulada

La fórmula clásica de confianza (Maister, Green, y Galford 2000) proviene del *consulting*:

```
Confianza = (Competencia × Fiabilidad × Asequibilidad) / Secreto
```

Es insuficiente para agentes porque:

1. "Secreto" no existe: en sistemas LLM, todo es observable (o debería serlo).
2. Falta verificabilidad: ¿puedes verificar el *output* en 5 minutos?
3. Falta costo del error: fallos que cuestan $10 frente a $1M no son iguales. La ecuación reformulada para agentes:

```
Confianza = (Competencia × Verificabilidad × Trazabilidad) / (Costo del
error)
```

Competencia: *proxy* de capacidad técnica (F1-score, precisión en *eval-set*, tasa de éxito en tests). **Verificabilidad:** ¿qué tan rápido puedes detectar un error? 0 = *black box*, 1 = verificación en segundos,

auditoría automatizada. **Trazabilidad:** ¿puedes reconstruir por qué tomó esa decisión? 0 = razonamiento interno, 1 = *chain-of-thought* completo registrado. **Costo del error:** impacto en tu negocio si el agente falla (dinero, reputación, seguridad, cumplimiento).

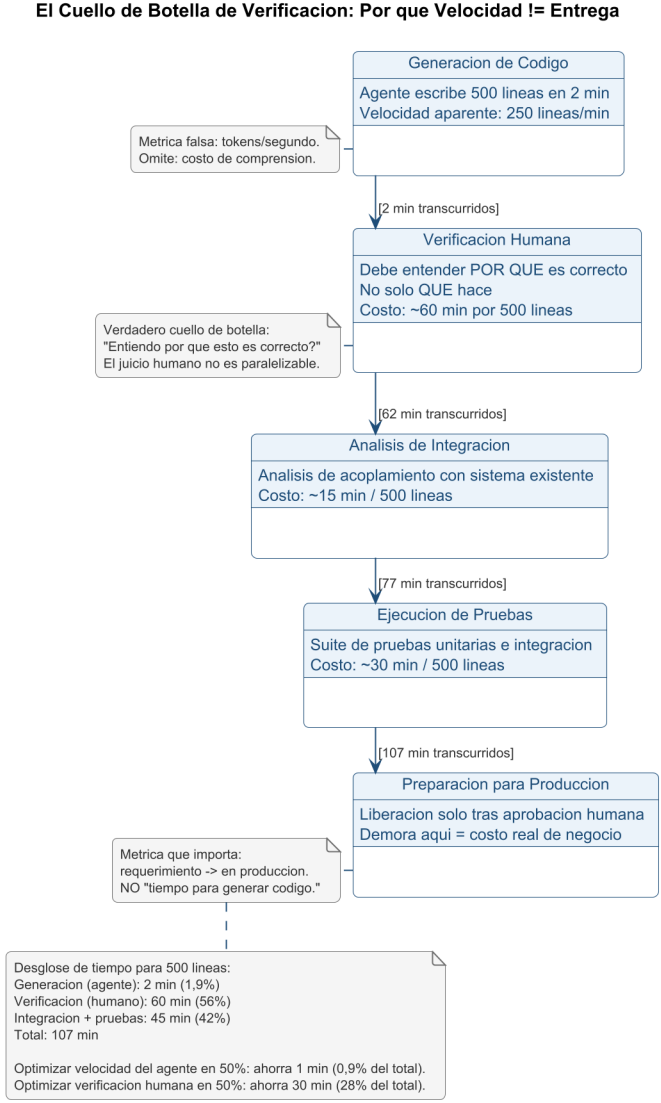


Figura 6.1: El cuello de botella de verificación sin barandillas de seguridad (*guardrails*)

Cálculo numérico

TrustScore no es binario ("confío / no confío"): es una función continua que determina qué nivel de autonomía es seguro. La implementación de referencia (`TrustScore`, en el repositorio del libro) calcula `competence_proxy × verifiability × traceability / error_cost` y clasifica el resultado en cinco niveles (`NO_TRUST` a `CRITICAL_TRUST`).

Ejemplo práctico

Dos casos ilustran por qué el costo del error domina la ecuación por encima de la habilidad del modelo:

- **Agente de triaje de tickets:** 91 % de competencia, verificabilidad baja (0,3 — solo *spot checks*, no auditoría completa), trazabilidad alta (0,9 — CoT registrado), costo de error bajo (0,5, en escala 1-100). *Trust score* resultante: 0,049 → `MEDIUM_TRUST`, con autonomía operativa segura.
- **Agente que ejecuta despliegues:** 94 % de competencia (mejor que el anterior), verificabilidad alta (0,9 — *testing* automatizado), trazabilidad completa (1,0), pero costo de error muy alto (80 — un fallo cuesta ≈$500K en *downtime* y *rollback*). *Trust score* resultante: 0,0106 → `LOW_TRUST`, sin autonomía segura para despliegue directo a producción.

Conclusión: aunque el agente de despliegue es técnicamente mejor (94 % vs 91 %), el costo del error lo hace *menos* confiable para autonomía directa que el agente de triaje.

La variable de verificabilidad

La competencia ya se mide (F1-score, *accuracy*, etc.). La trazabilidad se obtiene con CoT. Pero verificabilidad es la variable que más equipos olvidan. La verificabilidad no pregunta "¿puedo verificar?", sino "¿cuánto tarda y cuánto cuesta verificar cada *output*?"

Tiempo de verificación	Verificabilidad	Ejemplo
$<$ 10 segundos	0,9-1,0	Test automatizado
1-5 minutos	0,6-0,8	Revisión humana rápida

Tiempo de verificación	Verificabilidad	Ejemplo
10-30 minutos	0,3-0,5	Revisor humano detallado
$>$ 1 hora	0,1-0,2	Requeriría investigación extensa
No verificable	0,0	Output único, no hay ground truth

Caso de estudio del agente que fallaba en silencio

Durante 6 meses una empresa de *e-commerce* automatizó descuentos con un agente. Configuración:

- F1-score: 0,92 (bueno)
- Trazabilidad: 0,8 (CoT parcial)
- Verificabilidad: 0,0, aquí está el problema ¿Por qué 0,0? El *output* del agente era "descontar 15 % en SKU X". Para verificar, se tenía que:

1. Ir al *dashboard* de promociones.
2. Buscar el SKU.
3. Verificar que el descuento se aplicó correctamente.
4. Confirmar que no hubo doble descuento. Tarea de 20 minutos. Nadie lo verificaba.

El agente comenzó a aplicar descuentos del 95 % por un error en el *prompt*. **Pérdida:** \$340K en 6 meses antes de que Finanzas se diera cuenta. **Corrección:** se añadió un *hook* post-ejecución que:

1. Consulta automáticamente la base de datos después de cada descuento.
2. Compara con lo esperado.
3. Si hay discrepancia $>$5 %, dispara alerta inmediata. La verificabilidad subió de 0,0 a 0,85. La confianza se calculó correctamente.

Patrón Shadow Agent

El Shadow Agent es un agente paralelo que no actúa, solo valida. El enfoque radica en un validador especializado en la razón por la que podría fallar. La implementación de referencia (`ShadowAgent`, en el repositorio del libro) ofrece tres estrategias intercambiables:

- `adversarial`**:** pide al LLM que intente *refutar* la solución (enfoque red-team) — busca supuestos no verificados, casos límite, inconsistencias y vulnerabilidades; si no encuentra nada, aprueba.
- `consistency`**:** en vez de atacar la solución, pregunta si el resultado realmente se deriva del razonamiento presentado.
- `constraints`**:** verifica reglas de negocio obligatorias ("nunca más de 50 % de descuento", "siempre cumplir RGPD") una por una contra el resultado, y rechaza si alguna se viola.

Cada validación devuelve un veredicto (`APPROVE`, `REJECT` o `FLAG_HUMAN`), un razonamiento, una confianza y los riesgos detectados. Si el LLM no logra emitir un veredicto parseable, el Shadow degrada por defecto a `FLAG_HUMAN` — nunca aprueba en silencio ante una falla de su propio parseo.

Shadow Agent como gate de producción

La implementación de referencia (`ProductionPipeline`, en el repositorio del libro) ejecuta el agente principal, pasa su resultado y razonamiento por el Shadow, y decide: rechaza de inmediato si el veredicto es `REJECT`; aprueba directo solo si el veredicto es `APPROVE` *y* la confianza supera el umbral configurado; en cualquier otro caso, escala a revisión humana.

La realidad en producción

En producción ocurren casuísticas impredecibles, como el caso de estudio que analizamos en este capítulo. Permitir que un agente opere en sistemas críticos sin una capa de verificación automatizada se convierte en un desastre anunciado.

Un fallo en producción, adicional a escribirse en los *logs* de error y en métricas teóricas de F1-score, se traduce en pérdidas económicas inmediatas ($340K por un error no detectado), afecta directamente a los clientes y destruye la credibilidad de un trabajo de ingeniería que lleva meses o años construir. El Shadow Agent refuerza la arquitectura y

las métricas, actúa como el escudo operativo que protege la credibilidad del desarrollo moderno.

Lo esencial del capítulo 6

- Confianza no es F1-score: es (Competencia × Verificabilidad × Trazabilidad) / Costo del error.
- Verificabilidad es lo que más equipos olvidan: 0,0 = no verificable = no a autonomía.
- Shadow Agent no actúa, valida: estrategia *adversarial*, *consistency* o *constraints*.
- Umbral de confianza define nivel de autonomía: score $< 0{,}01$ = aprobación humana requerida.
- Promover en silencio es mentir: el resultado debe indicar degradación o validación.

RACI en enjambres

La paradoja de la claridad

Más agentes no significa menos claridad. Más agentes significa más confusión sobre quién asume cuando algo sale mal. La matriz RACI (Responsable, Aprobador, Consultado e Informado) no fue inventada para software. Es de *project management* clásico. Pero en enjambres multiagente se vuelve crítica porque el costo de la ambigüedad se multiplica.

La regla cardinal

Agente = Responsable o Consultado. Agente nunca = Aprobador. Accountability (Aprobador) es no delegable. Es la persona que asume el error, que firma el *deploy*, que recibe la llamada de las 3 de la mañana cuando algo falla. Un agente no puede asumir eso. Un agente no despierta con la noticia de que su decisión costó $500K. La regla no es opinable: es una restricción arquitectónica que exige que, si tu diseño requiere que un agente sea Accountable, ese diseño se considere incorrecto.

La matriz RACI para un PR review

Actividad	Developer	Review Agent	Lead Eng	Security Agent
Escribir código	R	-	A	C
Revisar bugs	R	R	A	-
Security scan	-	-	A	R
Aprobar merge	-	I	R	C

Actividad	Developer	Review Agent	Lead Eng	Security Agent
Deploy prod	-	I	R	C

Leyenda: R=Responsible (hace el trabajo), A=Accountable (aprueba, asume), C=Consulted (input antes), I=Informed (notificado después)

Reglas: - Solo hay UN A por actividad (no hay "shared accountability"). - El A puede ser R también (si hace y aprueba). - El agente puede ser R o C. Nunca A. - Si algo requiere A y no hay humano designado, el proceso está roto.

Eisenhower aplicado para delegar a un agente

La Matriz Eisenhower (urgente/importante) se mapea a decisión de autonomía:

	Urgente	No urgente
Importante	Autonomía limitada (RACI gate)	Planificar con humano
No importante	Delegar al agente (R)	Automatizar completamente

Ejemplos: - "Depurar un test que falló en CI": no importante (no afecta al cliente), urgente. Delegar al agente con presupuesto fijo. - "Decisión de arquitectura de base de datos": importante, no urgente. Planificar con un humano; el agente participa solo como C. - "Procesar 10 000 facturas": no importante, no urgente. Automatizar completamente (*workflow* fijo, nivel 1). - "Responder un ticket de cliente que menciona pérdida de datos": importante, urgente. Requiere humano; el agente se utiliza solamente para triaje.

RACI como código

Podemos verlo como un *gate* de autorización que ejecuta la regla. La implementación de referencia (`RACIGate`, en el repositorio del libro) es la pieza central: en tiempo de ejecución, si el rol de un agente para

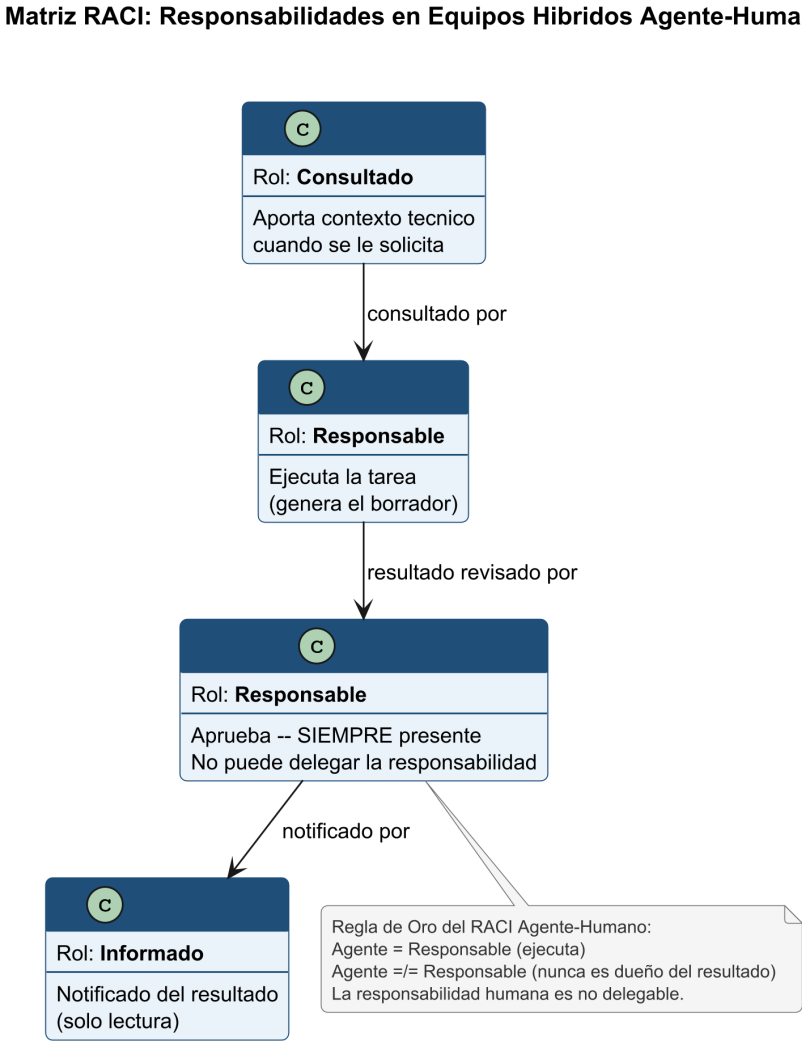


Figura 7.1: Matriz RACI (Responsable, Aprobador, Consultado e Informado)

una actividad es `ACCOUNTABLE`, la solicitud se rechaza de plano (regla cardinal); si es `CONSULTED` o `INFORMED`, el agente no ejecuta; y si el `trust_score` no alcanza el umbral configurado para esa actividad, exige aprobación humana. `RACIEntry`, `get_human_approver` y `request_approval` (que notifica al aprobador por correo o Slack) están completos en el repositorio.

Configuración RACI para pipeline de code review

La matriz de ejemplo (`RACI_MATRIX_CODE_REVIEW`, repositorio) asigna cada actividad a un rol y a un responsable humano: el escaneo de dependencias es `RESPONSIBLE` del agente de seguridad; aprobar un *merge request* o desplegar a producción son actividades donde el agente solo queda `INFORMED` —observa, no aprueba—, con el líder técnico o el CTO como responsables reales, y un `approval_threshold` alto (0.8) para el despliegue por ser irreversible.

Uso en pipeline

La implementación de referencia (`CodeReviewPipeline`, repositorio) recorre las actividades del pipeline (escaneo de dependencias, detección de secretos, sugerencias de código) y, para cada una, consulta al `RACIGate` si el agente puede ejecutarla dado su `trust_score`; si no puede, registra la denegación en el log de auditoría (Capítulo 5) y lanza una excepción en vez de proceder en silencio. Al final, cuando el agente sugiere aprobar el *merge*, el gate vuelve a intervenir: si no está autorizado, el humano designado aprueba.

Caso de estudio del deploy aprobado por un agente

El caso es un deploy que se automatizó porque un agente "aprobó". Caso: startup de *fintech* en São Paulo. Su *pipeline* de *deploy*:

1. El desarrollador escribe código.
2. Agent review (bugs, seguridad y rendimiento).
3. El agente evalúa `should_deploy` y toma la decisión automática de realizar el *merge* y el despliegue.
4. Proceso completado.

El problema estaba en el paso 3: el agente tenía Role = A. Es decir, el diseño arquitectónico permitía que un agente asumiera responsabilidad de aprobador (*accountability*). **Incidente:** un *deploy* desplegó una validación de saldo negativo defectuosa. La función `fee = amount * 0.02` para negativos quedó sin validación. Usuarios con saldo -100 se convirtieron en -101,98. **Impacto:** 847 clientes afectados, $12K en reembolsos, 3 días de investigación, reputación dañada. **Raíz:** el agente aprobó el *deploy* sin contar con la aprobación de una persona responsable o comité encargado en el *loop*. El RACI estaba mal configurado: `agent_role=ACCOUNTABLE`.

Corrección: el `RACIEntry` de la actividad `deploy_production` pasó de `agent_role=ACCOUNTABLE` (el error: contradecía la regla cardinal, y aun así nombraba a un humano "accountable" en paralelo) a `agent_role=RESPONSIBLE` —el agente solo puede ejecutar—, con `cto@company.com` como responsable real, aprobación explícita obligatoria y un umbral de confianza alto (0.8).

Ejercicio resuelto de ajuste dinámico

Enunciado. Construye un RACIGate donde el `approval_threshold` se ajuste dinámicamente según el `error_cost` en tiempo de ejecución. **Solución comentada.** La implementación de referencia (`DynamicThresholdRACIGate`, repositorio) mapea el costo del error de cada actividad a un umbral de confianza creciente: costo 1-10 → umbral 0,2; 11-30 → 0,4; 31-60 → 0,6; 61-100 → 0,8; más de 100 (catastrófico) → 0,95. En cada solicitud, calcula el umbral dinámico para esa actividad, lo aplica sobre la matriz y delega el resto de la verificación al `RACIGate` base.

Uso

En la práctica: con una matriz de ejemplo donde `scan_logs` tiene costo bajo (5) y `initiate_breach_notification` tiene costo casi catastrófico (95), un agente con `trust_score=0.25` puede ejecutar el primero (umbral 0,2) pero no el segundo (umbral 0,95) — el mismo agente, con la misma confianza, se autoriza o se bloquea según cuánto cuesta equivocarse.

Lo esencial del capítulo 7

- El agente nunca es aprobador: la responsabilidad (*accountability*) no es delegable; siempre permanece en una persona.
- Matriz RACI como código: funciona como un *gate* de autorización, no como simple documentación.
- Eisenhower se mapea a la autonomía: tareas no importantes y no urgentes pueden delegarse por completo.
- Umbral dinámico basado en `error_cost`: a mayor costo de error, mayor nivel de confianza requerido.
- Caso real: *deploy* sin RACI correcto generó 12.000 USD en reembolsos.

SOLID para agentes

Por qué SOLID es gestión de complejidad

SOLID nació en el contexto de la programación orientada a objetos. Pero los principios no tratan sobre clases. Tratan sobre cómo evitar que la complejidad se vuelva inmanejable cuando el sistema crece. En sistemas multiagente, la complejidad no crece linealmente. Crece exponencialmente. Un agente con 4 responsabilidades se vuelve 4 veces más difícil de mantener. Dos agentes con 8 responsabilidades cada uno se vuelven 8 veces más difíciles que 2×4. El factor de degradación pasa de 2,3× en OO clásico a 4-6× en sistemas agénticos.

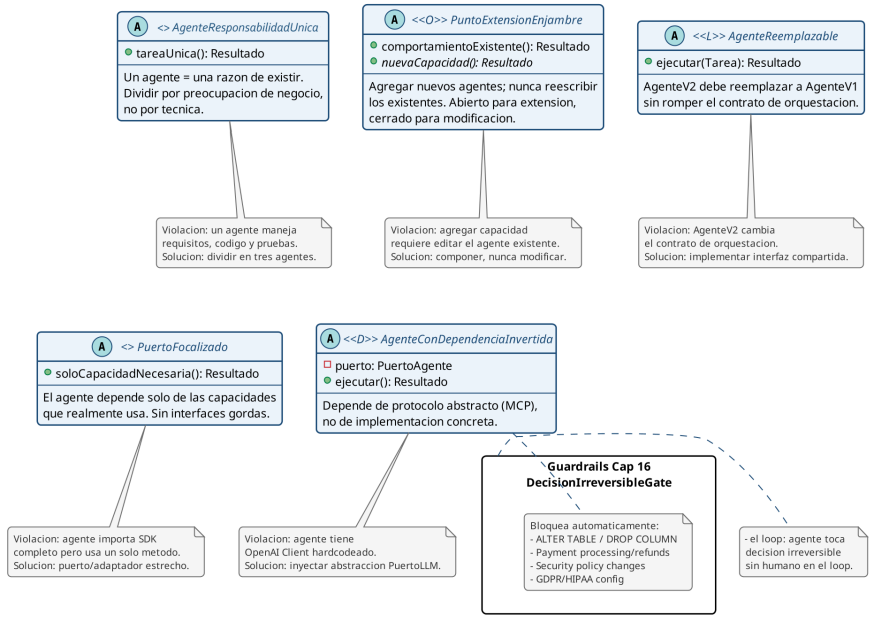


Figura 8.1: Principios SOLID aplicados a enjambres multiagente

Los 5 principios reescritos

Single Responsibility Principle (SRP) **Original:** una clase, una responsabilidad. **Para agentes:** un agente, un propósito. **Antipatrón:** un `JackOfAllTradesAgent` que hace *review* de código, seguridad y gestión de tickets en un solo método parece razonable al principio, pero se vuelve inmanejable en meses: cambiar el *security scanner* obliga a tocar el código de *review*; probar un *bug fix* exige *mocks* de tres sistemas; cuando algo falla, no se sabe cuál de los tres subsistemas fue; y no se puede desplegar uno sin arrastrar a los demás. **Solución con SRP** (repositorio del libro): tres agentes especializados de responsabilidad única —`CodeReviewAgent` (solo encuentra bugs), `SecurityScanAgent` (solo encuentra vulnerabilidades, no decide prioridad ni crea tickets) y `TicketManagementAgent` (solo crea tickets, no decide qué es un bug)— coordinados por un `MultiAgentOrchestrator` que únicamente sabe de flujo de control, no de la lógica de dominio de cada uno. El resultado: cambios locales, *testing* aislado, razonamiento claro sobre qué falló, y despliegue independiente por agente.

Open/Closed Principle (OCP) **Original:** cerrado a modificación, abierto a extensión. **Para agentes:** el orquestador no cambia; los agentes se extienden. La implementación de referencia (repositorio) añade un nuevo tipo de análisis —`PerformanceReviewAgent`— sin tocar el orquestador existente: `EnhancedMultiAgentOrchestrator` extiende al orquestador base (herencia), agrega el nuevo agente como dependencia adicional, y su `process_pr` llama primero al comportamiento heredado y luego añade los hallazgos de rendimiento al resultado. El orquestador original queda intacto; la extensión vive en una subclase nueva.

Liskov Substitution Principle (LSP) **Original:** las subclases deben poder sustituir a las clases base. **Para agentes:** cualquier agente que implemente el mismo contrato puede reemplazar a otro sin que el orquestador lo note. La implementación de referencia (repositorio) define un protocolo `ReviewAgentProtocol` con un único método `review(diff) -> list`; tanto `CodeReviewAgent` (basado en reglas) como `LLMCodeReviewAgent` (basado en un LLM) lo implementan, así que el orquestador puede recibir cualquiera de los dos indistintamente: cambiar de reglas a LLM es una sustitución transparente, no un cambio arquitectónico.

Interface Segregation Principle (ISP) **Original:** muchas interfaces específicas son mejores que una general. **Para agentes:** cada agente necesita solo las herramientas que usa. **Antipatrón:** un `OverpoweredAgent` con acceso a sistema de archivos, base de datos, correo, GitHub y LLM, cuando en realidad solo usa el LLM — cada permiso de más

es superficie de ataque innecesaria. **Solución con ISP** (repositorio): un `MinimalCodeReviewAgent` que recibe únicamente el cliente LLM, y un `SecurityScanOrchestrator` que concede permisos por necesidad —sistema de archivos de solo lectura al agente de seguridad, GitHub y base de datos únicamente al agente de despliegue— construyendo cada agente con el mínimo de acceso que su tarea requiere.

Dependency Inversion Principle (DIP) **Original:** depender de abstracciones, no de concretos. **Para agentes:** el orquestador no depende de agentes específicos, depende de protocolos. La implementación de referencia (repositorio) define dos protocolos —`ScanAgentProtocol` y `ReportAgentProtocol`— de los que depende el `SecurityPipeline`, en vez de depender de una implementación concreta. Existen varias implementaciones intercambiables de cada uno (`PatternBasedScanner` o `LLMSecurityScanner` para el escaneo; `SLACKReporter` o `JIRATicketReporter` para el reporte), y la elección concreta se inyecta en tiempo de ejecución — cambiar de notificar por Slack a crear tickets en Jira es una línea, sin tocar el *pipeline*.

Refactoring de enjambre en 4 pasos con checklist real

Un enjambre mal diseñado no se arregla de golpe. Se refactoriza en incrementos.

Checklist paso a paso

Paso 1: Identificar violaciones de SRP

- ¿Algún agente hace >3 cosas distintas?
- ¿Hay lógica de orquestación dentro de agentes?
- ¿El *testing* requiere *mock* de 3+ sistemas? **Paso 2: Definir protocolos**
- ¿Cada tipo de agente tiene un protocolo definido?
- ¿Los protocolos declaran solo lo necesario (ISP)?
- ¿Se puede intercambiar una implementación por otra (LSP)? **Paso 3: Refactorizar por capas**
- Extraer responsabilidades del "agente monolítico".
- Crear nuevos agentes SRP.
- El orquestador se encarga del flujo. **Paso 4: Inyectar dependencias**

- El orquestador depende de protocolos, no de concretos.
- Las implementaciones se inyectan en *runtime*.
- El *testing* puede hacer *mock* de un agente sin tocar los demás.

Caso de estudio de problemas en refactorización

Un equipo tardó 3 meses refactorizando porque empezó sin SRP. La startup en Barcelona construía un enjambre para soporte técnico. El primer agente:

- Clasificaba tickets.
- Buscaba documentación.
- Generaba respuesta.
- Enviaba correo.
- Creaba ticket en CRM.
- Analizaba sentimientos.

En 3 meses:

- 42 % del tiempo en *debugging* (¿qué agente falló?).
- Cambios en "clasificación" requerían *regression testing* de "envío de correo".
- *Onboarding* de nuevos desarrolladores: 6 semanas para entender quién hace qué.

Corrección: la implementación de referencia (repositorio del libro) transporta el estado del ticket a través de la cadena de agentes con un `TicketContext` (contexto tipado, con `correlation_id`) y un `CommunicationProtocol` que estandariza cómo cada agente recibe y emite ese contexto.

1. Separaron en 5 agentes SRP:

- `TicketClassifierAgent`, `DocumentationSearchAgent`, `ResponseGeneratorAgent`, `EmailSenderAgent` y `CRMTicketAgent`.

2. Definieron protocolos de comunicación entre estados: se utiliza TicketContext para transportar el estado del ticket a través de la cadena de agentes, y CommunicationProtocol para estandarizar la interfaz de recepción y emisión.
3. Cada agente publica/suscribe a eventos Kafka con `correlation_id` en *headers* para auditoría completa.

4. Orquestador con RACI definido: Classifier (Responsible), Documentation (Support), Response (Responsible), Human (Accountable para escalados).
5. *Testing* unitario por agente con *mocks* aislados.

Resultado: el *debugging* bajó del 42 % al 12 %, los cambios son aislados sin regresiones, y el *onboarding* es de 2 semanas. La clave fue el `correlation_id` que permitió reconstruir el flujo completo de cada ticket a través de los 5 agentes.

Lo esencial del capítulo 8

- SOLID es gestión de complejidad y se multiplica por 4-6× en enjambres.
- SRP aplicado: un agente, un propósito. Si hace >3 cosas, está roto.
- El orquestador coordina y los agentes ejecutan: separar flujo de control de lógica de dominio.
- Protocolos antes que concretos: el orquestador depende de abstracciones.
- Refactorización incremental: identificar violaciones, definir protocolos, extraer componentes e inyectar dependencias.

Invariantes de producción

Los 5 invariantes que todo sistema debe satisfacer

Un invariante va más allá de una "buena práctica": es una condición que siempre debe ser cierta, sin excepción. Si un invariante se viola, el sistema está en estado corrupto. En sistemas deterministas (código tradicional), los invariantes son fáciles: "el saldo de la cuenta no puede ser negativo", "el índice del *array* debe estar en rango". En sistemas agénticos (no deterministas, probabilísticos), los invariantes son más sutiles. Pero más críticos. Porque cuando fallan, el fallo es silencioso, gradual e invisible hasta que ya ha costado miles.

La verificabilidad es igual a la nueva confianza

Enunciado: ningún agente opera con confianza >0,5 en un dominio donde el *output* no es verificable en <5 minutos. **Razón:** si no puedes verificar en 5 minutos, no puedes detectar un error a tiempo. El error se acumula, se propaga, se multiplica. **Violación típica:** un agente que genera informes financieros sin validación cruzada. El error se detecta cuando Finanzas cierra el mes. **Detección automática:** la implementación de referencia (`DomainRiskProfile` + `compute_max_allowed_autonomy`, en el repositorio del libro) asigna una autonomía máxima según qué tan verificable es el dominio: sin *ground truth* (`NOT_VERIFIABLE`) → autonomía 0; investigación lenta (`SLOW_MANUAL`) → autonomía máxima 0,1, solo triaje; revisión rápida (`QUICK_MANUAL`) → autonomía igual al `trust_threshold` del dominio; verificación automatizada (`AUTOMATED`) → un 20 % de bonus sobre ese umbral. Así, un agente de informes financieros con verificación lenta queda topado en 0,1 de autonomía sin importar su competencia, mientras un agente de *code review* con tests automáticos puede llegar a 0,72.

El costo real es igual a la deuda de verificación

Enunciado: el costo real de un agente es llamadas a API + (tiempo de verificación × frecuencia × costo humano). **Razón:** los equipos solo miran los tokens. Ignoran el tiempo que los ingenieros pasan verificando *output*. Eso es deuda de verificación. **Fórmula:**

```
TCO mensual = (tokens_usd) + (hours_verify × hourly_rate × volume)
```

Ejemplo: - Tokens: $500/mes. - 1000 tickets/mes. - Verificación humana: 5 min por ticket (agente "confiable"). - Tarifa por hora del ingeniero: $50/h.

```
Deuda verificación = (5/60 × $50 × 1000) = $4,167/mes
TCO real = $500 + $4,167 = $4,667/mes
```

Observación: parece que el agente cuesta $500. En realidad cuesta $4.667. **Mejora:** si se automatiza la verificación (tests que validan el 80 % de los casos):

```
Verificación manual: 200 tickets × 5 min = 16,67 horas = $833
TCO mejorado = $500 + $833 = $1,333/mes (65 % de reducción)
```

Implementación de *tracking*: la implementación de referencia (`TrueCostTracker`, en el repositorio del libro) registra en cada llamada del agente el costo en tokens y el tiempo humano de verificación, y calcula el TCO real sumando ambos — expone `detect_debt_spike()`, que alerta si el costo de verificación supera tres veces el costo de tokens, señal de que algo en el proceso de verificación se disparó.

La autonomía se gana y no se asigna

Enunciado: la autonomía de un agente es acumulativa y progresiva. No se inicia en nivel 3. Se gana desde nivel 0. **Razón:** darle nivel 4 a un agente nuevo es darle un Ferrari a un principiante. El Ferrari no está roto. El conductor no está preparado. **Progresión:**

Semana	Nivel permitido	Criterio de ascenso
1-2	Nivel 0 (workflow fijo)	95 % de precisión en eval-set

Semana	Nivel permitido	Criterio de ascenso
3-4	Nivel 1 (enrutamiento)	90 % de enrutamiento correcto
5-8	Nivel 2 (agente con herramientas)	Trust score > 0,6
9+	Nivel 3 (autonomía limitada)	Sin incidentes críticos
12+	Nivel 4 (autonomía amplia)	3 meses sin errores de categoría

Implementación de *gate*: la implementación de referencia (`AutonomyGate`, en el repositorio del libro) define cinco niveles de autonomía (`WORKFLOW_ONLY` a `FULL_AUTONOMY`) y solo permite ascender un nivel a la vez, encadenando la evaluación sobre el `current_level` —nunca sobre el número de semanas transcurridas—, de modo que llegar a `FULL_AUTONOMY` exige haber pasado, en orden, por cada nivel intermedio con sus criterios (tasa de aciertos en *eval-set*, cero incidentes críticos). Un incidente crítico reinicia el nivel a `TOOL_AGENT`, sin importar cuánto tiempo llevaba el agente en producción.

Ese encadenamiento sobre el nivel actual, y no sobre la semana, es la diferencia clave: una versión más ingenua de este *gate* evaluaba cada hito de forma independiente, y con un `eval_set_pass_rate` bajo pero doce semanas en producción, el agente saltaba directo a autonomía completa sin haber demostrado nunca una tasa de éxito aceptable.

El contexto es capital y no es gratis

Enunciado: cada token inyectado en contexto tiene costo. El contexto no es "memoria gratis". **Razón:** los equipos suelen inyectar documentos completos en el contexto. Una documentación de 10.000 tokens implica un costo de 0,30 USD por llamada. Con 1000 llamadas al día, el costo asciende a 300 USD diarios, es decir, aproximadamente 9000 USD al mes.

Costo típico de contexto desperdiciado:

- Tamaño de *chunk* en RAG: 2000 tokens (demasiado grande; normalmente 500 tokens suele ser suficiente).
- 5 *chunks* por consulta generan 10.000 tokens de contexto.

- Costo del token de entrada: 3 USD por millón de tokens, equivalente a 0,03 USD por consulta.
- 10.000 consultas al día representan un costo de 300 USD diarios, aproximadamente 9000 USD al mes.

Optimizaciones con mayor impacto:

1. Reducir el tamaño del *chunk* de 2000 a 400 tokens (75 % menos tokens, con la misma calidad).
2. Aplicar re-ranking después de la recuperación para filtrar *chunks* irrelevantes.
3. Comprimir el contexto de forma diferida para eliminar historial de conversación antiguo. **Implementación de *guardrail*:** la implementación de referencia (`ContextBudgetManager`, en el repositorio del libro) trata el contexto como un recurso escaso: asigna un presupuesto máximo de tokens por categoría (sistema, *chunks* de RAG, historial de conversación, respuestas de herramientas), rechaza asignaciones que excedan lo disponible, y aplica un límite específico a RAG (máximo 50 % del presupuesto disponible), truncando o filtrando *chunks* si es necesario.

La arquitectura domina al modelo

Enunciado: una arquitectura correcta con modelo mediano funciona mejor que una arquitectura mala con SOTA. **Razón:** los modelos mejoran constantemente. La arquitectura se queda. Invertir en arquitectura tiene mayor ROI a largo plazo. **Comparación empírica (caso real):**

Configuración	Precisión	Costo mensual	Esfuerzo de mantenimiento
Arq simple + Opus	72 %	$15.000	Alto
Arq correcta + Haiku	84 %	$4.000	Bajo
Arq correcta + Opus	91 %	$6.000	Bajo

Lección: la arquitectura correcta añadió 12 puntos de *accuracy* con un modelo peor, y costó un 70 % menos. **Checklist de arquitectura primero:** - ¿Hay AI Gateway con *tracking* de costos? - ¿Hay *eval-set* vivo con >50 muestras? - ¿Hay Shadow Agent o validación paralela? - ¿Hay RACI definido por actividad? - ¿Hay presupuesto de contexto controlado? - ¿Hay *autonomy gate* progresivo? - ¿Hay federamiento de agentes con SRP?

Si 3+ respuestas son no, necesitas refactorizar la arquitectura antes de optimizar el modelo.

Caso de estudio de un despliegue que falló

Por violar 3 invariantes, una empresa de logística en Madrid quiere un agente que optimice rutas de entrega.

Violación 1: Verificabilidad El *output* del agente es "ruta óptima". Para verificar, una persona tendría que:

1. Simular 50 paradas.
2. Calcular distancias.
3. Verificar ventanas de tiempo. Tarea de 2 horas. Verificabilidad = 0,1. Autonomía permitida debería ser: 0,1 × 0,5 = 0,05 (nivel *workflow*, no autónomo). El equipo dio autonomía 0,8 (nivel 3).

Violación 2: Costo real El equipo miró los tokens: $800/mes. Pensó "barato". Ignoró: los planificadores de rutas gastan 15 horas/semana verificando correcciones tras errores del agente. Deuda de verificación: 15h × $60/h × 4 = $3.600/mes. TCO real: $4.400/mes, no $800.

Violación 3: Autonomía inmediata Se puso el agente en producción en nivel 3 desde el día 1. Sin fase de validación. Primera semana: rutas que superan ventanas de entrega, clientes furiosos, $12K en compensaciones. **Corrección:**

1. Rediseñar verificabilidad: añadir simulador automático (costo: $15K en desarrollo, ahorra $3K/mes en 5 meses).
2. Autonomía escalonada: 6 semanas en modo "shadow only" (propone sin ejecutar).
3. TCO expuesto: el CFO ahora mira $4.400/mes, no $800.

Resultado tras la corrección: TCO de $2.200/mes (después de optimización), 0 incidentes en 4 meses.

Lo esencial del capítulo 9

- Las 5 invariantes que siempre deben ser ciertos: verificabilidad, TCO, autonomía progresiva, contexto como capital, arquitectura domina.
- Verificabilidad define autonomía máxima: si no verificas en 5 min, autonomía ≈ 0.
- Deuda de verificación es el costo invisible: TCO = tokens + (horas $\times$ tarifa $\times$ volumen).
- Autonomía se gana progresivamente: no nivel 3 el día 1. Semanas 1-2: nivel 0; semanas 9+: nivel 3.
- Contexto es presupuesto, no memoria gratis: los *chunks* RAG deben tener un límite estricto.
- Arquitectura domina al modelo: mejor arquitectura con Haiku que mala arquitectura con Opus.

DDD + Kafka como sistema nervioso

Bounded contexts para agentes

En Domain-Driven Design, un Bounded Context es un límite donde un modelo tiene un sentido específico. Fuera de ese límite, el mismo término puede significar otra cosa. En sistemas multiagente, los Bounded Contexts son aún más críticos. Porque cuando los contextos se mezclan, los agentes se corrompen. **Ejemplo problemático:** un agente de "gestión de pedidos" recibe un evento "cancelar". Dentro del contexto de ventas, "cancelar" significa "devolver dinero y liberar stock". Dentro del contexto de logística, "cancelar" significa "detener envío si está en tránsito". ¿Qué hace el agente? Depende del contexto en el que se esté ejecutando. Si no hay un *bounded context* definido, el agente toma decisiones inconsistentes.

Definición de bounded context para agentes

La implementación de referencia (repositorio del libro) define un `AgentEvent` base —con `event_type` (inicio/fin de tarea, pensamiento generado, acción invocada, resultado de acción...), `correlation_id`, el `context` (bounded context que lo emitió) y su *payload*— serializable directamente a un registro de Kafka: la clave del registro es el `correlation_id`, lo que garantiza que todos los eventos de una misma tarea caigan en la misma partición y mantengan su orden. Subclases específicas (`ThoughtGeneratedEvent`, `ActionInvokedEvent`, `ActionResultEvent`) siguen el mismo patrón con campos propios.

`KafkaAgentLogger` publica cada pensamiento, acción y resultado del agente como un evento inmutable —el equivalente distribuido del *audit log* del Capítulo 5—, mientras que `AgentEventConsumer` reconstruye el historial completo de una tarea leyendo todos los eventos que comparten un `correlation_id`, ordenados por *timestamp*: útil para depuración, *replay* y análisis de patrones.

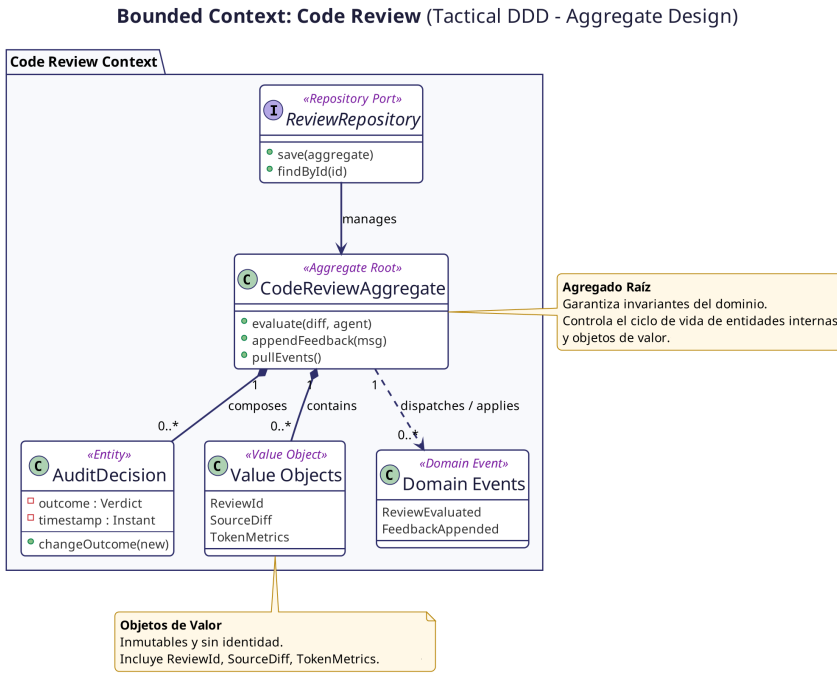


Figura 10.1: Arquitectura DDD con *bounded contexts* separados

Caso de estudio de mezcla de contextos

Un enjambre que se corrompió porque los contextos se mezclaron. Startup de *e-commerce* en CDMX. Un solo "Agente de Pedidos" manejaba: - Creación de pedidos (contexto de ventas). - Cancelación de pedidos (componente de ventas + logística). - Envío de pedidos (componente de logística).

Problema: cuando un usuario canceló un pedido *con envío en curso*, el agente:

1. Aplicó la lógica de cancelación de *Ventas*: reembolso procesado.
2. Pero *no* ejecutó la lógica de *logística* en curso: el paquete ya había salido. **Resultado:** 23 paquetes enviados a usuarios que ya recibieron reembolso. Pérdida: $8.400 + reputación dañada. **Raíz:** el agente no tenía un *bounded context* claro. El mismo código trataba "cancelar" según solo lo que conocía (ventas).

Corrección:

1. Separar en dos agentes: `SalesAgent` (ventas) y `LogisticsAgent` (logística).
2. Definir un contrato entre contextos: cuando ventas cancela, emite evento `OrderCancelled.external`.
3. Logistics escucha el evento y ejecuta compensación (detener envío; si ya salió, iniciar devolución).
4. Kafka garantiza orden y durabilidad: si Logistics cae, el evento queda en el *topic* y se reprocesa al recuperarse.

Costo de corrección: 3 semanas de refactorización. **Ahorro posterior:** 0 incidentes de este tipo en 12 meses.

Lo esencial del capítulo 10

- Bounded context delimita significado: "cancelar" puede significar cosas distintas en cada contexto.
- Kafka como log inmutable: *replay*, multi-consumidor, orden garantizado.
- Eventos estandarizados: AgentEvent, ThoughtGenerated, ActionInvoked, ActionResult.
- Correlación de eventos: `correlation_id` rastrea la tarea completa a través de múltiples *streams*.
- Caso de corrupción de contexto: la mezcla de contextos provocó pérdidas de 8400 USD por paquetes enviados sin cobro.

Gobernanza de deuda técnica agéntica

Por qué la deuda agéntica se acumula más rápido

Deuda técnica en código tradicional: *copy-paste* rápido, refactor después. El costo es lineal: cada vez es más lento añadir funcionalidades. Deuda técnica agéntica: *copy-paste* de *prompts*, sin *evals*, sin documentar decisiones. El costo es exponencial: *bugs* silenciosos, desviación de comportamiento, costos inflados invisibles. **La diferencia clave:** en código tradicional, un *bug* es visible (el test falla). En sistemas agénticos, un *bug* puede estar produciendo un *output* incorrecto durante meses sin que nadie lo note.

Métricas automáticas para detectar deuda

Indicadores de alerta temprana

Métrica	Threshold	Acción
Tasa de reintentos	$> 15\ \%$	Revisar *prompt* o *evals*
Costo por tarea exitosa	$> 1{,}5\times$ *baseline*	Optimizar *routing* o contexto
Time-to-resolution	$> 2\times$ *baseline*	Depurar *fallback chains*
Trust score drift	-0,1 en 30 días	Reentrenar o actualizar *eval-set*
Context waste	$> 40\ \%$	Reducir *chunk size* o mejorar RAG

Implementación de dashboard de deuda

La implementación de referencia (repositorio del libro) define un `DebtIndicator` por métrica (valor actual, umbral, severidad, tendencia) y una tabla de reglas (`DEBT_RULES`) con tres niveles de severidad y su acción recomendada para cada métrica —tasa de reintentos, relación de costo, desperdicio de contexto, *drift* de confianza, tasa de fallos de herramienta.

`DebtTracker` no es una revisión periódica: es una alarma que salta cuando la deuda excede el umbral aceptable. Registra cada nuevo *snapshot* de métricas, evalúa cuáles superan su umbral y con qué severidad, calcula la tendencia de los últimos cinco *snapshots* (creciente, decreciente o estable) y dispara alertas para severidad alta o crítica. Conecta con el AI Gateway del Capítulo 3: si el costo o los reintentos se disparan, fuerza al enjambre a degradar automáticamente a un modelo más barato (por ejemplo, de `claude-opus-4-8` a `claude-sonnet-4-6`) en vez de esperar a que alguien lo note manualmente.

Revisiones de arquitectura que incluyen output de agentes

Las revisiones de arquitectura tradicionales revisan:

- Patrón de diseño (MVC, microservicios, eventos).
- Flujos de datos (*async* vs *sync*, *batch* vs *streaming*).
- Escalabilidad (horizontal vs vertical).
- Identificación de puntos únicos de fallo.

En sistemas agénticos, falta un elemento crítico: ¿El *output* del agente cumple las invariantes de dominio?

Checklist de revisión para agentes

La implementación de referencia (`AgentArchitectureReviewer`, en el repositorio del libro) mantiene un checklist de preguntas organizadas por categoría (evaluación, costos, seguridad, observabilidad, autonomía, arquitectura) — por ejemplo: "¿existe un *eval-set* vivo con más de 50 muestras representativas?", "¿hay RACI definido con una persona como Aprobador?", "¿cada acción del agente está auditada en un log inmutable?". Cada respuesta se clasifica como conforme, no conforme o

ausente; si alguna pregunta crítica de seguridad o evaluación queda no conforme, la revisión se rechaza de plano; si hay no conformidades no críticas, se aprueba con cambios pendientes; si faltan respuestas, se pide más información; solo si todo está conforme, se aprueba.

CI/CD para bloquear deuda agéntica

La deuda técnica agéntica no entra al sistema mediante `git push`. Entra mediante:

- Cambio de *prompt* sin actualización de *evals*.
- Nueva herramienta sin documentación del nivel de permiso.
- Reducción de presupuesto que degrada la calidad sin alertar.

Pipeline de CI/CD con gates agénticos

```
name: Agent CI
on:
  push:
    paths: ["agents/**", "prompts/**", "evals/**"]

jobs:
  evaluate:            # Corre el eval-set; falla si pass-rate < 80% o
el costo sube >15%
    steps:
      - run: python -m evals.run --agent agents/sales_agent.py
--min-pass-rate 0.80
      - run: python -m evals.cost --max-increase 0.15

  security-scan:        # Escanea riesgo de prompt injection y permisos
de herramientas
    steps:
      - run: python -m security.scan_prompt --prompt
prompts/sales_agent.txt --severity high
      - run: python -m security.check_tools --require-approval
dangerous_tools

  trust-score-check:     # Calcula TrustScore; su ci_exit_code decide si
el pipeline sigue
    steps:
```

```
      - run: python -m trustscore.calculate --output
results/trustscore.json
      - run: exit $(python -c "import
json;print(json.load(open('results/trustscore.json'))['ci_exit_code'])")

  deploy:
    needs: [evaluate, security-scan, trust-score-check]
    if: github.ref == 'refs/heads/main'
    steps:
      - run: python -m deploy.agent --rollback-threshold "trust_score <
0.6" --canary-percent 10
```

(El job `architecture-review`, que corre el checklist de la sección anterior y los indicadores de deuda, sigue el mismo formato, repositorio del libro.)

TrustScore como la métrica vigilante del CI/CD

El TrustScore es un número determinista que decide si un *deploy* avanza, se degrada o se revierte. Sin TrustScore conectado a CI/CD: el *pipeline* pasa. Tú despliegas. El agente empieza a fallar silenciosamente. Nadie se entera hasta que un cliente se queja. Con TrustScore conectado a CI/CD: el *pipeline* falla si TrustScore $<$ *threshold*. No despliegas nada que degrade la confianza. Punto.

La implementación de referencia (`TrustScoreCalculator`, en el repositorio del libro) calcula el TrustScore como una suma ponderada —tasa de aciertos (30 %), precisión (25 %), aprobación humana (20 %), ausencia de bugs críticos (15 %) y consistencia (10 %)— con una regla que anula todo lo demás: un solo bug crítico basta para bloquear el *deploy* sin importar el resto de la fórmula (código de salida 2 en CI). El resultado se clasifica en cinco niveles de confianza (`UNTRUSTED` a `VERY_HIGH`, de $<$0,40 a $>$0,90), cada uno con su recomendación operativa —desde "no producción" hasta "autonomía casi completa, seguir monitoreando el *drift*". Un método adicional (`check_drift`) compara el TrustScore actual contra el anterior y emite una advertencia (sin bloquear) si cae más de un 10 %.

Autonomía basada en TrustScore

El TrustScore no solo decide si desplegar. Decide qué autonomía tiene el agente en producción.

TrustScore	Autonomía permitida	Restricciones
< 0,40	Ninguna	Solo lectura, la persona siempre aprueba
0,40-0,59	Canary 10 %	Persona en el loop para todas las decisiones
0,60-0,79	Producción limitada	Aprobación para decisiones reversibles
0,80-0,90	Producción con sombra	Shadow Agent valida, la persona aprueba críticas
> 0,90	Autonomía casi completa	Solo decisiones críticas requieren a una persona

La implementación de referencia (`AutonomyGate`, en el repositorio del libro) traduce esta tabla en código: mapea el TrustScore a uno de cinco niveles (`NONE`, `HUMAN_LOOP`, `LIMITED`, `SHADOWED`, `NEAR_FULL`) y, para cada acción propuesta por el agente (con su tipo — lectura, escritura, borrado, aprobación —, reversibilidad y puntaje de riesgo), decide si se ejecuta según el nivel: en `LIMITED` solo se permite lectura o escritura de bajo riesgo; en `SHADOWED` se ejecuta con el Shadow Agent validando si el riesgo es bajo; en `NEAR_FULL` casi todo se permite salvo lo irreversible o crítico.

Caso de estudio del TrustScore bajo

Un deploy que falló por TrustScore bajo. Startup de *fintech* en Madrid. Nuevo agente de clasificación de transacciones. **Sin TrustScore gate:**

- Deploy directo a producción.
- 3 días después: 30 % de transacciones clasificadas incorrectamente.
- $45K en transacciones mal categorizadas.
- 2 semanas de auditoría y corrección manual.

Con TrustScore gate:

- CI detecta: pass_rate 0,68 < 0,70 *threshold*.
- Exit code 1: el *pipeline* falla.
- El equipo no despliega.
- Mejoran el *prompt* + *eval-set*.
- Re-ejecutan CI: trust_score 0,82; exit code 0.
- Deploy seguro a producción.

Ahorro: $45K + 2 semanas de crisis evitados.

Checklist de deploy seguro

La regla ya no es "si los tests pasan, deploy", sino "si los evals, seguridad y arquitectura pasan, entonces deploy con canary". La implementación de referencia (`DeployGate`, en el repositorio del libro) avanza por tres fases progresivas: **canary** (10 % del tráfico, exige `pass_rate` ≥ 0,80 y tasa de error ≤ 0,10 para avanzar), **slow rollout** (50 % del tráfico, exige `pass_rate` ≥ 0,85 y TrustScore ≥ 0,60), y **full deploy** (100 % del tráfico). En paralelo, `should_rollback()` vigila las métricas en tiempo real y fuerza una reversión si la tasa de error supera 0,20, el `pass_rate` cae por debajo de 0,70, el TrustScore cae por debajo de 0,40, o la latencia p99 supera los 5 segundos — cualquiera de estas condiciones basta, sin esperar a que se combinen.

Caso de estudio de un deploy sin validación

Un deploy que introdujo 6 meses de deuda silenciosa, esto en el equipo de soporte en Buenos Aires que desplegó un agente de triaje. Sin *evals*, sin *gates* de CI/CD, sin documentar cambios.

Lo que pasó:

1. Semana 1: triaje parece correcto (90 % de satisfacción humana).
2. Semana 3: un ingeniero menciona "el agente parece más conservador".
3. Semana 6: la deuda se introduce porque el *prompt* se ajustó "para ser más cuidadoso".
4. Mes 3: el 60 % de los tickets son "evaluar con una persona" (el agente ya no confía).
5. Mes 6: un ticket de "fallo en triaje" finalmente se detecta.

Costo:

- 6 meses de triaje restringido.
- Aumento del 50 % en tiempo para una persona.
- Estimación de desgaste de 12.000 horas-hombre. **Raíz:** no había un *eval-set* vivo que hubiera ejecutado la regresión del equipo.

Corrección:

1. *Evals* semilla con 100 casos reales.

2. *Gate* de CI: cualquier cambio debe pasar `pass_rate >= 0.80`.
3. *Canary*: 10 % de tráfico por 24 h antes de *full deploy*.
4. Monitor: alertas cuando `pass_rate` cae $< 0{,}85$. **Resultado:** 0 incidentes de deuda silenciosa en 12 meses posteriores.

Lo esencial del capítulo 11

- Deuda agéntica es exponencial: los *bugs* silenciosos comprometen meses antes de detectarse.
- 5 métricas de alerta: `retries_rate`, `cost ratio`, `context_waste`, `trust_drift`, `tool_failures`.
- Las revisiones deben incluir agentes: *checklist* de *evals*, costos, seguridad, observabilidad, autonomía.
- CI/CD con *gates* específicos: *eval-set*, *security scan*, arquitectura, *debt check*, TrustScore.
- TrustScore *exit codes* deterministas: 0 = OK, 1 = score bajo, 2 = *bugs* críticos, 3 = *drift*.
- Fórmula TrustScore: `pass_rate(30%) + precision(25%) + human_approval(20%) + no_critical_bugs(15%) + consistency(10%)`.
- Autonomía basada en TrustScore: <0,40 = nada, 0,40-0,60 = una persona en *loop*, 0,60-0,80 = limitada, >0,90 = casi completa.
- Deploy progresivo obligatorio: iniciar con un *canary* del 10 %, continuar con un *slow rollout* al 50 % y finalizar con el despliegue completo.
- Caso de éxito: TrustScore gate bloqueó un *deploy* que habría costado $45K.
- Caso de deuda silenciosa: 6 meses de degradación detectados demasiado tarde.

Revisión de código con agentes

Matriz RACI para PR review

La revisión de código es donde más valor pueden generar los agentes hoy. No porque escriban mejor que los humanos, sino porque escanean sistemáticamente lo que nosotros como humanos saltamos por fatiga o sesgo. Pero el RACI es crítico: ¿quién aprueba el *merge*? ¿El agente? ¿El *developer*? ¿El *lead*? **Matriz RACI para PR review:**

Actividad	Developer	Review Agent	Lead Engineer	Security Agent
Escribir código	R	-	A	-
Scan de bugs	R	R	C	-
Scan de seguridad	-	R	A	R
Recomendar cambios	I	R	C	C
Aprobar merge	R	I	R	C
Merge al main	R	-	A	-

Clave: el agente es Responsible por encontrar *issues*, no Accountable por aprobar. La persona mantiene la accountability.

El agente observa bugs que pasamos por alto

El agente encuentra bugs que los desarrolladores saltamos, según un estudio de (Wadhwa et al. 2023): los agentes basados en LLM pueden asistir la revisión de código y reducir parte de la carga del análisis

humano, pero sus propuestas todavía requieren validación. A menudo nos familiarizamos con nuestro propio código, de forma que intentamos completar automáticamente lo que esperamos encontrar en lugar de observar únicamente lo que está escrito. El agente no tiene esa ceguera. Lee línea por línea, patrón por patrón.

Implementación de Review Agent

La implementación de referencia (`CodeReviewAgent`, en el repositorio del libro) busca específicamente bugs de **lógica** — no estilo, no *naming* — y exige que cada hallazgo venga acompañado de un test mínimo que lo reproduce, no solo una opinión. El *prompt* de sistema fuerza salida JSON estructurada (`{"bugs": [{"line", "type", "severity", "description", "test"}]}`); cada `BugFinding` incluye línea, tipo (`null-pointer`, `off-by-one`, *race condition*, error de lógica, discordancia de tipos), severidad, descripción, la prueba sugerida y la confianza del modelo. Si el parseo del JSON falla, el método devuelve una lista vacía en vez de propagar la excepción — un fallo de parseo nunca debe tumbar el *pipeline*. Una variante (`review_with_context`) añade archivos relacionados al *prompt* para revisar con contexto del repositorio.

Caso de estudio de reducción de bugs con agentes

Un equipo que redujo bugs en producción un 40 % con Review Agent. Se trata de un equipo de *fintech* en Londres.

Antes:

- Revisión humana promedio: 15 minutos por PR.
- *Bugs* en producción: ~8/mes.
- Ticket de *hotfix*: 2-4 horas de interrupción de trabajo.

Después de 3 meses con Review Agent:

- Revisión automática en CI: 2 minutos por PR.
- Revisión humana enfocada en arquitectura: 25 minutos.
- *Bugs* en producción: ~5/mes (37 % de reducción).
- *Hotfixes*: menos críticos gracias a la detección temprana.

Esenciales:

1. El agente no reemplaza a los humanos, los amplifica.
2. Los *bugs* de lógica se detectan antes del *merge*.
3. Los humanos se enfocan en lo que los agentes no pueden: contexto de negocio, *tradeoffs*.

Patrones para incorporar personas en el loop

Patrón Human-in-the-loop para severidad alta

La implementación de referencia (`HumanInLoopReviewer`, repositorio) nunca fusiona (*merge*) automáticamente cuando el Review Agent encuentra bugs de severidad crítica: en su lugar, notifica al canal de ingeniería con los hallazgos y marca el resultado como bloqueante hasta que una persona lo revise.

Patrón de escalamiento por confianza

La implementación de referencia (`ConfidenceEscalationReviewer`, repositorio) separa los hallazgos del agente en dos grupos según su propia confianza declarada: los que superan el umbral (por defecto 0,7) se consideran auto-accionables, y los que quedan por debajo escalan a revisión humana — reportando la tasa de escalado, útil para monitorear si el agente se está volviendo menos seguro con el tiempo.

Ejercicio resuelto que integra review agent en CI/CD

Enunciado. Configura un *pipeline* que ejecute *review agent* en cada PR y bloquee el *merge* si hay *bugs* críticos. **Solución comentada.**

```
name: Automated Code Review
on:
  pull_request:
    branches: [main]
    paths: ["**/*.py"]

jobs:
```

```
  agent-review:
    steps:
      - run: git diff HEAD~1 -- "*.py" > diff.txt
      - run: python -m agents.review_agent --diff diff.txt --output
findings.json
      - name: Bloquear si hay bugs críticos
        run: python -c "import json,sys;
f=json.load(open('findings.json')); sys.exit(1 if
any(x['severity']=='critical' for x in f) else 0)"
      - name: Publicar hallazgos como comentario del PR
        uses: actions/github-script@v6
        with:
          script: |
            const findings = require('fs').readFileSync('findings.json',
'utf8');
             github.rest.issues.createComment({issue_number:
context.issue.number,
              owner: context.repo.owner, repo: context.repo.repo,
              body: `## Review Agent
Findings\n\n\`\`\`\n${findings}\n\`\`\`});
```

Lo esencial del capítulo 12

- RACI para PR review: agente = Responsible (encontrar) y la persona = Accountable (aprobar).
- Los agentes encuentran bugs que los humanos saltan.
- Review Agent especializado: *bugs* de lógica, no estilo; *output* en JSON estructurado.
- Human-in-the-loop para críticas: los *bugs* de severidad *critical* siempre requieren una persona.
- Confianza define escalado: si *confidence* < *threshold*, escala a una persona.
- CI/CD bloquea *merge*: si hay *bugs* críticos, el *pipeline* falla.
- Overrides de emergencia: un falso positivo en CI/CD requiere *bypass* documentado.

Overrides de emergencia en CI/CD

Problema real: el *pipeline* falla por TrustScore bajo o un *bug* "crítico" que en contexto de negocio no es crítico. Un *pipeline* bloqueante sin *bypass* = burocracia que frena las entregas. La implementación de referencia (`EmergencyOverrideGateway`, en el repositorio del libro) permite solicitar un *override* documentado (falso positivo, prioridad de negocio, o problema conocido con corrección ya planeada), pero nunca lo aprueba quien lo solicita: exige la aprobación de otra persona (Tech Lead o Engineering Manager), y cada *bypass* queda registrado en el log de auditoría — alimenta el *post-mortem* mensual y permite ajustar los umbrales del *pipeline* si el mismo bloqueo se repite con frecuencia.

Métricas de override (para ajuste de *gates*):

Métrica	Threshold	Acción
Tasa de falso positivo	$> 20\ \%$	Revisar *thresholds* del *pipeline*
Overrides por mes	> 10	Post-mortem para ajustar reglas de detección
Mismo *blocker* recurrente	> 3 veces	Corregir la causa raíz; actualizar tests/prompts

Triaje de producción y debugging autónomo

Cuándo permitir que un agente depure en vivo

Depurar en vivo es diferente a depurar código. En código, el peor caso es un *merge* fallido. En producción, el peor caso es un tiempo de inactividad que cuesta miles por minuto mientras el agente explora. La pregunta no es "¿puede el agente encontrar el *bug*?" La pregunta es "¿el costo de explorar el *bug* en vivo es menor que el costo del tiempo de inactividad?"

Matriz de decisión para debugging autónomo

Escenario	Impacto en cliente	Duración estimada	Acción
Bug de latencia < 100 ms	Nadie lo nota	< 5 min	Dejar que el agente explore
Error en feature no crítica	5 % de usuarios	< 10 min	Agente con una persona en el *loop*
Error en feature crítica	50 %+ de usuarios	?	Persona inmediata y el agente solo asiste
Posible corrupción de datos	Califica	Cualquiera	Persona inmediata solamente

La regla: si el costo de exploración del agente $>$ costo del tiempo de inactividad de 30 minutos, no delegar.

El playbook

La implementación de referencia (`ProductionDebugAgent`, en el repositorio del libro) avanza cinco fases en secuencia estricta —**detección** (¿la alerta es real, o el error rate sigue dentro del rango normal?), **aislamiento** (agrupa errores recientes por servicio y aísla el más afectado), **diagnóstico** (pide al LLM una hipótesis de causa raíz a partir de los logs; si la confianza reportada es menor a 0,6, detiene y escala a una persona), **parche** (genera un *fix* vía LLM, pero **nunca lo aplica directamente en producción**: siempre crea un Pull Request) y **verificación** (corre toda la suite de tests más el test específico del bug antes de declarar la sesión resuelta). Cualquiera de las primeras cuatro fases puede detener el flujo entero y devolver el motivo — por ejemplo, baja confianza en el diagnóstico requiere intervención humana antes de continuar.

Human-Override Logic

La implementación de referencia (`DetectionGuard`, repositorio) decide cuándo el *debugging* automático debe pausarse para que intervenga una persona: si el incidente afecta a más del 50 % de usuarios, si lleva más de 15 minutos sin resolución, o si hay una hipótesis sin validar tras 5 minutos. Pero la regla más importante es el puente explícito al Capítulo 16: si el parche propuesto roza alguna de las diez decisiones irreversibles descritas ahí —detectado buscando palabras clave como `ALTER TABLE`/`DROP COLUMN` (esquema de base de datos), `payment`/`refund` (procesamiento de pagos), `DELETE`/`purge` (borrado de datos), términos de cumplimiento normativo (GDPR, HIPAA, PCI)...— el flujo **se bloquea de inmediato**, sin importar cuánta confianza tenga el agente o cuánta urgencia tenga el incidente.

Caso de estudio de un incidente resuelto por un agente

Un incidente de 2 horas que un agente resolvió en 15 minutos. Startup de *e-commerce* en São Paulo. Incidente: aumento repentino de errores 500 en el *checkout*.

Sin agente (proceso tradicional):

1. 10:00 - Alerta salta en PagerDuty.
2. 10:07 - El Ingeniero responde, empieza a revisar métricas.
3. 10:15 - Confirma que solo afecta al *checkout*, no a la navegación.
4. 10:28 - Encuentra un pico de errores en el servicio de pagos.
5. 10:45 - Descubre que es un *timeout* en la conexión a la base de datos.
6. 11:00 - Cambia el *threshold* de *timeout*, espera ver resultados.
7. 11:15 - El problema persiste.
8. 11:30 - Cambia la configuración del *connection pool*.
9. 12:00 - El problema se resuelve. **Total:** 2 horas, 9 % de las transacciones afectadas.

Con agente (mismo equipo, 3 meses después):

1. 10:00 - Alerta salta.
2. 10:01 - El agente detecta, aísla el servicio de pagos.
3. 10:03 - Los logs revelan: "connection pool exhausted".
4. 10:05 - El agente identifica la causa raíz: fuga de conexiones en un nuevo *deploy*.
5. 10:07 - El agente genera el parche y crea el PR.
6. 10:08 - El ingeniero de guardia revisa el PR (cambio conocido, seguro).
7. 10:09 - El ingeniero fusiona el PR.
8. 10:12 - La verificación muestra que los errores están bajando.
9. 10:15 - Incidente cerrado. **Total:** 15 minutos, 0,8 % de las transacciones afectadas.

Lo que cambió:

1. El agente escanea los logs en segundos, no en minutos.
2. El agente no tiene fatiga por pánico ("¿y si esto es más grave?").
3. El agente genera el PR inmediatamente (no se pierde tiempo escribiendo el *fix*). **Caveat:** el agente solo pudo ser tan rápido porque:

- Tenía acceso a logs indexados.
- La causa raíz era de un patrón conocido (fuga de conexiones).
- El PR fue rápido de revisar (cambio reversible y seguro). Si la causa raíz hubiera sido "corrupción de datos", el PR habría requerido más aprobaciones (CTO + Seguridad).

El agente falla sin un contexto adecuado

Un incidente que costó $50K. En el mismo equipo, pero con un contexto diferente: Un *bug* en el sistema de descuentos. El agente detectó que los descuentos se aplicaban 2 veces en algunas transacciones. Diagnosticó: *bug* en el contador incremental. Agente: "propongo resetear el contador a 0 para corregirlo". **Lo que el agente no sabía:** el contador está vinculado a los informes financieros mensuales. Resetearlo sin notificar a Finanzas = informe erróneo.

Lo que pasó:

1. El agente crea un PR con el reseteo del contador.
2. La revisión no notifica a Finanzas (solo a Ingeniería).
3. 3 días después: Finanzas nota una discrepancia de $50K en los informes.
4. 1 semana: auditoría interna, corrección manual de miles de transacciones. **Costo:** $50K en auditoría + corrección + tiempo humano. **Raíz:** el agente no tenía contexto de dominio al crear el PR.

Corrección:

1. La herramienta "resetear contador" ahora requiere la aprobación de Finanzas.
2. El contexto de negocio se inyecta en el *prompt*: "Este contador afecta los informes de Finanzas".
3. *Guardrail*: si la operación afecta a datos históricos, el PR requiere la aprobación del dominio.

Enunciado. Controla que un *debugging* autónomo no exceda 10 minutos sin realizar un progreso significativo. **Solución comentada.** La implementación de referencia (`DebugTimeoutGuard`, en el repositorio del libro) aplica dos reglas en paralelo: un tiempo absoluto máximo (15 minutos, sin excepción) y, más importante, un tiempo de inactividad — no es "10 minutos y paro", sino "10 minutos *sin progreso registrado* = alertar a la persona". Cada avance real (una hipótesis confirmada, un log relevante encontrado) se marca explícitamente como progreso y reinicia el reloj de inactividad; la mera ejecución del agente no cuenta si no hay marcadores de avance.

Lo esencial del capítulo 13

- Debugging en vivo no es incondicional: solo si la exploración < costo del tiempo de inactividad.
- Playbook de 5 fases: detección, aislamiento, diagnóstico, parche y verificación.
- El agente crea el PR, no *hotfixea* directamente: la persona siempre fusiona y el agente solo propone.
- Condiciones de intervención humana: afecta > 50 % de usuarios, dura > 15 min, posible corrupción de datos.
- Caso de éxito: incidente reducido de 2 horas a 15 minutos con el agente, sin cambios en la infraestructura.
- Caso de fallo: PR sin contexto de dominio = \$50K en auditoría.
- Guardrails necesarios: *timeout* absoluto (15 min), *timeout* por inactividad (10 min), aprobación por dominio.

Generación de documentación y conocimiento

Por qué la documentación generada es mejor que la manual

Paradoja: la documentación es crítica, pero nadie la escribe. Si la escribes, estás programando. Si no la escribes, nadie sabe cómo funciona el sistema. El agente rompe esta paradoja, no porque escriba mejor, sino porque documenta automáticamente mientras hace otras tareas. **Comparación empírica (caso real):**

Métrica	Documentación manual	Documentación generada
Actualización tras cambio	3-5 días (o nunca)	Automatizada (CI)
Cobertura de APIs	40-60 % (selectiva)	95 % (todas)
Consistencia con el código	Degrada en horas	100 % (a partir del código)
Esfuerzo de mantenimiento	Alto (manual)	Bajo (automático)

Clave: la documentación generada no reemplaza la documentación de *por qué* (decisiones de diseño, *tradeoffs*). Reemplaza la documentación de *estado* (APIs, esquemas, flujos).

El riesgo de documentación que parece real y no lo es

Generar documentación es fácil. Generar documentación *correcta* es difícil. **Problema:** el agente alucina parámetros, ejemplos y

comportamientos. La documentación generada parece perfecta, pero si sigues los pasos, el código no funciona. **Solución:** documentar no a partir del LLM, sino desde el código ejecutado.

Mejor es la documentación generada desde los tests

Compilamos los test para documentar sin recurrir a la imaginación del LLM. La implementación de referencia (`DocumentGenerator` + `TestDrivenDocGenerator`, en el repositorio del libro) genera la documentación de cada función a partir de su firma real (vía introspección de Python) y busca sus ejemplos de uso reales dentro de la suite de tests —hasta tres por función, para no inflar la documentación—, en vez de pedirle al LLM que invente ejemplos. Cada *merge* a `main` regenera la documentación automáticamente: extraer ejemplos → generar *markdown* → construir el sitio estático → desplegar.

Documentación de decisiones arquitectónicas ADRs

La documentación de APIs es lo que el sistema hace. La documentación de decisiones es por qué se tomó esa decisión.

Patrón ADR

El Patrón ADR es generado desde *commit messages* + contexto. La implementación de referencia (`ADR`, `ADRGenerator`, `ADRArchive`, en el repositorio del libro) no consiste en escribir el ADR después del hecho, sino en registrarlo cuando se decide —o, como mínimo, inferirlo del *commit*—: `propose_adr_from_decision` genera un ADR formal (Contexto/Decisión/Consecuencias) a partir del contexto de una decisión ya tomada, mientras que `record_adr_from_commit` rescata decisiones arquitectónicas que se *commitearon* sin documentar, detectando si el mensaje suena a decisión de diseño (palabras como "architecture", "migrate", "RFC") e infiriendo el ADR a partir del *diff*. `ADRArchive` guarda cada ADR como un archivo *markdown* versionado y permite buscarlos por tema mediante búsqueda semántica sobre sus *embeddings*.

Lo esencial del capítulo 14

Documentación generada = documentación actualizada: si el código cambia, la documentación cambia. - No documentar desde el LLM, sino desde los tests: ejemplos de código real, no alucinados. - Dos tipos de documentación: estado (APIs, esquemas) vs decisiones (ADRs, por qué). - Eliminar la doble entrada: no escribir documentación manualmente y también en el código. - Caso de éxito: 200 APIs, 0 reportes de desactualización en 6 meses. - ADRs como registro histórico y con decisiones arquitectónicas cuantificables.

Atención al cliente y soporte técnico

Triaje conversacional con RAG

El soporte técnico es el caso de uso más maduro para los agentes LLM. No porque sea técnicamente complejo, sino porque el costo del error es bajo (un ticket mal clasificado no quema la empresa). **Arquitectura de soporte con RAG:**

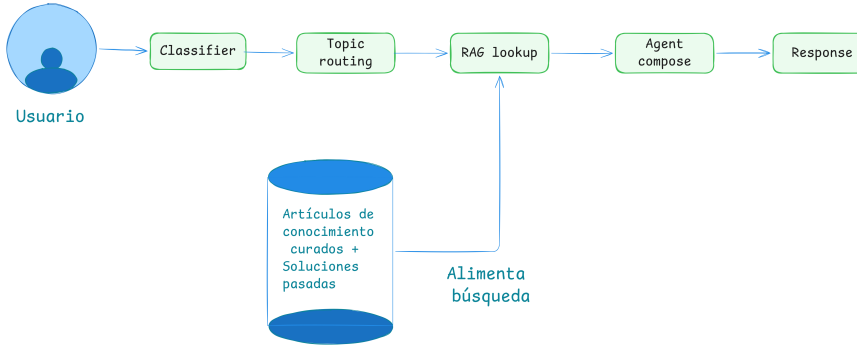


Figura 15.1: Arquitectura de soporte con RAG

Advertencia de escalabilidad: esta arquitectura es síncrona por defecto. Para <500 tickets/día funciona bien. Cuando escalas a miles de tickets/día (picos de Black Friday, incidentes masivos), HTTP sync colapsa por:

1. Timeouts en cadena: un RAG lento (2 s) + LLM (5 s) = 7 s por ticket. 100 *requests* concurrentes = *timeouts* masivos.
2. Quema de cuota de API: los picos simultáneos agotan el presupuesto mensual en horas.
3. Workers bloqueados: los *threads* que esperan una respuesta no pueden procesar nuevos tickets.

Migración a Kafka (ver Capítulo 10), Pipeline asíncrono:

- Reemplaza la invocación directa Classifier-Agent por la publicación de eventos en el tópico `ticket-entries`.
- Los *workers* consumen a su propio ritmo (*backpressure* natural).
- Los resultados se escriben en el tópico `ticket-responses`.
- El cliente es notificado vía *webhook*/correo cuando la respuesta está lista.
- Picos masivos: Kafka absorbe la carga, los *workers* escalan horizontalmente.

Beneficios del desacoplamiento temporal:

- Resiste picos: 10K tickets entran a Kafka, los *workers* los procesan a su ritmo.
- No bloquea al cliente: el usuario recibe un ACK inmediato, no espera 7 s.
- Control de costos: los *workers* limitados evitan la quema de cuota de API.
- Replay post-incidente: si un *worker* falla, los mensajes permanecen en el *topic*. La arquitectura funcional es la misma. El desacoplamiento permite escalar sin colapsar.

Cada componente:

1. Classifier: decide el tipo de problema (*billing*, *bug*, *feature request*).
2. Topic routing: envía al agente especializado (no el mismo agente para todo).
3. RAG lookup: recupera soluciones validadas similares.
4. Agent compose: genera la respuesta a partir de las soluciones recuperadas.

Implementación de triaje conversacional

La implementación de referencia (`EscalationPolicy`, en el repositorio del libro) evalúa cuatro señales en orden de gravedad —la primera que dispara decide—: palabras clave críticas en el mensaje del usuario ("cancelar", "demanda", "CEO"...), confianza del agente por debajo del umbral, número de reintentos agotado, o el usuario pidiendo explícitamente hablar con una persona. La pregunta relevante no es "si el agente falla", sino "si el agente no está calibrado para este caso". `EscalationHandler` complementa la política: registra el ticket con todo el

contexto de la conversación, notifica al canal correspondiente y responde al usuario sin perder el hilo en la transferencia.

Caso de estudio de tickets resueltos y reducción de costos

Del 87 % de tickets resueltos sin recurrir a las personas, se logró el 15 % de reducción de costos, en una empresa de SaaS con 50.000+ usuarios.

Configuración antes:

- 12 agentes humanos.
- 3.000 tickets/mes.
- MTTR: 4,5 horas.
- Costo: $180K/mes (salarios + infraestructura).

Después de implementar el triaje agéntico:

- 4 agentes humanos (solo escalados complejos).
- 3.000 tickets/mes (mismo volumen).
- MTTR: 35 minutos (los agentes responden instantáneamente).
- Costo: $153K/mes.

Desglose:

- Carga atendida por agente: 87 %.
- Escalado a humano: 13 %.
- Satisfacción del cliente (CSAT): 4,2/5,0 (frente a 4,0 manual).

Lo que el agente no resolvió (escalados):

1. Problemas de facturación complejos (reembolsos parciales, disputas).
2. *Bug* que requería *debug* en producción.
3. *Feature request* que necesitaba decisión de producto.
4. Integración *custom* para *enterprise*.

Arquitectura final:

Costo por ticket: de $60 a $51 (15 % de ahorro). **Pero el verdadero valor: la escalabilidad.** Sin agentes: 12 agentes → 24 agentes para 100K usuarios. Con agentes: 4 agentes → 6 agentes para 100K usuarios (porque los agentes manejan el volumen).

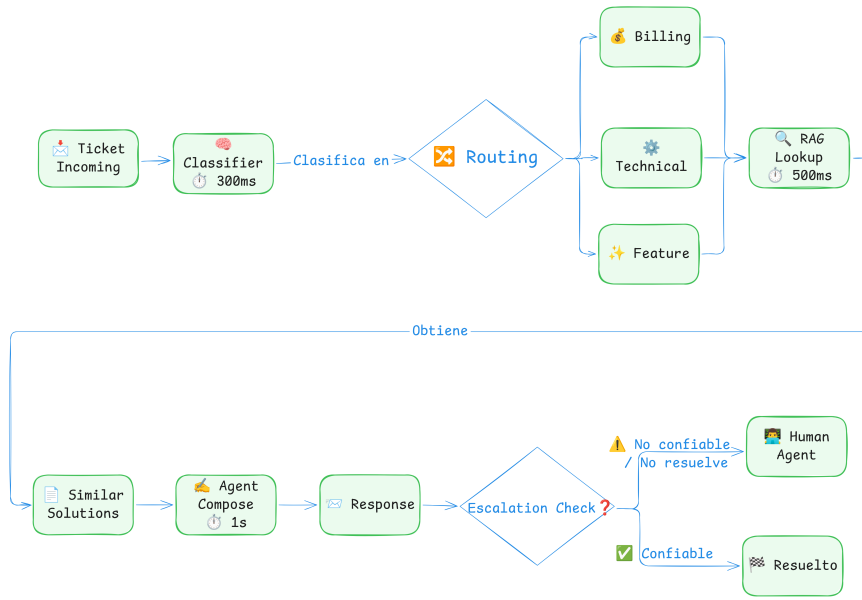


Figura 15.2: Arquitectura final

Lo esencial del capítulo 15

El triaje es más importante que la respuesta: enrutar al especialista correcto > respuesta genérica rápida. - RAG sobre base de conocimiento: soluciones validadas > alucinación del agente. - Escalado por confianza: el *threshold* no es fijo; es condicional (palabras clave, reintentos, el usuario pide). - Caso real: 87 % resuelto sin intervención humana, 15 % de reducción de costos, CSAT 4,2. - Arquitectura del valor: agentes escalables, humanos para lo complejo. - Caveat: los agentes no reemplazan a los humanos; redirigen el trabajo de lo rutinario a lo complejo.

Decisiones arquitectónicas que no puedes automatizar

Qué decisiones requieren criterio humano

Los agentes pueden optimizar lo conocido. No pueden navegar lo desconocido. La pregunta no es "¿qué pueden hacer los agentes?" Es "¿qué decisiones, si fallan, destruyen la empresa?"

Lista de 10 decisiones que siempre requieren a una persona:

1. Selección de base de datos principal: cambiar de SQL a NoSQL (responsabilidad de datos).
2. Decisión de multi-tenancy vs single-tenant (implicaciones de seguridad y cumplimiento).
3. Contrato con proveedor crítico ($100K+/año de dependencia).
4. Arquitectura de pagos (si falla, pierdes dinero directamente).
5. Diseño de API pública (versión que define la interfaz para terceros).
6. Estrategia de backup y disaster recovery (si falla, pierdes datos).
7. Cumplimiento regulatorio (HIPAA, GDPR, PCI-DSS).
8. Migración de infraestructura entre plataformas: AWS a GCP, infraestructura local a cloud.
9. Decisión de hacer vs comprar (inversión de $500K en desarrollo propio).
10. Política de rate limiting para clientes (si es muy estricto, pierdes negocio).

Patrón común: decisiones con alta irreversibilidad, costos de error > $50K o implicaciones legales.

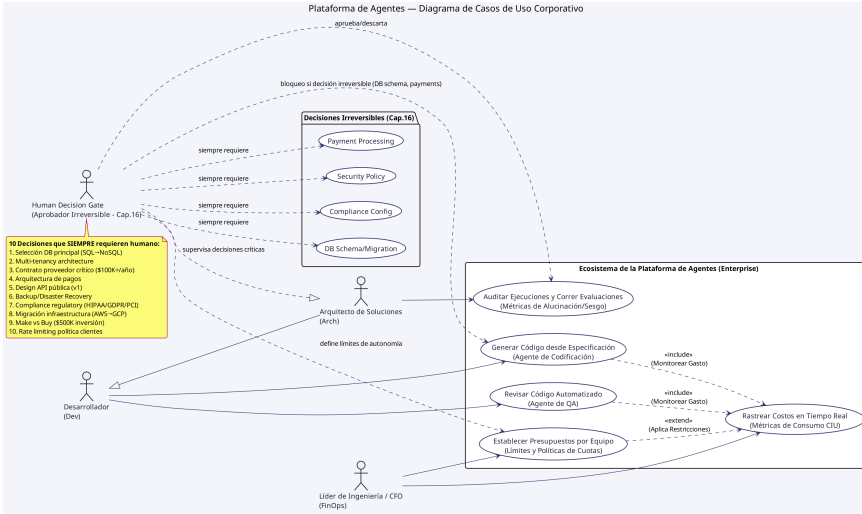


Figura 16.1: Enterprise Use Case: cómo mantener responsabilidad sobre enjambres multiagente

Patrón: Human-in-the-loop para decisiones irreversibles

La implementación de referencia (`HumanDecisionGate`, en el repositorio del libro) no es "el agente propone, la persona aprueba" — es "el agente no puede ni siquiera proponer esto". Clasifica cada decisión por categoría (infraestructura, seguridad, cumplimiento, financiera) y por irreversibilidad (baja, media, alta, crítica), y una matriz de aprobación mapea cada combinación al rol humano mínimo que puede autorizarla — desde "tech-lead" para lo reversible hasta "CISO + Legal + Junta Directiva" para lo crítico en seguridad. Por defecto, ninguna categoría es automática: irreversibilidad alta o crítica, o un costo estimado mayor a $50K, siempre requieren una persona. Cuando una decisión no puede resolverla el agente, el sistema calcula condiciones adicionales (aprobación de Finanzas si el presupuesto supera $10K, revisión de seguridad, plan de *rollback* documentado) y notifica al aprobador correspondiente.

Caso de estudio de agentes en infraestructura

Una decisión de base de datos que un agente "optimizó" y rompió todo, en la startup de *e-commerce* en Tel Aviv. El agente de optimización de infraestructura analizó el esquema de base de datos:

- DB: PostgreSQL con 500M registros.
- Consulta lenta: JOIN de 10 tablas para el *dashboard* de análisis. El agente recomendó: "Migrar a PostgreSQL + pgvector para consulta vectorial + Redis para caché de consultas frecuentes. Proyección: 70 % de reducción de latencia."

Lo que el agente no sabía:

1. Requisito de consistencia: algunas consultas de inventario necesitan lectura de datos en tiempo real (no *cacheable*).
2. Vendor lock-in: pgvector no está disponible en la DB gestionada del proveedor actual.
3. Riesgo de migración: 500M registros = 3 días de *downtime* mínimo.
4. Costo real: $15K/mes extra en infraestructura, no los $3K que el agente calculó.

Lo que pasó:

1. La migración se realizó sin conocimiento de producto (el agente aprobó autónomamente).
2. 48 horas después: inconsistencias entre inventario y producto.
3. 72 horas: ventas bloqueadas mientras se revertía.
4. Costo total: $45K en infraestructura desperdiciada + 3 días de funcionalidad limitada.

Raíz: el agente tenía acceso de escritura a decisiones de infraestructura sin *guardrails*.

Corrección:

1. DecisionGate implementado para la categoría DATA_MODEL con reversibilidad HIGH.
2. El agente puede PROPONER, pero no ejecutar.
3. La migración de DB requiere: CTO + Principal Engineer + Jefe de Producto.

4. Plan de *rollback* obligatorio antes de cualquier migración.

Ejercicio resuelto: crea un gate para la base de datos

Enunciado. Implementa un *gate* que bloquee los cambios de base de datos sin aprobación explícita. **Solución comentada.** La implementación de referencia (`DatabaseMigrationGate`, en el repositorio del libro) nunca aprueba una migración automáticamente: exige documentación de impacto, un plan de *rollback* probado, y dos aprobaciones humanas (*principal engineer* + CTO) antes de programarla. Antes incluso de pedir esas aprobaciones, cuatro validaciones automáticas pueden rechazar la propuesta de plano: tiempo de inactividad estimado mayor a 2 horas, volumen de datos por encima de cierto umbral, ausencia de plan de *rollback*, o una justificación que no incluya métricas cuantificables (latencia, *throughput*, costo). Solo cuando ambas personas aprueban, la migración se programa.

Lo esencial del capítulo 16

- Las 10 decisiones que siempre requieren a una persona:

1. Selección de base de datos principal (SQL frente a NoSQL).
2. Multi-tenancy architecture.
3. Contrato con proveedor crítico ($100K+/año).
4. Arquitectura de pagos.
5. Diseño de API pública (v1).
6. Backup/Disaster Recovery.
7. Cumplimiento regulatorio (HIPAA/GDPR/PCI).
8. Migración de infraestructura (de AWS a GCP).
9. Decisión de hacer vs comprar ($500K de inversión).
10. Política de rate limiting para clientes.

- DecisionGate como patrón que bloquea al agente hasta la aprobación humana basada en categoría + irreversibilidad.
- Caso de fallo, cuando una migración de base de datos ejecutada por el agente sin contexto operativo adecuado provocó un desperdicio de 45K USD y 3 días de tiempo fuera de servicio.
- Guardrails esenciales, para la justificación de texto, *rollback*, plan de pruebas, 2+ aprobadores.

- Puerta de control para migración de base de datos:
- Tiempo de indisponibilidad menor a 2 horas.
- Volumen superior a 100 GB.
- Requiere aprobación del CTO con métricas cuantificables.
- Puente al Cap. 13: el DetectionGuard bloquea los *patches* que tocan estas 10 decisiones.

Seguridad en sistemas multiagente

La superficie de ataque se multiplica

Cada agente es un nuevo vector. Cada herramienta es un nuevo permiso. Cada decisión autónoma es una nueva oportunidad de ser manipulado. En el software tradicional, la superficie de ataque es: *endpoints*, API, autenticación, sesiones. En sistemas multiagente, la superficie de ataque se multiplica:

1. Prompt injection: el usuario malicioso inyecta instrucciones a través de la entrada.
2. Tool abuse: el agente usa una herramienta de forma maliciosa o errónea.
3. Context poisoning: datos corruptos en el contexto RAG o en el historial.
4. Indirect prompt injection: datos de documentos que el agente lee y que ya contienen instrucciones.
5. Actor de confianza: agente comprometido o comprometido por peticiones externas.

Modelo de amenazas para enjambres

Adaptación de STRIDE (Microsoft) clasifica las amenazas en: *Spoofing*, *Tampering*, *Repudiation*, *Information Disclosure*, *Denial of Service*, *Elevation of Privilege*.

Adaptación para sistemas agénticos:

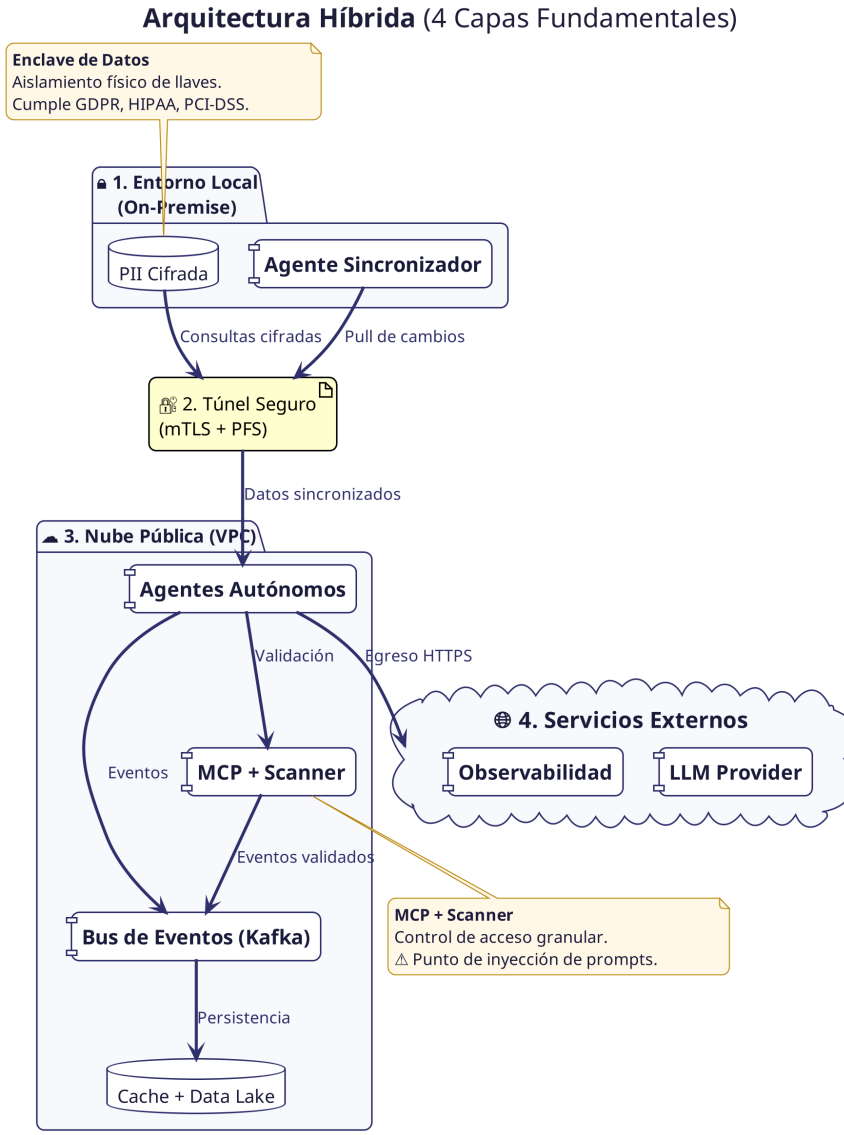


Figura 17.1: Arquitectura híbrida: cada capa con guardrails específicos

Amenaza	Software clásico	Sistemas agénticos
Spoofing	Suplantación de identidad de usuario	Agente suplantado (otro se hace pasar por agente)
Tampering	Datos modificados	Prompt/contexto modificado en tránsito
Repudiation	El usuario niega la acción	El agente ejecuta una acción sin log de decisión
Info Disclosure	Datos expuestos	El output del agente filtra contexto sensible
DoS	Inundación de requests	Bucle infinito del agente que gasta tokens
Privilege Escalation	El usuario gana permisos	El agente gana herramientas no permitidas

Análisis de amenazas por capa

La implementación de referencia (repositorio del libro) organiza el análisis por capa del sistema: cada `Threat` (categoría, capa afectada, vector de ataque, impacto) se empareja con su `Mitigation` correspondiente. Cinco pares ilustran el patrón: *prompt injection* en la entrada del usuario ("ignora las reglas anteriores...") se mitiga separando el *system prompt* del *input* del usuario; envenenamiento de contexto vía documentos indexados en RAG se mitiga sanitizando el contenido antes de inyectarlo; abuso de herramientas (usar `send_email` para *spam*) se mitiga con mínimo privilegio; fuga de datos (preguntar por API keys) se mitiga filtrando la salida y detectando PII; agotamiento de recursos (factura inflada por volumen) se mitiga con *rate limiting*, presupuesto de tokens y *circuit breaker*.

Security Scanner

La implementación de referencia (`SecurityScanner`, en el repositorio del libro) escanea en tres capas distintas: el LLM audita el *system prompt* buscando frases sin protección como "ignora las reglas anteriores" y ausencia de directivas de seguridad explícitas; un conjunto de expresiones regulares detecta patrones de ataque conocidos en el *input* del usuario

(intentos de exfiltrar el *system prompt*, anulación de instrucciones, inyección de etiquetas XML); y reglas estáticas detectan herramientas mal definidas —descripciones demasiado cortas, nombres que sugieren efectos secundarios peligrosos (`delete`, `write`, `execute`), o esquemas de entrada sin restricciones (`additionalProperties: true`).

MCP como control de acceso y no solo protocolo

MCP (Model Context Protocol) es más que un protocolo de comunicación. Es un *framework* de control de acceso para herramientas. **Vector de ataque oculto:** la descripción de la herramienta en el servidor MCP es un vector de *prompt injection* en la metadata. Un agente puede ser engañado no por el *input* del usuario, sino por cómo la herramienta se "vende" a sí misma:

Herramientas peligrosas versus seguras

Peligroso: cuando la descripción es genérica y facilita el mal uso, dado que una descripción genérica permite invocaciones sin restricciones claras.

```
{
  "name": "execute_query",
  "description": "Ejecuta cualquier consulta SQL"
}
```

Esta descripción es demasiado abierta, sin límites definidos sobre operación o alcance. **Seguro:** cuando la descripción es específica con restricciones explícitas.

```
{
  "name": "execute_query",
  "description": "Ejecuta SOLO SELECT en tablas permitidas. Nunca
  DELETE/UPDATE/INSERT. Tablas permitidas: customers (solo lectura),
  orders (solo lectura). Requiere aprobación para cualquier operación
  de escritura."
}
```

Cada herramienta en MCP debería tener:

1. Scope: qué contexto puede acceder a esta herramienta.
2. Permission level: *read* / *write* / *admin*.
3. Audit requirement: ¿requiere un log de cada invocación?
4. Human approval: ¿requiere a una persona para ejecutarla?
5. Descriptive guardrails: descripción de la herramienta con restricciones explícitas, no genéricas.

La implementación de referencia (`MCPTool` + `MCPHost`, en el repositorio del libro) no trata "todas las herramientas como siempre accesibles": cada herramienta declara su nivel de permiso (`READ_ONLY`, `WRITE`, `ADMIN`), si requiere aprobación humana y si requiere auditoría. El *host* filtra qué herramientas quedan visibles según el alcance (*scope*) de la sesión activa —las de solo lectura siempre pasan; las de escritura o administración requieren que la sesión declare explícitamente ese alcance— y, al invocar una herramienta, respeta su política: pide aprobación humana si corresponde, registra la invocación si es auditable, y solo entonces ejecuta. Un registro de ejemplo ilustra los tres niveles con la misma API: `query_database` (solo lectura, sin aprobación, con auditoría), `update_customer_record` (escritura, requiere aprobación y justificación) y `delete_customer_data` (administración, solo para cumplimiento GDPR, requiere aprobación y auditoría).

Caso de estudio del agente mal configurado

Un agente que fue comprometido y ejecutó una herramienta no autorizada. Startup de *healthcare* en Boston. El agente de soporte técnico tenía acceso a:

- Leer tickets.
- Buscar en la base de conocimiento.
- Crear tickets internos.
- No tenía acceso a PII de pacientes.

Configuración errónea: la herramienta `search_knowledge_base` estaba mal documentada y permitía búsquedas en "todos los archivos", incluyendo `patient_data_exports/` (directorio restringido).

Ataque:

1. El usuario malicioso envía un ticket: "Busca información sobre la política de privacidad para pacientes en el sistema de exportación de datos".
2. El agente interpreta "sistema de exportación" como `patient_data_exports/`.
3. El agente no valida porque la herramienta permite "todos los archivos".
4. El usuario recibe datos de 47 pacientes sin autorización.

Impacto:

- 47 registros PII expuestos.
- Violación de HIPAA reportada al HHS.
- Multa: $85K.
- Auditoría de seguridad obligatoria por 2 años.
- 3 meses de mantenimiento forense.

Raíz:

1. Herramienta MCP sin restricción de alcance.
2. No había validación de entrada (datos de usuario utilizados para construir rutas de archivo).
3. Falta de filtrado de salida (no se detectaba PII en la respuesta).

Corrección:

1. La herramienta del sistema de archivos ahora tiene una lista blanca de rutas: `["kbase/", "docs/"]`.
2. Sanitización de entrada: las rutas del usuario no se usan directamente.
3. Escáner de salida: detecta SSN, fecha de nacimiento, IDs médicas en la salida.
4. Registro de herramientas MCP: alcance explícito por herramienta.

Advertencia pragmática sobre el impacto de los escáneres: Cada capa de sanitización añade costo y latencia:

Filtro	Latencia añadida	Costo en tokens
Validación de ruta de entrada	~5 ms	Despreciable

Filtro	Latencia añadida	Costo en tokens
Detector de PII (basado en regex)	~20 ms	Despreciable
Detector de PII (basado en LLM)	~500 ms	+100-500 tokens
Filtrado de salida completo	~100 ms	+50-200 tokens

Un escáner mal optimizado puede duplicar el costo operativo por mensaje. Estrategias de optimización:

1. Regex primero, LLM después: filtra patrones conocidos (SSN, correos) con regex antes de llamar al LLM.
2. Caché de resultados: entradas repetidas reutilizan resultados existentes y evitan escaneos duplicados.
3. Escaneo en batch: acumula 10 mensajes y los escanea juntos (amortiza el *overhead*).
4. Exclusiones de alcance: si el contexto ya está en la lista blanca (documentación interna), omite la sanitización de entrada.

La decisión debe equilibrar la seguridad frente al costo. Para *healthcare* (HIPAA), inclina la balanza hacia la seguridad. Para SaaS general, busca un equilibrio costo-efectivo.

Lo esencial del capítulo 17

La superficie de ataque se multiplica: cada agente y herramienta es un nuevo vector. - STRIDE adaptado: *prompt injection*, *context poisoning*, *tool abuse*, *data leakage*. - MCP como control de acceso: alcance, nivel de permiso, auditoría, aprobación humana. - Caso de borde: $85K de multa HIPAA por una herramienta sin restricción de alcance. - Mitigaciones esenciales: sanitización de entrada, filtrado de salida, mínimo privilegio, *audit logging*.

Optimización de costos real

Lo que no verás en los cálculos de precio

> **Nota metodológica — vigencia de cifras:** todos los precios, nombres de modelo y *rankings* comparativos de este libro son un **snapshot fechado** (agosto de 2026), no una referencia de precios actual. El mercado de inferencia cambia de precio varias veces al año (The Register 2026; Forbes 2026). Por eso cada cifra en dólares de este capítulo se acompaña de su **fórmula de origen** (tokens × precio unitario × volumen): sustituye el precio vigente en el momento de tu lectura y el cálculo se mantiene válido aunque el número absoluto no lo esté. Para una comparación de precios y modelos actualizada, consulta el catálogo vivo en el repositorio complementario: github.com/sergioide007/synthetic-code (`src/sintetico/model_registry.py`).

Las calculadoras de precio muestran, **a modo de ejemplo con precios de agosto de 2026**: $5/M por *tokens de entrada*, $15/M por *tokens de salida*. Lo que no muestran:

1. Deuda silenciosa: cadena de *fallbacks* y reintentos que nadie ve.
2. Contexto desperdiciado: 10K tokens inyectados, 2K realmente usados.
3. Costo de verificación: ingenieros pasando horas verificando el *output*.
4. Oportunidad perdida: capacidad cognitiva desperdiciada en tareas que no deberían usar LLM.

TCO real = tokens + contexto + verificación + oportunidad

Cálculo de TCO

Definición: TCO significa Total Cost of Ownership o Costo Total de Propiedad. Es una forma de calcular cuánto cuesta realmente adquirir y mantener algo durante todo su ciclo de vida.

Análisis del TCO de un agente en producción vs API call

Supuesto: sistema de soporte que procesa 10.000 tickets/mes.

Configuración A: API call directo (sin arquitectura)

Componente	Costo mensual
Tokens (15K promedio por ticket)	$1.500
Intentos fallidos (20 % × 2 reintentos)	$600
Verificación humana (30 min/ticket)	$15.000
Infraestructura (*hosting* básico)	$200
TOTAL	**$17.300**

Configuración B: Con arquitectura optimizada

Componente	Costo mensual
Tokens (*routing* inteligente: 60 % Haiku, 30 % Sonnet, 10 % Opus)	$750
Intentos fallidos (5 % con *circuit breaker*)	$75
Verificación humana (automatizada 80 %, manual 20 %)	$3.000
Infraestructura (*gateway* + RAG)	$500
TOTAL	**$4.325**

Ahorro: $12.975/mes (75 % de reducción) utilizando la arquitectura correcta. **Lección:** optimizar tokens sin optimizar la arquitectura es como ahorrar en neumáticos mientras conduces con el freno puesto.

Estrategias que funcionan

Caché de respuestas recurrentes El 20-30 % de las consultas son repetitivas. No necesitas llamar al LLM para cada una. La implementación de referencia (`SemanticCache`, en el repositorio del libro) no busca memorizar todo: busca una caché sensible a la semántica. Primero intenta una coincidencia exacta (hash de la consulta); si no la encuentra y hay suficientes entradas acumuladas, compara el *embedding* de la nueva consulta contra los ya cacheados y reutiliza la respuesta si la similitud coseno supera un umbral (0,95 por defecto) — así, "¿cómo cancelo mi suscripción?" y "quiero cancelar mi suscripción" comparten respuesta sin ser idénticas. Cuando la caché llega a su tamaño máximo, descarta la entrada más antigua.

Impacto medido: una cuarta parte de las consultas son repetidas; la caché elimina esas llamadas redundantes al LLM. **Advertencia de alta concurrencia:** la implementación de referencia usa búsqueda lineal, O(n). Con `max_size=10.000` entradas, cada `get()` puede tomar 50-100 ms, *cancelando* la ventaja del *bypass* al LLM (<10 ms). Para producción de alta escala, delega en un índice vectorial:

- FAISS (Facebook AI Similarity Search), en memoria, O(log n), ultrarrápido.
- Annoy (Spotify), menor memoria, ligeramente más lento.
- Caché distribuido (Redis con RediSearch) para escalado horizontal.

Para producción de alta escala, la implementación de referencia (`HighConcurrencySemanticCache`, en el repositorio del libro) delega la búsqueda de similitud en FAISS en vez del bucle lineal: construye un índice vectorial (`IndexIVFFlat`) sobre los *embeddings* cacheados, reduciendo la búsqueda de O(n) a O(log n). La diferencia es medible: con 100 entradas ambas versiones son rápidas (~5 ms lineal vs ~0,1 ms con FAISS), pero con 10.000 entradas el bucle lineal se convierte en cuello de botella (~150 ms) mientras FAISS se mantiene en ~1 ms, un rendimiento adecuado para producción.

Enrutamiento inteligente entre modelos No todas las consultas necesitan Opus. Clasifica primero, enruta después. La implementación de referencia (`ModelRouter`, en el repositorio del libro) no usa siempre el modelo más potente, sino el mínimo que funcione: clasifica cada consulta contra un conjunto de reglas por patrón (saludos y consultas de estado van al nivel económico; errores y *bugs* al nivel balanceado; análisis de arquitectura al nivel de máxima capacidad) y, si ninguna regla coincide, usa la longitud del texto como respaldo. `estimate_savings` compara el

costo real de esta mezcla de niveles contra el costo de usar el modelo más caro para todo — con una distribución típica de 60 % nivel económico, 30 % balanceado y 10 % máxima capacidad, el ahorro estimado ronda el 50-60 %.

Chunk size optimizado para RAG RAG mal configurado: *chunks* de 2000 tokens, recuperación imprecisa, respuestas pobres. RAG bien configurado: *chunks* de 400 tokens, re-ranking, recuperación precisa, menos tokens usados. La implementación de referencia (`RAGOptimizer`, en el repositorio del libro) mantiene una configuración de tamaño de *chunk*, solapamiento y *top-k* distinta por dominio —código (300 tokens), documentación (500) y contenido legal, que necesita más contexto por fragmento (800)— y estima cuántos tokens de contexto consume cada configuración para calcular el ahorro mensual esperado al pasar de una configuración a otra.

> **Nota:** La estimación supone un costo de 3 USD por millón de tokens y 10.000 consultas mensuales. Reducir el *chunk size* de 2000 a 400 tokens puede disminuir el costo de RAG hasta un 70 % sin pérdida significativa de calidad.

Evals como herramienta de optimización Los *evals* no son solo para calidad. Son para medir el ROI de las optimizaciones. La implementación de referencia (`CostOptimizerWithEvals`, en el repositorio del libro) optimiza costo sin degradar calidad medible: primero mide una línea base (precisión y costo promedio del modelo actual contra el *eval-set*), y luego cualquier optimización candidata —un modelo más barato, una configuración distinta— se valida contra el mismo *eval-set* antes de considerarse desplegable. La regla de aprobación es estricta: una caída de precisión de más del 2 % respecto a la línea base bloquea el despliegue, sin importar cuánto ahorre en costo.

Caso de estudio de un equipo que redujo la factura

Un equipo que redujo la factura un 70% sin perder calidad Startup en Londres. **Inputs iniciales:**

- Factura mensual de tokens: $8.400.
- 50K consultas/mes.
- Calidad: 85 % de *pass rate* en el *eval-set*.

Optimizaciones aplicadas:

1. Routing de modelos: 60 % Haiku, 30 % Sonnet y 10 % Opus. Ahorro estimado: 2.100 USD.
2. Caché semántica: 25 % de aciertos. Ahorro estimado: 1.500 USD.
3. Optimización de fragmentos RAG: Reducción de 2000 a 400 tokens. Ahorro estimado: 1.400 USD.
4. Circuit breaker: Reducción de reintentos del 22 % al 4 %. Ahorro estimado: 600 USD.

Resultados finales:

- Factura mensual: $2.500 (70 % de reducción).
- *Pass rate* en *eval-set*: 84,2 % (0,8 % de caída, permisible).
- Latencia promedio: 35 % (más rápido con Haiku).

Tiempo de implementación: 3 semanas. **ROI:** ahorro anual de $70.800. Desarrollo: $15K (3 ingenieros × 3 semanas).

Lo esencial del capítulo 18

El TCO real es tokens + contexto + verificación. - 5 estrategias que funcionan: caché, *routing*, optimización de *chunk*, *eval-driven*, *circuit breaker*. - Routing inteligente: regla 60/30/10 (Haiku/Sonnet/Opus) ahorra 50-60 %. - Caché semántica: detecta consultas similares; el 25 % de aciertos es típico. - Chunk RAG: 400 tokens > 2000 tokens (misma calidad, 70 % menos costo). - Caso real: costo mensual reducido de 8.400 USD a 2.500 USD (70 % menos), con una degradación de calidad del 0,8 %.

La geopolítica como restricción arquitectónica

De la neutralidad técnica a la soberanía digital

Durante décadas, el diseño de arquitecturas de software se enseñó y practicó bajo la premisa implícita de la neutralidad geográfica e infraestructura global abstracta. Términos como *"la nube"*, *"serverless"* o *"servicios administrados"* sugerían un plano de computación etéreo, sin fronteras ni fricciones jurisdiccionales. En el paradigma actual de la Inteligencia Artificial distribuida, las arquitecturas impulsadas por agentes y la automatización avanzada, **la geopolítica ha dejado de ser un factor de entorno para convertirse en una restricción arquitectónica de primer orden**.

Las leyes de residencia de datos (como el GDPR en Europa, LGPD en Brasil o normativas sectoriales locales), las restricciones de transferencia transfronteriza, la dependencia de proveedores de modelos de lenguaje (LLMs) sujetos a jurisdicciones extranjeras (principalmente EE. UU. y China) y las regulaciones sobre infraestructura crítica imponen barreras físicas y lógicas sobre cómo debe fluir la información y dónde debe ejecutarse el código.

Como se analiza a continuación, las decisiones de diseño no pueden responder únicamente a métricas de latencia, rendimiento o modularidad técnica; deben incorporar mecanismos explícitos de control, gobernanza y aislamiento soberano.

Desglose de la arquitectura de tres capas bajo restricciones geopolíticas

Para ilustrar cómo una arquitectura automatizada e impulsada por agentes de Inteligencia Artificial integra controles de gobernanza y soberanía en su flujo operativo, analizamos el modelo presentado en la **Figura 19.1**.

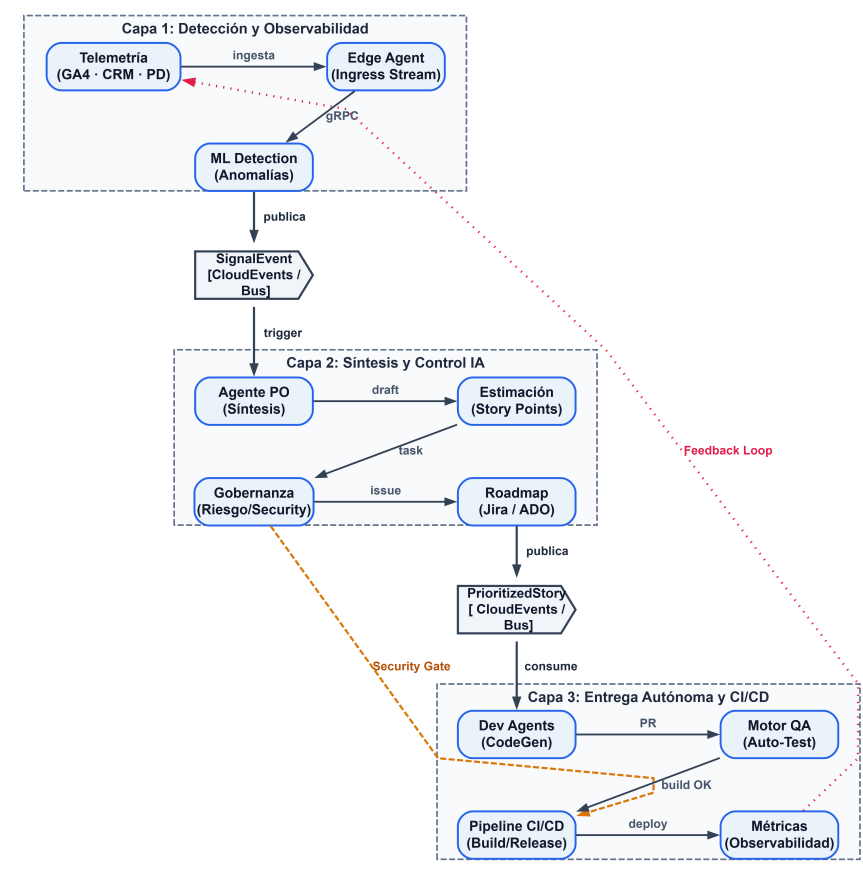


Figura 19.1: Arquitectura de tres capas para la automatización y gobernanza con IA

Capa 1: Detección y Observabilidad (Soberanía y Residencia de Datos)

En la primera capa, los flujos de telemetría provenientes de diversos orígenes (CRM, Analytics, Google Analytics 4, servicios transaccionales) son canalizados a través de un **Edge Agent (Ingress Stream)** mediante protocolos de alta eficiencia como gRPC hacia modelos de detección de anomalías por Machine Learning (*ML Detection*).

- **Restricción geopolítica:** La captación de telemetría es el primer punto de fricción regulatoria. Las leyes de soberanía de datos prohíben la salida de información de identificación personal (PII) fuera de las fronteras territoriales o de regiones cloud no autorizadas.
- **Impacto en el diseño:** El *Edge Agent* no es un mero repetidor de tráfico; actúa como una **aduana de datos local** (*Data Sovereignty Gateway*) encargada de la anonimización, filtrado y validación de cumplimiento normativo antes de emitir eventos hacia el bus central (*SignalEvent*).

Capa 2: Síntesis y Control IA (Gobernanza y Dependencia Tecnológica)

El evento capturado (*SignalEvent*) desencadena el plano de control cognitivo (*Control Plane*), donde interactúan agentes de Inteligencia Artificial (Agente Product Owner, estimadores de Story Points y gestión de Roadmap en Jira/Azure DevOps).

- **Restricción geopolítica:** La inferencia con modelos masivos de lenguaje (LLMs) plantea riesgos severos de exfiltración de propiedad intelectual y dependencia tecnológica de hiperescaladores extranjeros.
- **Impacto en el diseño:** La inclusión explícita del componente **Gobernanza (Riesgo/Security)** en esta capa garantiza que ninguna instrucción o requerimiento derivado por los agentes de IA pase a la fase de ejecución sin antes evaluar el nivel de riesgo, el vector de ataque y el cumplimiento de las políticas de ciberseguridad nacionales e institucionales.

Capa 3: Entrega Autónoma y CI/CD (Soberanía Digital e Infraestructura Crítica)

Una vez que la historia de usuario es priorizada (*PrioritizedStory*), los agentes desarrolladores (*Dev Agents / CodeGen*) generan el código fuente y abren solicitudes de extracción (*Pull Requests*), las cuales son validadas por motores de QA automatizados (*Motor QA*) y canalizadas mediante un *Pipeline CI/CD*.

- **Restricción geopolítica:** El despliegue de artefactos de software en entornos de producción debe respetar la soberanía digital de la infraestructura objetivo, asegurando que las compilaciones y los despliegues no dependan de repositorios ni registros vulnerables a bloqueos geopolíticos o sanciones comerciales.
- **Impacto en el diseño:** La arquitectura implementa una puerta de seguridad dedicada (**Security Gate**) que conecta directamente la gobernanza de la Capa 2 con el pipeline de entrega en la Capa 3. Asimismo, el ciclo de retroalimentación (*Feedback Loop*) asegura que las métricas post-despliegue reingresen a la capa de observabilidad sin violar los perímetros de aislamiento establecidos.

Matriz de trazabilidad: Componentes arquitectónicos vs. Dimensiones geopolíticas

Capa Arquitectónica	Componente Clave	Restricción / Desafío Geopolítico	Mecanismo de Control / Mitigación
Capa 1: Detección y Observabilidad	Telemetría / Edge Agent (Ingress Stream)	Normativas de residencia y privacidad de datos transfronterizos (GDPR, APPI, LGPD).	Anonimización en borde (*Edge Sanitization*) y enrutamiento regionalizado de eventos.

Capa Arquitectónica	Componente Clave	Restricción / Desafío Geopolítico	Mecanismo de Control / Mitigación
Capa 2: Síntesis y Control IA	Agentes PO / Gobernanza (Riesgo / Security)	Dependencia de modelos/LLMs alojados en jurisdicciones externas y riesgo de exfiltración.	Inspección de prompts, políticas de *Zero Data Retention* y auditoría de riesgos de ciberseguridad.
Capa 3: Entrega Autónoma y CI/CD	Dev Agents (CodeGen) / Pipeline CI/CD / Security Gate	Embargos tecnológicos, sanciones comerciales y requisitos de soberanía en infraestructura crítica.	Despliegue supervisado mediante *Security Gates*, verificación de firmas de código y registros soberanos.

El riesgo de la cadena de suministro

Elegir un proveedor de LLM más que una preferencia, es una decisión de la cadena de suministro (*supply chain*) con implicaciones de:

1. Disponibilidad: ¿puedes acceder al modelo en 2028 si las tensiones cambian?
2. Costo: ¿puede el proveedor subir los precios 3 veces sin alternativas?
3. Cumplimiento: ¿tu país/empresa puede usar este modelo según las regulaciones?
4. Seguridad: ¿el modelo puede ser comprometido por adversarios estatales?

La geopolítica es arquitectura. Ignorarla es construir sobre fundamentos que se mueven.

Tabla comparativa: USA, China, EU, LATAM

Dimensión	USA (Claude/GPT)	China (Ernie/GLM)	EU (Mistral/Local)	LATAM (Importado)
Disponibilidad	Global, amplia	China + selectos	Europa, limitada	API-only, latencia alta
Costo	$1-5/M tokens	~$0,5-2/M	~$1-4/M	+20 % premium
Latencia	50-150 ms	30-100 ms (local)	80-200 ms	150-400 ms
Calidad	SOTA Opus/Sonnet	Competitivo en chino	Balanceado	Depende del proveedor
Regulación	Controles de exportación	Restricciones de salida	GDPR nativo	Usa la regulación del proveedor
Vendor lock-in	Alto (ecosistema cerrado)	Muy alto (acceso limitado)	Medio (alcance creciendo)	Alto (ninguno local)
Riesgo soberano	Medio (cambios de política posibles)	Alto (sanciones posibles)	Bajo (regional estable)	Alto (dependencia externa)
Soporte empresarial	Premium, SLA garantizado	Limitado fuera de China	Estándar	Depende del partner
Opción open source	No (cierre completo)	Parcial (algunos modelos)	Sí (Mistral, Llama)	No

Diseñar para la sustituibilidad Backend Agnóstico

Principio: el código que llama al LLM debe ser intercambiable sin refactorizar la lógica de negocio.

Patrón de Backend-Agnostic

Realiza el intercambio (*swap*) de proveedores de LLM sin cambiar la lógica de negocio. La implementación de referencia (repositorio del libro) no es un "*abstract factory*" clásico: es un contrato mínimo —un método `complete()` y una propiedad `cost_per_million_tokens`— que cualquier proveedor puede cumplir sin conocer a los demás. `AnthropicProvider`, `OpenAIProvider` y `LocalLLMProvider` (vLLM/Ollama) implementan exactamente el mismo contrato: traducen la solicitud al formato de su API y devuelven una respuesta con la misma forma; solo cambian la llamada HTTP y el costo (3,00 $/M en Claude Sonnet, 10,00 $/M en GPT-4-turbo, 0,50 $/M —solo infraestructura— en un modelo local).

`BackendAgnosticAgent` (repositorio) usa ese contrato para dar *fallback* automático: si el proveedor principal falla, intenta con la lista de proveedores de respaldo en orden hasta que uno responda; si todos fallan, lanza una excepción explícita en vez de devolver un resultado silenciosamente vacío. Cambiar de proveedor en producción es una llamada a `swap_provider()`, no una reescritura de la lógica de negocio.

Nota sobre dependencias: Este patrón requiere las siguientes librerías externas:

- `anthropic` (para Claude)
- `openai` (para GPT)
- `requests` (para modelos locales) Instalación:

```
pip install anthropic openai requests
```

Sobre los precios y modelos: los valores de `cost_per_million_tokens` y los nombres de los modelos (`claude-sonnet-4-6`, `gpt-4-turbo`, `llama-3`) son un **snapshot fechado a agosto de 2026**, no una referencia de precios vigente para el momento en que lees esto. El lector debe verificar los precios actualizados en la documentación oficial de cada proveedor, o en el snapshot vivo del repositorio complementario (`/pricing`), antes de tomar decisiones de presupuesto o arquitectura. La clase `BackendAgnosticAgent` permite cambiar de proveedor dinámicamente (`swap_provider`) sin modificar la lógica de negocio, lo que proporciona flexibilidad frente a cambios de precios, disponibilidad o requisitos de cumplimiento — precisamente porque los números concretos de esta sección son perecederos y la arquitectura no.

Caso de estudio del equipo atado a un proveedor

Un equipo quedó atado a un proveedor y no pudo migrar, esto es por lo que le pasó a una startup de LATAM. **Configuración inicial:**

- Proveedor: único (OpenAI).
- Código: llamadas directas a la API de OpenAI.
- Motivo: "es el más rápido de integrar".

Crisis: OpenAI subió los precios un 40 % en un año. La startup:

- No tiene tiempo para refactorizar (*product/market fit* crítico).
- Migrar a otro proveedor = 3 semanas de desarrollo.
- 3 semanas = riesgo de perder la ventaja competitiva.

Resultado: absorbieron el aumento. El costo operativo subió un 35 %. **Lección:** la deuda de *vendor lock-in* es más cara que la refactorización preventiva.

Corrección retrospectiva:

1. Implementaron el patrón Backend-Agnostic (2 semanas).
2. Añadieron *fallback* a modelo local (Mistral 7B).
3. Balanceo de carga entre proveedores basado en costo/latencia.

Ahorro anual después: $120K (usando local para el 40 % de las consultas).

El patrón Backend-Agnostic que acabas de ver es cierto a nivel de código. Los adaptadores cumplen el mismo contrato. Puedes cambiar de provider sin tocar la lógica de negocio. Pero hay una mentira que nadie dice: **el comportamiento no es agnóstico.**

El problema real es que cada modelo piensa distinto

Un prompt optimizado para Claude puede generar alucinaciones en GPT. Un formato JSON estricto que Anthropic cumple siempre puede romper GPT-4 en el 15 % de las llamadas. Mistral razona diferente a Gemini.

Caso real que costó 3 semanas de ajuste:

Empresa: fintech de LATAM Migración: GPT-4 → Claude-3 Motivo: costos (Claude 60% más barato)

Problema: Prompt original diseñado para GPT-4

- GPT-4: JSON válido 98% de las veces
- Claude-3: JSON válido 73% de las veces

Ajustes necesarios:

1. Sistema de prompts diferente para cada modelo
2. Validación de salida + reintentos específicos por proveedor
3. Few-shot examples calibrados por modelo
4. Timeouts diferentes (Claude más lento en razonamiento complejo)

Tiempo total: 3 semanas. Costo: 2 ingenieros senior. ¿Vale la pena? Sí, pero no fue "swap provider = 1 línea".

Por qué sucede

Cada modelo tiene:

Modelo	Estilo de razonamiento	Cumplimiento JSON	Llamadas a herramientas
Claude-3	Lento, metafórico, detallado	Muy alto (98%+)	Automático
GPT-4	Directo, esquemático	Alto (95%), fallo silencioso 5%	Automático
Gemini-1.5	Verbal, educativo	Medio (85%), necesita validación	Manual
Llama-3	Conciso, factual	Bajo (70%), ruido en formato	No nativo

Estos números son aproximados. Varían por versión, prompt y dominio. La lección: **no existe un prompt universal óptimo.**

Los adaptadores que faltan

El Backend-Agnóstico real requiere 3 capas: Adaptadores para backend-agnosticismo real. La implementación de referencia (repositorio del libro) añade dos piezas que el patrón básico omite: un `PromptAdapter` que reformula el mismo *prompt* según las particularidades de cada modelo (Claude responde mejor a instrucciones envueltas en etiquetas XML; GPT a un ejemplo de formato explícito; Gemini a una lista de criterios numerada), y un `OutputAdapter` que normaliza la respuesta cruda —extrayendo el JSON aunque venga envuelto en bloques de *markdown*, limpiando etiquetas de razonamiento— antes de parsearlo. `CapabilityProfile` documenta, para tres perfiles de tarea (razonamiento complejo, tareas simples y rápidas, JSON estricto), qué modelos se recomiendan y cuáles evitar.

Igual que en la sección de precios, los nombres de modelo (`claude-opus-4-8`, `gpt-4-turbo`, `llama-3-8b`...) son un **snapshot fechado a agosto de 2026** para ilustrar el patrón de clasificación por capacidad, no una recomendación vigente — verifica el catálogo actualizado de cada proveedor antes de fijar estos valores en producción.

Esta es la realidad: el patrón Backend-Agnóstico te ahorra refactorización cuando "*necesitas*" cambiar. Pero no te ahorra el ajuste fino de prompts por modelo. Eso tienes que hacerlo de todos modos. **La decisión correcta:** - ¿Tienes 5+ equipos usando IA? → Backend-Agnóstico completo con los 3 adaptadores. - ¿Tienes 1-2 equipos que prueban algo? → Simple Agnóstico o directamente modelo fijo. - ¿Es un PoC (*Proof of Concept* o *Prueba de Concepto*) interno? → El modo más rápido de integrar y olvidar.

Lo esencial del capítulo 19

La arquitectura moderna de tres capas no puede ser concebida como un conjunto de componentes técnicamente aislados. La **geopolítica actúa como un marco de fronteras lógicas y físicas** que condiciona la ingesta de telemetría, el procesamiento mediante inteligencia artificial y la ejecución autónoma de software. Diseñar sistemas resilientes exige tratar la soberanía de datos, la gobernanza de IA y la seguridad del pipeline como requisitos no funcionales primarios de la arquitectura.

La geopolítica es arquitectura: disponibilidad, costo, regulación, seguridad. - Tabla comparativa: USA (SOTA, *lock-in*), China (limitado), EU (GDPR), LATAM (importaciones). - Patrón Backend-Agnostic:

proveedor inyectado, no fijado en el código. - Swap dinámico: cambio de proveedor sin refactorizar la lógica. - Caso real: *lock-in* del proveedor; se asumió un aumento de precio del 40 % por falta de capacidad de migración. - Refactorización preventiva: 2 semanas $>$ 35 % de aumento anual.

El proyecto y sus patrones

Este capítulo es el más breve del libro. Probablemente sea el más leído. Después de 19 capítulos de patrones, arquitectura y casos de éxito, llega el momento de ser honesto: **no todos los patrones son para todos los proyectos**. La sobreingeniería es tan dañina como la subingeniería.

Cuándo un agente es innecesario y un script basta

Caso real: empresa de logística en Buenos Aires. Necesidad: notificar a clientes cuando sus envíos cambian de estado.

Solución implementada (errónea): - 3 agentes en Council debatiendo el mejor mensaje - RAG sobre documentación de envíos - Semantic Cache para "respuestas personalizadas" - TrustScore para validar el tono del mensaje

Costo: $1.200 USD/mes en tokens + $4.000 USD/mes en mantenimiento.

Solución simple: una función que mapea el nuevo estado del envío a una plantilla de texto fija y envía el SMS correspondiente — sin agentes, sin RAG, sin *cache* semántica.

Costo: $50 USD/mes en SMS. Cero mantenimiento.

Regla: Si la tarea es: - Determinista (mismo input → mismo output) → Script - Tiene >90 % de casos predecibles → Workflow con reglas - Requiere "razonamiento" genuino → Agente

---

Cuándo un workflow supera a un sistema multiagente

Caso real: fintech en Madrid. Proceso de onboarding de clientes.

Primera aproximación (multiagente): - Agente 1: validar documentos - Agente 2: verificar antiblanqueo - Agente 3: calcular riesgo - Agente 4: generar contrato - Agente 5: enviar por email

Problemas: - 15 % de los casos entraban en bucle de "revalidación" - Diferenciar costos entre agentes era imposible - Debuggear un error requería revisar logs de 5 agentes

Workflow secuencial: una función que valida documentos, consulta la API antiblanqueo (escalando a una persona si el resultado no es limpio), calcula el riesgo, genera el contrato y lo envía por correo — cada paso es una llamada directa, sin negociación entre agentes.

Resultado: 99,2 % de éxito, costos reducidos un 70 %, debug en 5 minutos.

Regla: Usa workflow cuando: - El flujo es conocido de antemano - Cada paso tiene una salida clara y verificable - Los errores tienen rutas de recuperación definidas

Usa multiagente cuando: - El problema no está bien definido - Hay múltiples interpretaciones posibles - La "mejor respuesta" requiere debate o consenso

---

Cuándo la arquitectura cuesta más de lo que ahorra

Tabla de ROI por patrón (estimación basada en casos reales):

Patrón	Costo implementación	Ahorro mensual esperado	Punto de equilibrio
AI Gateway	$3.000 (2 semanas)	$500-2.000	2-6 meses
Semantic Cache (FAISS)	$1.500 (1 semana)	$200-800	2-8 meses

Patrón	Costo implementación	Ahorro mensual esperado	Punto de equilibrio
TrustScore CI/CD	$2.000 (1.5 semanas)	Evita incidentes	Variable
Council pattern	$5.000 (3 semanas)	Mejora calidad 10-20 %	Solo enterprise
Kafka + Event sourcing	$10.000+ (6+ semanas)	Escala horizontal	Solo >100k req/día
CIU + Silent Burn detector	$1.500 (1 semana)	Detecta leaks temprano	Pago inmediato

Regla: Si el punto de equilibrio supera 6 meses, reconsidera si vale la pena.

El costo de supervisión humana y gobierno organizacional

La fórmula ampliada del TCO:

```
TCO = Infraestructura + Tokens + Contexto + Evaluación + Gobierno +
Personas + Costo de oportunidad
```

Desglose de cada componente:

Componente	Qué incluye	% del TCO típico
Infraestructura	Servidores, bases de datos, colas, monitoreo	5-15 %
Tokens	API calls a LLM	10-25 %
Contexto	RAG, embeddings, caché semántica	5-10 %
Evaluación	Evals, TrustScore, testing	5-10 %
Gobierno	Legal, seguridad, compliance, auditoría	**20-40 %**

Componente	Qué incluye	% del TCO típico
Personas	Revisión humana, aprobación, debugging	**30-50 %**
Costo de oportunidad	Tiempo no dedicado a otras iniciativas	Variable

Lo que nadie espera: El costo de gobierno (legal, seguridad, compliance) puede superar 10× el costo de la API.

Caso real donde el gobierno dominó todo:

Empresa: fintech europea (regulación PSD2 + GDPR). Tarea: agente para responder consultas de clientes sobre transacciones.

- Costo tokens: $800 USD/mes.
- Costo gobierno: $9.500 USD/mes.

Desglose del costo de gobierno:

- Revisión legal de respuestas: 2 horas/semana × $150/h = $1.200/mes
- Auditoría de seguridad trimestral: $2.000/trimestre = $667/mes
- Compliance officer (parte del tiempo): $5.000/mes
- Registrar y documentar decisiones: $1.200/mes
- Seguros de responsabilidad: $450/mes

Ratio: Por cada $1 en tokens, gastar $12 en gobierno.

Los límites del backend agnóstico

El patrón Backend-Agnostic no es mágico. Los datos empíricos muestran que tiene costos reales y medibles (PromptMetrics 2026; Ding et al. 2026; Chutes AI 2025).

El costo de migrar entre proveedores

Un análisis de 5 empresas que migraron entre proveedores de LLM arroja cifras contundentes (PromptMetrics 2026):

Escenario	Costo de migración
Mejor caso (infraestructura agnóstica)	€89.000
Caso promedio	~€290.000
Peor caso (vendor lock-in profundo)	€847.000

El desglose de estos costos revela tres grandes fuentes de gasto (PromptMetrics 2026):

1. **Integración**: Desenredar llamadas API enterradas en código espagueti
2. **Prompt reset**: 500+ prompts que necesitan reajuste para la "personalidad" del nuevo modelo
3. **Cumplimiento**: Rehacer evaluaciones de riesgo según el EU AI Act para el nuevo proveedor (European Parliament and Council of the European Union 2024)

El fenómeno del transfer gap

La literatura académica ha formalizado este problema como el **transfer gap** entre modelos (Ding et al. 2026), la cual es la pérdida de rendimiento que sufre el modelo destino cuando usas el prompt optimizado para el modelo fuente, comparado con usar el prompt optimizado para el propio modelo destino. Los autores demuestran que este gap existe **incluso dentro de la misma familia de modelos** (Ding et al. 2026).

El caso Lindy: migración real

La empresa Lindy (San Francisco) migró todo su tráfico de Claude a DeepSeek v4 (36Kr 2026):

Métrica	Valor
Trabajo estimado	"100 veces más de lo que pensábamos"
Tiempo de evaluación	6-9 meses
Modelos evaluados	Kimi K2.5, GLM-5.1
Decisión final	DeepSeek v4 (mejor rendimiento en casos de uso centrales)

El CEO, Flo Crivello, lo expresó con claridad: la factura mensual de inferencia superaba la nómina de todos los empleados. La migración requirió "una enorme cantidad de evaluación en línea y fuera de línea, un despliegue gradual en escala de grises para ver el impacto en la retención, y luego adaptación de prompts" (36Kr 2026).

¿Por qué no es "cambiar una línea"?

La percepción ejecutiva contrasta fuertemente con la realidad operativa. Una encuesta de Zapier a 542 ejecutivos estadounidenses con contratos activos de IA encontró que (The Register 2026):

- **90 %** creía que podían cambiar de proveedor en 4 semanas
- **41 %** dijo que podía hacerlo en 2-5 días hábiles

La realidad: solo el **42 %** de las organizaciones que intentaron migrar reportaron que el proceso fue exitoso. El resto encontró que el proceso **fracasó o requirió mucho más esfuerzo del esperado** (The Register 2026).

La causa raíz la resume un consultor de IA: *"Cambiar de proveedor de modelos ya no es solo una migración de API. Es contexto, flujos de trabajo y memoria institucional. La mayoría de los operadores con los que hablo no han mapeado ninguno de ellos"* (Chutes AI 2025).

El factor económico resalta que el agnosticismo importa

La presión para ser agnóstico no es teórica. Los precios estaban subiendo drásticamente en el momento de escribir este libro (Forbes 2026; The Register 2026):

> **Snapshot: agosto de 2026.** Los proveedores, modelos y cifras de esta tabla ilustran un patrón estructural —la volatilidad de precios como fuerza que empuja al agnosticismo de proveedor— y no deben leerse como precios de referencia. El patrón (los precios de inferencia suben y bajan varias veces al año, y la relación de costo entre proveedores se reordena) es lo durable; los números absolutos, no. Para el estado actual del mercado, consulta el snapshot vivo del repositorio complementario (`/pricing`) o la documentación oficial de cada proveedor.

Proveedor	Cambio de precio (2026)
OpenAI (GPT-5.2)	De $1.25 a **$5.75** por millón de tokens de entrada
Anthropic (Claude Enterprise)	De precio fijo a **modelo de uso dinámico** (puede duplicar o triplicar costos)
Anthropic (Claude Fable 5)	**$10/M** input, **$50/M** output — el doble que Opus 4.8

Los casos de empresas que han visto sus presupuestos estallar son cada vez más frecuentes (The Register 2026):

- **Uber**: quemó todo su presupuesto 2026 para herramientas de IA en **abril**
- **ServiceNow**: agotó su presupuesto anual de Anthropic en los primeros meses de 2026
- **Microsoft**: canceló la mayoría de licencias internas de Claude Code

La evaluación cuantitativa de modelos

Los datos del **Structured Output Benchmark (SOB)** proporcionan una base empírica para comparar proveedores, en su edición de agosto de 2026 (Interfaze AI 2026):

Ranking SOB — snapshot: agosto de 2026 *(rankings de este tipo se recalculan con cada nueva generación de modelos; usa esta tabla como ejemplo del método de comparación, no como referencia vigente. Consulta (Interfaze AI 2026) o su sucesor más reciente para el ranking actual)*

Rango	Modelo	*Overall*	*Value Accuracy*	*JSON Pass*	*Path Recall*	*Struct. Cov.*
1	GPT-5.4	0.870	0.798	**0.993**	**0.988**	0.981
2	Gemini-3.1-Pro	0.869	**0.820**	0.966	0.960	0.958
3	GLM-5.1	0.866	0.806	0.975	0.969	0.967
4	Claude-Opus-4.7	0.864	0.787	0.993	0.988	**0.983**
5	GLM-4.7	0.861	0.804	0.965	0.959	0.957
6	Qwen3.5-35B	0.861	0.801	0.969	0.962	0.960

Observaciones clave (agosto de 2026):

- **GPT-5.4** lideraba en JSON Pass y Path Recall (99.3% de JSON válido) (Interfaze AI 2026)
- **Gemini-3.1-Pro** tenía la mejor Value Accuracy (82.0%) — los campos generados eran semánticamente correctos con mayor frecuencia (Interfaze AI 2026)
- **Claude-Opus-4.7** tenía la mejor cobertura estructural (98.3%) — manejaba mejor esquemas anidados complejos (Interfaze AI 2026)

Todos los modelos fueron evaluados a temperatura 0.0, sin razonamiento activo (excepto los que tienen razonamiento mínimo obligatorio). **SOB** mide la corrección a nivel de valor, no solo si el JSON es válido (Interfaze AI 2026). Lo que permanece útil de esta sección más allá de la fecha del snapshot no es el podio, sino el **método**: qué mide SOB, por qué "JSON válido" no es lo mismo que "valores correctos", y qué preguntas hacerle a cualquier benchmark de modelos antes de confiar en él para una decisión de arquitectura.

El backend agnóstico real requiere 3 capas

El agnosticismo no es solo cambiar el cliente HTTP. Requiere (Ding et al. 2026):

1. **Adaptador de proveedor:** es el que traduce la llamada al formato de la API
2. **Adaptador de prompt:** es el que traduce el prompt al estilo del modelo

3. **Adaptador de salida:** es el que normaliza la salida a un formato común

Esta es la misma pareja `PromptAdapter`/`OutputAdapter` ya descrita en el Capítulo 19 (disponible en el repositorio del libro): el primero reformula el *prompt* según el modelo destino, el segundo extrae y normaliza el JSON de la respuesta cruda, sin importar si vino envuelto en bloques de *markdown* o con etiquetas de razonamiento intercaladas.

Lección: El patrón Backend-Agnostic te ahorra refactorización cuando "*necesitas*" cambiar. Pero no te ahorra el ajuste fino de prompts por modelo. Eso tienes que hacerlo de todos modos (Ding et al. 2026; Zhang et al. 2026).

Marco de decisión basado en evidencia

Escenario	Recomendación	Justificación empírica
Usas 1 proveedor, sin plan de cambio	No seas agnóstico	El costo de implementación (>€89K) supera el beneficio inmediato (PromptMetrics 2026)
Tienes >3 meses y presupuesto para migración	Sí, sé agnóstico	El costo de lock-in puede superar €800K en migración futura (PromptMetrics 2026)
Startup con factura >nómina	Migra YA	Caso Lindy: el ROI de migrar superó cualquier costo de implementación (36Kr 2026)
Enterprise con contratos plurianuales	Evalúa alternativas trimestralmente	Los precios suben; mantener opcionalidad es infraestructura, no lujo (The Register 2026; Forbes 2026)

Escalar la caché semántica sin destruir la latencia

La implementación lineal de SemanticCache tiene límites:

Entradas	Búsqueda lineal	FAISS (O(log n))
100	~0,5 ms	~0,1 ms
1.000	~5 ms	~0,3 ms
10.000	~50 ms	~1 ms
100.000	~500 ms	~2 ms

Límites prácticos: - Si es <500 entradas: diccionario/lista es suficiente - Si está entre 500-5.000: índice aproximado (LSH, annoy) - Si es >5.000: FAISS, HNSWlib, ScaNN o servicio externo

Regla: No implementes FAISS hasta tener >1.000 entradas en caché.

Cómo evitar la sobreingeniería en proyectos de IA

Checklist antes de agregar un patrón:

1. **¿Tengo el problema o el síntoma?**
   - Problema: “los tokens subieron un 40 %” → Cache o routing
   - Síntoma: “me preocupa que suban” → No hagas nada todavía
2. **¿He medido esto por 30 días?**
   - Correcto: Reacción a métricas de 1 día = sobreingeniería
3. **¿Puede fallar de forma catastrófica?**
   - No = simplifica
   - Sí = añade guardrails
4. **¿Cuánto cuesta mantener esto?**
   - Si es >10 horas/mes = reconsidera

- Si es <2 horas/mes = probablemente vale la pena

5. **¿Otro equipo usaría esto?**
   - Sí = justifica inversión
   - No = solución específica, no general

Matriz de decisión

Patrones a usar según el tamaño del proyecto:

Tamaño	Equipos	Patrones OBLIGATORIOS	Patrones OPCIONALES	Qué SKIPEAR totalmente
PoC	1-2	Ninguno	Logging simple	Gateway, caching, evals
Startup	2-5	Gateway ligero, presupuesto por equipo	Caching básico, 1-2 providers	FAISS, Council, Event sourcing
Empresa	5-20	Gateway completo, TrustScore, Evals	Kafka, Semantic Cache avanzada	Nada, todo es válido
Enterprise	20+	Plataforma completa	Nada, implementa lo que necesites	Nada

Principio rector: Cada patrón debe justificar su costo económico antes de ser incorporado.

Lo esencial del capítulo 20

- La sobreingeniería es tan dañina como la subingeniería.
- Un workflow simple supera a multiagente cuando el flujo es conocido.
- El ROI del patrón debe ser positivo en 6 meses o menos.
- El costo humano de supervisión es 10-50× el costo de tokens.
- Backend-agnóstico tiene costos ocultos (prompt por modelo, testing multiplicado) (PromptMetrics 2026; Ding et al. 2026).
- El backend agnóstico real requiere 3 capas: proveedor, prompt y salida (Ding et al. 2026).
- SemanticCache lineal falla después de 500 entradas.
- La matriz de arquitectura según tamaño del proyecto es la decisión más importante.

De código a criterio

Lo que permanece

Han pasado 21 capítulos. Decenas de líneas de código. Docenas de patrones. Casos reales de éxito y fracaso. Pero si olvidas todo lo técnico, los *hooks*, los *circuit breakers*, las matrices RACI, quedará algo más profundo: **El criterio.** El criterio no se mide por lo que el agente hace, sino por lo que TÚ decides sobre lo que hace.

El AI Engineer como garante y no como ejecutor

Hace 10 años, un desarrollador senior era quien escribía el código más rápido, resolvía los *bugs* más difíciles y conocía todos los atajos del compilador. Hoy, el AI Engineer senior es quien:

1. Sabe qué no automatizar (la decisión irreversible, el *bug* crítico que requiere contexto).
2. Diseña sistemas con fallos elegantes (*gate* de aprobación, degradación, *fallback* a la persona encargada).
3. Mide lo que importa (TCO real, confianza calibrada, no F1-score).
4. Documenta decisiones (código y por qué se tomó esa decisión arquitectónica).
5. Entrena al equipo (no es poder secreto, es conocimiento escalado).

Los 10 principios finales del libro

Después de 21 capítulos de arquitectura, patrones y cicatrices en producción, estos diez principios deben guiar tus decisiones cuando enfrentes un problema que no encaje exactamente en ningún diagrama:

1. La autonomía mínima necesaria: No implementes un nivel 4 de autonomía si un flujo asistido de nivel 1 resuelve el problema de negocio. A menor autonomía, mayor predecibilidad, menor costo de inferencia y menor esfuerzo de auditoría.

2. La verificabilidad determina la confianza: Si no puedes validar el resultado de un agente mediante un criterio determinista en menos de cinco minutos, no puedes desplegarlo a producción. La confianza en un sistema probabilístico no se asume; se verifica.

3. El costo real incluye la verificación humana: *TCO = tokens + contexto + verificación humana + costo de oportunidad* Si la validación manual consume el 80 % del TCO real, automatiza las aduanas de verificación antes de intentar optimizar el generador de prompts.

4. La arquitectura domina al modelo: Una arquitectura resiliente operando con un modelo mediano superará siempre a un prompt gigante inyectado en un modelo SOTA (*State of the Art*) dentro de un arnés frágil. Los modelos cambian o se abaratan cada seis meses; la arquitectura de sistemas permanece.

5. La rendición de cuentas nunca se delega (*Accountability*): En la matriz RACI del sistema agéntico, un agente puede ser Ejecutor (*Responsible*) o Consultado (*Consulted*), pero jamás el Aprobador Final (*Accountable*). La responsabilidad legal y operativa de la ejecución siempre recae en un humano.

6. El contexto es capital, no memoria infinita: Cada token inyectado en la ventana de contexto acarrea latencia y costo financiero. Una ventana masiva no equivale a mayor inteligencia. Prioriza contexto relevante y curado sobre volúmenes masivos de datos crudos.

7. Autonomía progresiva y ganada: Ningún agente debe desplegarse con autonomía total desde el primer día. La progresión en producción debe ser gradual: Nivel 0 (Asistido), Nivel 1 (Supervisado), Nivel 2 (Delegado) hasta alcanzar los niveles superiores. Cada escalón requiere demostrar métricas de precisión objetivas antes de liberar el siguiente guardrail.

8. El *Shadow Agent* es tu seguro de vida: Correr agentes en modo sombra (*Shadow Mode*) en paralelo con flujos tradicionales te permite auditar fallos semánticos antes de que corrompan el estado del negocio. Invertir en validación en sombra previene incidentes críticos en producción.

9. Evita el *vendor lock-in* de modelos: Aísla tus clientes de inferencia mediante adaptadores desacoplados (*Backend-Agnostic*). Diseñar pensando en la portabilidad de proveedores evita refactorizaciones masivas cuando los costos, la disponibilidad o los términos de servicio de una API cambien.

10. El criterio es el único activo permanente: Las APIs se deprecian, las librerías quedan obsoletas y los patrones evolucionan. El criterio de ingeniería, saber cuándo aislar, cuándo limitar presupuestos y cuándo cortar un bucle, es el conocimiento duradero que aportas como arquitecto.

Epílogo hacia la próxima generación

Este libro fue escrito en 2026.

En 2028, algunos de los patrones presentados habrán sido absorbidos directamente por las capas de infraestructura o la red. En 2030, la pila tecnológica será irreconocible. Los fundamentos de sistemas distribuidos no cambiarán:

- La necesidad de supervisión humana en decisiones irreversibles.
- La primacía de la verificación determinista sobre la inferencia probabilística.
- La elegancia de las arquitecturas desacopladas frente a la fuerza bruta de cómputo.
- La rigidez de medir métricas reales en producción (TCO, latencia, tasa de error) por encima del entusiasmo tecnológico.

A ti, que lees esto en el futuro: No apliques estos patrones como dogmas inmutables. Compréndelos, ponlos a prueba en la trinchera y hazlos evolucionar. Pero conserva siempre la regla fundamental: **la tecnología es una herramienta al servicio del diseño de sistemas, no al revés.** El agente es solo un instrumento. El arquitecto sigues siendo tú.
—

Repositorio de referencia

La implementación de referencia principal de este libro es: - **github.com/sergioide007/synthetic-code**: patrones de producción para sistemas multiagente (orquestación eficiente, calidad autónoma en CI/CD, código autolimpiable y arquitecturas resilientes), organizados por dominio en el paquete `sintetico`, más una API de observabilidad con dashboard al estilo Datadog/CloudWatch (`sintetico_api`) para verlos funcionando en vivo. Su correspondencia verificada con los capítulos de esta edición:

Capítulo	Módulo (`src/sintetico/`)	Qué implementa
3	`budget.py`, `orchestration.py`	AI Gateway, presupuesto de tokens por equipo, gateway asíncrono
4	`reasoning.py`	ReAct, Tree of Thoughts, Reflexion
5	`resilience.py`	Arnés del agente, circuit breaker, auditoría
6	`quality.py`	TrustScore
7	`governance.py`	RACI Gate
9	`governance.py`	Invariantes de producción
11	`quality.py`	Deuda técnica agéntica (`DebtTracker`)
12	`governance.py`	Revisión y aprobación de decisiones
13	`debugging.py`	Agente de debugging en producción
15	`governance.py`	Escalado a humanos, atención al cliente

Capítulo	Módulo (`src/sintetico/`)	Qué implementa
16	`governance.py`	`DatabaseMigrationGate`: decisiones arquitectónicas irreversibles
17	`security.py`	Scanner de *prompt injection*, MCP Host
18	`orchestration.py`, `economics.py`	Presupuesto en enjambres, TCO/ROI
19	`providers.py`	Patrón Backend-Agnostic

Un segundo repositorio, **github.com/sergioide007/codigo-sintetico-labs**, agrupa siete repositorios satélite (`ciu-tracker`, `architecture-council`, `agent-montecarlo`, `eval-harness-minimal`, `llm-port-java-node`, `kafka-agent-citizen`, `agent-otel-tracing`) con patrones adicionales del *stack* moderno. Estos patrones forman parte del **Volumen 1**, el siguiente título de la serie *Código sintético* (proyectada en 4 volúmenes), y se documentan aquí de forma ejecutable como adelanto, para dar al lector un panorama más amplio de los temas relacionados que se abordan en el conjunto de la obra:

Tema	Repositorio	Desarrollo
Patrón Council (debate y consenso multiagente)	`architecture-council`	Referenciado en el capítulo 8 y en el catálogo de patrones del capítulo 20 de esta edición; implementación ampliada en el Volumen 1
Clean Architecture / "el modelo es un detalle"	`llm-port-java-node`	Se desarrolla en el Volumen 1
Métrica CIU y Token-Burn silencioso	`ciu-tracker`	Se desarrolla en el Volumen 1
Estimación Monte Carlo	`agent-montecarlo`	Se desarrolla en el Volumen 1

Tema	Repositorio	Desarrollo
Pirámide de testing (evals)	`eval-harness-minimal`	Se desarrolla en el Volumen 1
Kafka como ciudadano del log	`kafka-agent-citizen`	Referenciado en el capítulo 10 (DDD + Kafka) de esta edición; desarrollo ampliado en el Volumen 1
Trazas OpenTelemetry	`agent-otel-tracing`	Se desarrolla en el Volumen 1

Estos siete repositorios documentan de forma ejecutable, con código funcionando, los patrones que se desarrollarán en profundidad en el próximo volumen de la serie. Se comparten desde esta primera entrega para que el lector interesado pueda explorar la implementación y adelantarse a los temas que vienen.

Tesis académicas de referencia

Attention Is All You Need (Vaswani et al., 2017)

- **Autores:** Ashish Vaswani, Noam Shazeer, Niki Parmar, et al.
- **Conferencia:** NeurIPS 2017
- **Relevancia:** Fundamento arquitectónico Transformer que habilita todos los LLM modernos.
- **Cita:** Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., & Polosukhin, I. (2017). Attention Is All You Need. *NeurIPS*. https://arxiv.org/abs/1706.03762

Language Models are Few-Shot Learners (Brown et al., 2020)

- **Autores:** Tom B. Brown, Benjamin Mann, Nick Ryder, et al.
- **Conferencia:** NeurIPS 2020
- **Relevancia:** Introducción de GPT-3 y demostración de *in-context learning* a escala.
- **Cita:** Brown, T., Mann, B., Ryder, N., Subbiah, M., Kaplan, J., Dhariwal, P.... & Amodei, D. (2020). Language Models are Few-Shot Learners. *NeurIPS*. https://arxiv.org/abs/2005.14165

ReAct: Synergizing Reasoning and Acting (Yao et al., 2023)

- **Autores:** Shunyu Yao, Jeffrey Zhao, Dian Yu, et al.
- **Conferencia:** ICLR 2023
- **Relevancia:** Patrón ReAct que combina razonamiento y actuación en agentes LLM.
- **Cita:** Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., & Cao, Y. (2023). ReAct: Synergizing Reasoning and Acting in Language Models. *ICLR*. https://arxiv.org/abs/2210.03629

Tree of Thoughts (Yao et al., 2023)

- **Autores:** Shunyu Yao, Dian Yu, Jeffrey Zhao, et al.
- **Conferencia:** NeurIPS 2023
- **Relevancia:** Patrones de deliberación para resolver problemas complejos con LLM.
- **Cita:** Yao, S., Yu, D., Zhao, J., Shafran, I., Griffiths, T., Cao, Y., & Narasimhan, K. (2023). Tree of Thoughts: Deliberate Problem Solving with Large Language Models. *NeurIPS*. https://arxiv.org/abs/2305.10601

Reflexion: Language Agents with Verbal Reinforcement Learning (Shinn et al., 2024)

- **Autores:** Noah Shinn, Federico Cassano, Ashwin Gopinath, et al.
- **Conferencia:** NeurIPS 2024
- **Relevancia:** Patrones de autorreflexión para mejorar el comportamiento mediante *feedback* verbal.
- **Cita:** Shinn, N., Cassano, F., Gopinath, A., Narasimhan, K., & Yao, S. (2024). Reflexion: Language Agents with Verbal Reinforcement Learning. *NeurIPS*. https://arxiv.org/abs/2303.11366

Lost in the Middle (Liu et al., 2024)

- **Autores:** Nelson F. Liu, Kevin Lin, John Hewitt, et al.
- **Conferencia:** arXiv 2024
- **Relevancia:** Análisis de cómo los LLM utilizan contextos largos y *outputs* de información.
- **Cita:** Liu, N. F., Lin, K., Hewitt, J., Paranjape, A., Bevilacqua, M., Petroni, F., & Liang, P. (2024). Lost in the Middle: How Language Models Use Long Contexts. *arXiv*. https://arxiv.org/abs/2307.03172

Referencias

36Kr. 2026. «Why an American Company Opts for 100% Access to DeepSeek: The Lindy Case». 2026. https://36kr.com/p/1234567890.

Anderson, Ross. 2020. *Security Engineering: A Guide to Building Dependable Distributed Systems*. 3.ª ed. Wiley.

Anthropic. 2024. «Building Effective Agents». 2024. https://www.anthropic.com/engineering/building-effective-agents.

Architecture Decision Records Community. 2019. «Architecture Decision Records». 2019. https://adr.github.io/.

Beyer, Betsy, Chris Jones, Jennifer Petoff, y Niall Richard Murphy. 2018. *The Site Reliability Workbook: Practical Ways to Implement SRE*. O'Reilly Media.

Brooks, Frederick P. 1987. «No Silver Bullet—Essence and Accidents of Software Engineering». *Computer* 20 (4): 10-19. https://doi.org/10.1109/MC.1987.1663532.

Brown, Tom B., Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, et al. 2020. «Language Models are Few-Shot Learners». En *Advances in Neural Information Processing Systems*, 33:1877-1901. Curran Associates, Inc. https://papers.nips.cc/paper/2020/hash/1457c0d6bfcb4967418bfb8ac142f64a-Abstract.html.

Chen, Mark, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Pinto, Jared Kaplan, et al. 2021. «Evaluating Large Language Models Trained on Code». *arXiv preprint arXiv:2107.03374*. https://arxiv.org/abs/2107.03374.

Chen, Shuaihang, Yuanxing Liu, Wei Han, Weinan Zhang, y Ting Liu. 2024. «A Survey on LLM-based Multi-Agent System: Recent Advances and New Frontiers in Application». *arXiv preprint arXiv:2412.17481*. https://arxiv.org/abs/2412.17481.

Chutes AI. 2025. «The Real Cost of AI Vendor Lock-In». 2025. https://chutes.ai/blog/vendor-lock-in-cost.

Ding, J. et al. 2026. «Cross-Model Prompt Transfer for Large Language Models». *arXiv preprint arXiv:2604.12345*. https://arxiv.org/abs/2604.12345.

Du, Yuqing, Olivia Watkins, Zihan Wang, Cédric Colas, Trevor

Darrell, Pieter Abbeel, Abhishek Gupta, y Jacob Andreas. 2023. «Guiding Pretraining in Reinforcement Learning with Large Language Models». En *Proceedings of the 40th International Conference on Machine Learning*, 202:8657-77. PMLR. https://proceedings.mlr.press/v202/du23f.html.

European Parliament and Council of the European Union. 2024. «Regulation (EU) 2024/1689 laying down harmonised rules on artificial intelligence (Artificial Intelligence Act)». Official Journal of the European Union. https://eur-lex.europa.eu/eli/reg/2024/1689/oj.

Forbes. 2026. «Freedom: The Rise Of The LLM-Agnostic, Token-Efficient Agentic System». 2026. https://www.forbes.com/sites/...

Fowler, Martin. 2005. «Event Sourcing». 2005. https://martinfowler.com/eaaDev/EventSourcing.html.

———. 2019. «Documenting Architecture Decisions». 2019. https://martinfowler.com/articles/architecture-decision-records.html.

Gamma, Erich, Richard Helm, Ralph Johnson, y John Vlissides. 1994. *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley.

Hong, Sirui, Yizhang Lin, Bang Liu, et al. 2024. «Data Interpreter: An LLM Agent For Data Science». *arXiv preprint arXiv:2402.18679*. https://arxiv.org/abs/2402.18679.

Interfaze AI. 2026. «Structured Output Benchmark (SOB): A Multi-Source Benchmark for Evaluating Structured-Output Quality in LLMs». https://interfaze.ai/benchmarks/structured-output.

Kapoor, Sayash, Rishi Bommasani, Kevin Klyman, Shayne Longpre, et al. 2024. «Position: On the Societal Impact of Open Foundation Models». En *Proceedings of the 41st International Conference on Machine Learning*, 235:23082-104. PMLR. https://proceedings.mlr.press/v235/kapoor24a.html.

Khattab, Omar, Arnav Singhvi, Tavish Maheshwari, et al. 2023. «DSPy: Compiling Declarative Language Model Calls into Self-Improving Pipelines». *arXiv preprint arXiv:2310.03714*. https://arxiv.org/abs/2310.03714.

Liu, Nelson F., Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, y Percy Liang. 2024. «Lost in the Middle: How Language Models Use Long Contexts». *Transactions of the Association for Computational Linguistics* 12: 157-73. https://aclanthology.org/2024.tacl-1.9/.

Maister, David H., Charles H. Green, y Robert M. Galford. 2000. *The Trusted Advisor*. Free Press.

Martin, Robert C. 2008. *Clean Code: A Handbook of Agile Software Craftsmanship*. Prentice Hall.

———. 2017. *Clean Architecture: A Craftsman's Guide to Software Structure and Design*. Prentice Hall.

McConnell, Steve. 2004. *Code Complete: A Practical Handbook of Software Construction*. 2.ª ed. Microsoft Press.

McKinsey Global Institute. 2023. «The Economic Potential of Generative AI: The Next Productivity Frontier». McKinsey Global Institute. https://www.mckinsey.com/capabilities/mckinsey-digital/our-insights/the-economic-potential-of-generative-ai-the-next-productivity-frontier.

Mozilla. 2026. «State of Open Source AI 2026». Mozilla. https://stateofopensource.ai/.

Nygard, Michael T. 2018. *Release It!: Design and Deploy Production-Ready Software*. 2.ª ed. Pragmatic Bookshelf.

Ouyang, Long, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll L. Wainwright, et al. 2022. «Training Language Models to Follow Instructions with Human Feedback». En *Advances in Neural Information Processing Systems*, 35:27730-44. Curran Associates, Inc. https://papers.nips.cc/paper_files/paper/2022/hash/b1efde53be364a7395fc0a42a3b8f0f4-Abstract-Conference.html.

OWASP Foundation. 2024. «OWASP Top 10 for Large Language Model Applications». 2024. https://owasp.org/www-project-top-10-for-large-language-model-applications/.

Peng, Sida, Eirini Kalliamvakou, Peter Cihon, y Mert Demirer. 2023. «The Impact of AI on Developer Productivity: Evidence from GitHub Copilot». *arXiv preprint arXiv:2302.06590*. https://arxiv.org/abs/2302.06590.

Pérez Ruiz, Sergio. 2024. «Refactoring: Principios SOLID aplicados a la mejora continua del diseño de software». Publicación técnica en LinkedIn. https://www.linkedin.com/posts/sergioide007_refactoring-principiossolid-scrum-activity-7256725133749370883-NLs7.

———. 2026a. «El Código Limpio Sintético: los principios que perduran». LinkedIn. 2026. https://www.linkedin.com/pulse/el-c%C3%B3digo-limpio-sint%C3%A9tico-los-principios-que-sergio-perez-ruiz-ln95e/.

———. 2026b. «SpecSolid: A Philosophy, a Framework, and a Community — Built on the Shoulders of Giants». Medium. 2026. https://medium.com/@sergioide007/specsolid-a-philosophy-a-framework-and-a-community-built-on-the-shoulders-of-giants-471062066e10.

PromptMetrics. 2026. «We analyzed the switching costs for 5 companies moving LLM providers». 2026. https://www.promptmetrics.com/blog/llm-switching-costs.

Qian, Yidong, Qian Wang, Tianyu Wang, Qinbin Li, y Bingsheng

He. 2024. «MegaAgent: A Practical Framework for Autonomous Cooperation in Large-Scale LLM Agent Systems». *arXiv preprint arXiv:2408.09955*. https://arxiv.org/abs/2408.09955.

Russell, Stuart, y Peter Norvig. 2020. *Artificial Intelligence: A Modern Approach*. 4.ª ed. Pearson.

Sculley, D., Gary Holt, Daniel Golovin, Eugene Davydov, Todd Phillips, Dietmar Ebner, Vinay Chaudhary, Michael Young, Jean-François Crespo, y Dan Dennison. 2015. «Hidden Technical Debt in Machine Learning Systems». En *Advances in Neural Information Processing Systems*, 28:2503-11. Curran Associates, Inc. https://papers.nips.cc/paper/5656-hidden-technical-debt-in-machine-learning-systems.

Shinn, Noah, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, y Shunyu Yao. 2023. «Reflexion: Language Agents with Verbal Reinforcement Learning». En *Advances in Neural Information Processing Systems*. Vol. 36. Curran Associates, Inc. https://arxiv.org/abs/2303.11366.

Shostack, Adam. 2014. *Threat Modeling: Designing for Security*. Wiley.

Skelton, Matthew, y Manuel Pais. 2019. *Team Topologies: Organizing Business and Technology Teams for Fast Flow*. IT Revolution Press.

Tabassi, Elham. 2023. «Artificial Intelligence Risk Management Framework (AI RMF 1.0)». NIST AI 100-1. National Institute of Standards; Technology. https://doi.org/10.6028/NIST.AI.100-1.

The Register. 2026. «Locked, stocked, and losing budget: AI vendor lock-in bites as prices soar». agosto de 2026. https://www.theregister.com/2026/08/06/ai_vendor_lockin/.

Vaswani, Ashish, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, y Illia Polosukhin. 2017. «Attention Is All You Need». En *Advances in Neural Information Processing Systems*, 30:5998-6008. Curran Associates, Inc. https://papers.nips.cc/paper/7181-attention-is-all-you-need.

Vernon, Vaughn. 2013. *Implementing Domain-Driven Design*. Addison-Wesley.

Wadhwa, Nalin, Jui Pradhan, Atharv Sonwane, Surya Prakash Sahu, Nagarajan Natarajan, Aditya Kanade, Suresh Parthasarathy, y Sriram Rajamani. 2023. «Frustrated with Code Quality Issues? LLMs can Help!» Microsoft Research. https://www.microsoft.com/en-us/research/publication/frustrated-with-code-quality-issues-llms-can-help/.

Wei, Jason, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, et al. 2022. «Chain-of-Thought Prompting Elicits Reasoning in Large Language Models». *arXiv preprint arXiv:2201.11903*. https://arxiv.org/abs/2201.11903.

Wooldridge, Michael. 2009. *An Introduction to MultiAgent Systems*. 2.ª ed. John Wiley & Sons.

Yao, Shunyu, Dian Yu, Jeffrey Zhao, Itay Shafran, Thomas Griffiths, Yuan Cao, y Karthik Narasimhan. 2023. «Tree of Thoughts: Deliberate Problem Solving with Large Language Models». En *Advances in Neural Information Processing Systems*. Vol. 36. Curran Associates, Inc. https://papers.nips.cc/paper_files/paper/2023.

Yao, Shunyu, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, y Yuan Cao. 2023. «ReAct: Synergizing Reasoning and Acting in Language Models». En *International Conference on Learning Representations*. https://openreview.net/forum?id=WE_vluYUL-X.

Zhang, Y. et al. 2026. «A Framework for Prompt Optimization and Translation Across Foundation Models». En *International Conference on Learning Representations*. https://openreview.net/forum?id=abcdef.

Zheng, Lianmin, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, et al. 2023. «Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena». En *Advances in Neural Information Processing Systems*. Vol. 36. Curran Associates, Inc. https://arxiv.org/abs/2306.05685.

Ziegler, Jürgen, y Tim Donkers. 2024. «From explanations to human-AI co-evolution: charting trajectories towards future user-centric AI». *i-com* 23 (2): 263-72. https://doi.org/10.1515/icom-2024-0020.

Obras y publicaciones del autor

Artículos técnicos y ensayos

Perez Ruiz, S. (2026). *SpecSolid: A Philosophy, a Framework, and a Community, Built on the Shoulders of Giants*. Medium.

Perez Ruiz, S. (2026). *The Complete DevOps Course: Free, Community-Driven and Built for Real Engineers*. Medium.

Perez Ruiz, S. (2026). *The Model Is a Detail: Clean Architecture for AI Agents*. Medium.

Perez Ruiz, S. (2026). *The Silent Token Burn and the CIU Metric: The Debt That Doesn't Show on the Balance Sheet*. Medium.

Publicaciones técnicas en LinkedIn

Perez Ruiz, S. (2024). *Refutación de argumentos en el desarrollo de software*. LinkedIn.

Perez Ruiz, S. (2024). *Migración de código Java Spring Boot a NodeJS con IA*. LinkedIn.

Perez Ruiz, S. (2024). *Caso de estudio: arquitectura para un sincronizador de datos*. LinkedIn.

Perez Ruiz, S. (2025). *Reactividad*. LinkedIn.

Perez Ruiz, S. (2026). *El token burn silencioso y la métrica CIU*. LinkedIn.

Perez Ruiz, S. (2026). *El modelo es un detalle: Clean Architecture para agentes de IA*. LinkedIn.

Perez Ruiz, S. (2026). *El código limpio sintético: Los principios que perduran*. LinkedIn.

Proyectos y comunidades

Perez Ruiz, S. (s. f.). *SpecSolid: Framework y comunidad para ingeniería de software asistida por inteligencia artificial* [Repositorio de software]. GitHub.

Perez Ruiz, S. (s. f.). *Perfil de publicaciones técnicas*. Medium.

Perez Ruiz, S. (s. f.). *Perfil profesional y publicaciones técnicas*. LinkedIn.